

ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA HỆ THỐNG THÔNG TIN



BÁO CÁO ĐỒ ÁN THỰC HÀNH MÔN HỌC
HỆ QUẢN TRỊ CƠ SỞ DỮ LIỆU

ĐỀ TÀI
XÂY DỰNG HỆ THỐNG
QUẢN LÝ GIAO DỊCH VÀ VÍ ĐIỆN TỬ

Lớp: IS210.Q21

GVHD: KS. Lê Võ Đình Kha

Nhóm sinh viên thực hiện:

- | | |
|---------------------|----------------|
| 1. Nguyễn Đăng Khôi | MSSV: 23520773 |
| 2. Hồ Như Hồng Ngọc | MSSV: 23521021 |
| 3. Võ Ngọc Anh Thy | MSSV: 23521565 |
| 4. Nguyễn Nhật Tân | MSSV: 24521580 |

TP. Hồ Chí Minh, 2026.

NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

....., ngày.....tháng.....năm 2026

Người nhận xét
(Ký tên và ghi rõ họ tên)

MỤC LỤC

CHƯƠNG 1: TỔNG QUAN ĐỀ TÀI.....	9
1.1. Giới thiệu bài toán	9
1.1.1. Lý do chọn đề tài	9
1.1.2. Phát biểu bài toán	9
1.1.3. Mục tiêu đề tài	10
1.2. Xác định yêu cầu và phân tích yêu cầu	10
1.2.1. Yêu cầu chức năng	10
1.2.2. Yêu cầu phi chức năng	11
CHƯƠNG 2: THIẾT KẾ CƠ SỞ DỮ LIỆU	12
2.1. Mô hình ERD (Thực thể kết hợp)	12
2.2. Mô hình quan hệ.....	13
2.3. Bảng thuyết minh quan hệ và thuộc tính.....	14
2.4. Mô tả ràng buộc toàn vẹn.....	20
2.4.1. Ràng buộc khóa chính	21
2.4.2. Ràng buộc khóa ngoại	24
2.4.3. Ràng buộc khác	29
CHƯƠNG 3: XÂY DỰNG VÀ QUẢN LÝ GIAO TÁC	37
3.1. Xây dựng và mô tả Trigger trong đồ án môn học	37
3.2. Xây dựng và mô tả Procedure, Function trong đồ án môn học.....	47
3.2.1. Procedure	47
3.2.2. Function	95
CHƯƠNG 4: XỬ LÝ ĐỒNG THỜI.....	112
4.1. Giao tác.....	112
4.2. Vấn đề trong môi trường truy xuất đồng thời	112
4.2.1. Lost Update (Mất dữ liệu cập nhật).....	112
4.2.2. Non-repeatable Read (Đọc không nhất quán)	113
4.2.3. Phantom Read (Đọc bóng ma)	113
4.2.4. Dirty Read (Đọc dữ liệu rác)	113
4.2.5. Deadlock.....	114
4.3. Mức cô lập.....	114

4.3.1. Read uncommitted (Đọc dữ liệu chưa xác nhận)	114
4.3.2. Read Committed (Đọc dữ liệu đã xác nhận)	114
4.3.3. Repeatable Read (Đọc lặp lại được).....	115
4.3.4. Serializable (Tuần tự hóa)	115
4.4. Các phương thức khóa cơ bản.....	115
4.4.1. Shared Lock (Khóa chia sẻ – Ký hiệu: S)	115
4.4.2. Exclusive Lock (Khóa độc quyền – Ký hiệu: X)	115
4.5. Minh họa đồ án môn học trên hệ quản trị cơ sở dữ liệu Oracle.....	116
4.5.1. Trường hợp Lost Update	116
4.5.2. Trường hợp Non-repeatable Read	119
4.5.3. Trường hợp Phantom Read.....	123
4.5.4. Trường hợp Dirty Read	127
4.5.5. Trường hợp Deadlock.....	128
CHƯƠNG 5: THIẾT KẾ GIAO DIỆN.....	131
5.1. Sơ đồ liên kết các màn hình	131
5.2. Danh sách các màn hình.....	131
5.3. Mô tả các màn hình	133
5.3.1. Màn hình Đăng ký	133
5.3.2. Màn hình Tạo mã PIN	134
5.3.3. Màn hình Đăng nhập	135
5.3.4. Màn hình Trang chủ.....	136
5.3.5. Màn hình Cài đặt	137
5.3.6. Màn hình Hồ sơ cá nhân.....	139
5.3.7. Màn hình Cập nhật hồ sơ.....	140
5.3.8. Màn hình Danh sách liên kết tài khoản	141
5.3.9. Màn hình Lịch sử giao dịch.....	142
5.3.10. Màn hình Quét mã QR	143
5.3.11. Màn hình Danh sách ưu đãi.....	144
5.3.12. Màn hình Mã QR của tôi	145
5.3.13. Màn hình Chuyển tiền	146
5.3.14. Màn hình Lựa chọn ưu đãi	147
5.3.15. Màn hình Nạp tiền	148
5.3.16. Màn hình Rút tiền	149

5.3.17. Màn hình Xác nhận giao dịch.....	150
5.3.18. Màn hình Kết quả giao dịch	151
CHƯƠNG 6: SỰ CỐ VÀ CƠ CHẾ PHỤC HỒI.....	152
6.1. Mô phỏng tình huống	152
6.2. Quá trình thực hiện.....	152
6.3. Tổng quan.....	154
CHƯƠNG 7: PHÂN TÍCH HIỆU NĂNG XỬ LÝ VÀ CẢI THIỆN.....	155
7.1. Mục tiêu.....	155
7.2. Kịch bản đặt ra và truy vấn	155
7.3. Mô tả truy vấn	160
7.4. Kết quả đo được	161
7.4.1. Đoạn mã truy vấn chưa tối ưu	161
7.4.2. Đoạn mã truy vấn đã tối ưu	162
7.5. Phân tích hiệu năng	162
7.5.1. Phân tích Thời gian đáp ứng (Response Time) & Thông lượng (Throughput).....	162
7.5.2. Phân tích Chi phí I/O (Đọc/Ghi ổ cứng) và CPU	162
7.5.3. Đánh giá trong môi trường xử lý đồng thời (Concurrency)	163
7.6. Đề xuất cách cải thiện	163
7.6.1. Tối ưu logic và cấu trúc truy vấn.....	163
7.6.2. Tối ưu Chỉ mục (Index) và Điều kiện lọc.....	163
CHƯƠNG 8: KẾT LUẬN	165
8.1. Kết quả đạt được	165
8.2. Hạn chế.....	166
8.3. Hướng phát triển.....	166
8.4. Bảng phân công công việc	167

DANH MỤC HÌNH

Hình 2-1. Mô hình thực thể mối kết hợp	12
Hình 2-2. Mô hình quan hệ	13
Hình 5-1. Sơ đồ liên kết các màn hình.....	131
Hình 5-2. Màn hình Đăng ký	133
Hình 5-3. Màn hình Tạo mã PIN	134
Hình 5-4. Màn hình Đăng nhập	135
Hình 5-5. Màn hình Trang chủ.....	136
Hình 5-6. Màn hình Cài đặt	137
Hình 5-7. Màn hình Hồ sơ cá nhân.....	139
Hình 5-8. Màn hình Cập nhật hồ sơ.....	140
Hình 5-9. Màn hình Danh sách liên kết tài khoản	141
Hình 5-10. Màn hình Lịch sử giao dịch.....	142
Hình 5-11. Màn hình Quét mã QR.....	143
Hình 5-12. Màn hình Danh sách ưu đãi	144
Hình 5-13. Màn hình Mã QR của tôi	145
Hình 5-14. Màn hình chuyển tiền	146
Hình 5-15. Màn hình Lựa chọn ưu đãi	147
Hình 5-16. Màn hình Nạp tiền	148
Hình 5-17. Màn hình Rút tiền	149
Hình 5-18. Màn hình Xác nhận giao dịch.....	150
Hình 5-19. Màn hình Kết quả giao dịch	151
Hình 6-1. Dữ liệu ban đầu.....	152
Hình 6-2. Hệ thống ORACLE khởi động lại	153
Hình 6-3. Kết quả sau khi ORACLE tự động phục hồi.....	154
Hình 7-1. Thời gian thực hiện truy vấn (ms)	161
Hình 7-2. Thời gian thực hiện truy vấn (ms)	162

DANH MỤC BẢNG

Bảng 2-1. Bảng TRANSACTIONS	15
Bảng 2-2. Bảng TRANSACTION_TYPES	15
Bảng 2-3. Bảng TRANSACTION_FEES	16
Bảng 2-4. Bảng TRANSACTION_LIMITS	16
Bảng 2-5. Bảng USERS	17
Bảng 2-6. Bảng WALLETS	17
Bảng 2-7. Bảng PAYMENT_METHODS	18
Bảng 2-8. Bảng FUND_ORDERS	19
Bảng 2-9. Bảng AUDIT_LOGS	19
Bảng 2-10. Bảng LOGIN_SESSION	19
Bảng 2-11. Bảng VOUCHERS	20
Bảng 2-12. Bảng TRANSACTION_REF_SEQUENCE	20
Bảng 5-1. Danh sách các màn hình trong ứng dụng	132
Bảng 5-2. Bảng mô tả màn hình Đăng ký	133
Bảng 5-3. Bảng mô tả màn hình Tạo mã PIN	134
Bảng 5-4. Bảng mô tả màn hình Đăng nhập	135
Bảng 5-5. Bảng mô tả màn hình Trang chủ	137
Bảng 5-6. Bảng mô tả màn hình Cài đặt	138
Bảng 5-7. Bảng mô tả màn hình Hồ sơ cá nhân	140
Bảng 5-8. Bảng mô tả màn hình Cập nhật hồ sơ	140
Bảng 5-9. Bảng mô tả màn hình Danh sách liên kết tài khoản	141
Bảng 5-10. Bảng mô tả màn hình Lịch sử giao dịch	142
Bảng 5-11. Bảng mô tả màn hình Quét mã QR	143
Bảng 5-12. Bảng mô tả màn hình Danh sách ưu đãi	144
Bảng 5-13. Bảng mô tả màn hình Mã QR của tôi	145
Bảng 5-14. Bảng mô tả màn hình Mã QR của tôi	146
Bảng 5-15. Bảng mô tả màn hình Lựa chọn ưu đãi	147
Bảng 5-16. Bảng mô tả màn hình Lựa chọn ưu đãi	148
Bảng 5-17. Bảng mô tả màn hình Rút tiền	149
Bảng 5-18. Bảng mô tả màn hình Xác nhận giao dịch	150
Bảng 5-19. Bảng mô tả màn hình Kết quả giao dịch	151

DANH MỤC TỪ VIẾT TẮT

STT	Từ viết tắt	Từ đầy đủ	Nghĩa tiếng Việt
1	ACID	Atomicity, Consistency, Isolation, Durability	Tính thuộc tính của giao dịch hệ quản trị CSDL: Tính nguyên tố, Tính nhất quán, Tính cô lập, Tính bền vững
2	ERD	Entity Relationship Diagram	Sơ đồ quan hệ - thực thể
3	KYC	Know Your Customer	Xác minh danh tính khách hàng
4	PIN	Personal Identification Number	Mã số định danh cá nhân
5	FK	Foreign Key	Khóa ngoại
6	CTE	Common Table Expressions	Biểu thức bảng tạm thời
7	PGA	Program Global Area	Vùng nhớ toàn cục chương trình

CHƯƠNG 1: TỔNG QUAN ĐỀ TÀI

1.1. Giới thiệu bài toán

1.1.1. Lý do chọn đề tài

Trong kỷ nguyên số hóa hiện nay, sự chuyển dịch mạnh mẽ từ giao dịch tiền mặt sang thanh toán điện tử đã đặt nền móng cho sự phát triển nhanh chóng của các hệ sinh thái công nghệ tài chính (Fintech). Trong đó, ví điện tử đóng vai trò là một hạ tầng tài chính thiết yếu, phục vụ hàng triệu người dùng với tần suất giao dịch liên tục mỗi ngày. Sự phụ thuộc ngày càng lớn vào các nền tảng này đòi hỏi các hệ thống lưu trữ và xử lý dữ liệu về giao dịch trực tuyến phải có độ hoàn thiện cao nhất về cả hiệu năng truy xuất lẫn tính an toàn và bảo mật thông tin.

Dưới góc độ quản trị cơ sở dữ liệu, việc xây dựng một Hệ thống quản lý giao dịch tài chính và Ví điện tử là một bài toán mang tính thực tiễn và có độ phức tạp cao. Khác với các hệ thống quản lý thông tin thông thường, dữ liệu tài chính mang tính nhạy cảm tuyệt đối, yêu cầu cơ sở dữ liệu phải tuân thủ nghiêm ngặt các thuộc tính ACID (Atomicity, Consistency, Isolation, Durability) trong mọi tình huống. Bất kỳ một sai sót nhỏ nào trong quá trình kiểm soát truy cập đồng thời (concurrency control) – chẳng hạn như lỗi mất mát trong quá trình cập nhật hay xung đột trong quá trình đọc ghi dữ liệu – đều có thể dẫn đến thất thoát tài sản thực tế và phá vỡ cấu trúc của toàn bộ hệ thống.

Xuất phát từ những thách thức thực tiễn đó, nhóm chúng tôi quyết định lựa chọn đề tài "Xây dựng Hệ thống quản lý giao dịch tài chính và Ví điện tử". Đề tài này không chỉ là cơ hội để nhóm áp dụng các lý thuyết về chuẩn hóa mô hình dữ liệu, mà còn là môi trường thực nghiệm để đi sâu vào các kỹ thuật nâng cao của Hệ quản trị Cơ sở dữ liệu. Thông qua việc tự thiết kế kiến trúc lưu trữ, lập trình nghiệp vụ lõi tại tầng dữ liệu, và xử lý các bài toán tranh chấp, nhóm mong muốn xây dựng được một nền tảng vững chắc, đáp ứng các tiêu chuẩn khắt khe của một sản phẩm Fintech thực tế.

1.1.2. Phát biểu bài toán

Bài toán đầu tiên và mang tính nền tảng của hệ thống là việc tổ chức và quản lý vòng đời của các thực thể cốt lõi bao gồm định danh người dùng và tài khoản ví điện tử. Hệ thống cần lưu trữ thông tin cá nhân một cách có cấu trúc, đồng thời phân loại rõ các trạng thái xác thực danh tính (KYC Status: Pending, Verified, Rejected) để cấp quyền giao dịch tương ứng nhằm tuân thủ các quy định tài chính. Bên cạnh đó, mỗi người dùng có thể sở hữu tài khoản ví với các trạng thái hoạt động khác nhau (Active, Frozen, Closed) và cần được liên kết an toàn với các phương thức thanh toán ngoại vi (tài khoản ngân hàng, thẻ tín dụng) để làm cơ sở cho dòng tiền ra và vào hệ thống.

Bài toán thứ hai tập trung vào việc xử lý linh hoạt các nghiệp vụ tài chính và quản lý hệ thống biểu phí phức tạp. Một ví điện tử tiêu chuẩn phải hỗ trợ đa dạng các loại hình giao dịch (Transaction Types) như nạp tiền, rút tiền, chuyển khoản nội bộ và thanh toán dịch vụ. Tương ứng với mỗi loại hình, kiến trúc dữ liệu phải đủ linh hoạt để tự động tính toán phí giao dịch dựa trên các bộ quy tắc được quy định sẵn (Transaction Fees). Các quy tắc này phải xử lý được tỷ lệ phần trăm, mức phí cố định, giới hạn phí tối thiểu/tối đa và có

mốc thời gian áp dụng hiệu lực (Effective Dates) rõ ràng, đảm bảo tính minh bạch tuyệt đối trong quá trình hạch toán.

Bài toán thứ ba, cũng là khía cạnh quan trọng nhất, yêu cầu bảo vệ tính toàn vẹn và khả năng truy vết của dòng tiền. Trong các luồng tương tác với công thanh toán hay ngân hàng, hệ thống phải thiết lập cơ chế kiểm soát tính lũy đẳng (Idempotency) dựa trên các mã tham chiếu duy nhất để ngăn chặn rủi ro nạp hoặc trừ tiền nhiều lần do độ trễ mạng. Hơn thế nữa, mọi thay đổi dù là nhỏ nhất trên số dư ví đều phải được lưu vết tự động (Audit Trail). Dữ liệu lưu vết phải thể hiện rõ trạng thái số dư trước - sau giao dịch, và giá trị biến động, tạo thành một sổ đối soát độc lập, phục vụ cho công tác kiểm toán và giải quyết khiếu nại.

1.1.3. Mục tiêu đề tài

Mục tiêu tiên quyết của đề tài là phân tích, thiết kế và chuẩn hóa thành công mô hình cơ sở dữ liệu cho hệ thống ví điện tử. Nhóm sẽ tiến hành xây dựng Mô hình Thực thể - Kết hợp (ERD) bao quát toàn bộ các quy trình nghiệp vụ và chuyển đổi sang mô hình dữ liệu vật lý (Physical Model) đạt các dạng chuẩn. Tại tầng vật lý này, các ràng buộc toàn vẹn (Integrity Constraints) sẽ được thiết lập chặt chẽ – từ khóa chính, khóa ngoại cho đến các điều kiện kiểm tra (Check constraints) như đảm bảo số dư ví không bao giờ bị âm, hoặc các mã giao dịch đối soát từ hệ thống bên ngoài phải luôn là duy nhất.

Mục tiêu thứ hai là áp dụng được các logic nghiệp vụ lõi xuống tầng cơ sở dữ liệu của Hệ quản trị. Nhóm sẽ tập trung lập trình các thủ tục lưu trữ (Stored Procedures) để đóng gói toàn bộ quy trình của một giao dịch (từ bước kiểm tra điều kiện, tính phí, đến cập nhật số dư các bên liên quan) thành một khối thống nhất không thể chia cắt. Đồng thời, nhóm sẽ triển khai hệ thống Triggers để tự động kiểm tra các sự kiện thay đổi số dư trong ví, từ đó chèn dữ liệu vào bảng lưu vết (Audit Logs) một cách bắt buộc, ngăn chặn mọi nỗ lực can thiệp thay đổi số dư mà không để lại lịch sử.

Mục tiêu cuối cùng là giải quyết bài toán xử lý đồng thời (Concurrency Control) và mô phỏng được tính đúng đắn của hệ thống. Nhóm sẽ nghiên cứu, cấu hình các cơ chế quản lý giao dịch (Transaction Management), thiết lập các mức độ cô lập (Isolation Levels) và kỹ thuật khóa (Locking) phù hợp cho từng loại nghiệp vụ. Việc này nhằm giải quyết triệt để tình trạng tranh chấp dữ liệu khi có nhiều luồng cùng yêu cầu trừ hoặc cộng tiền trên một tài khoản ở cùng một thời điểm. Kết quả của quá trình này sẽ được thể hiện thông qua một bộ dữ liệu giả lập và các kịch bản truy vấn kiểm thử, khẳng định sự ổn định của kiến trúc đã xây dựng.

1.2. Xác định yêu cầu và phân tích yêu cầu

1.2.1. Yêu cầu chức năng

1.2.1.1. Quản lý người dùng

- Tạo mới, chỉnh sửa và quản lý thông tin người dùng.
- Xác thực và đảm bảo tính duy nhất của thông tin cá nhân khách hàng như tên tài khoản, số điện thoại, email.
- Người dùng có thể cập nhật thông tin cá nhân và thay đổi mật khẩu, mã PIN.

1.2.1.2. Quản lý tài khoản

- Tạo mới ví điện tử và liên kết ví với tài khoản người dùng.
- Quản lý số dư và cập nhật số dư thông qua các giao dịch được thực hiện.
- Hỗ trợ liên kết ví với tài khoản ngân hàng hoặc thẻ thanh toán.
- Quản lý hạn mức giao dịch của từng ví theo quy định của hệ thống.

1.2.1.3. Quản lý giao dịch

- Hỗ trợ người dùng thực hiện các giao dịch như nạp tiền, rút tiền, chuyển tiền, thanh toán.
- Ghi nhận đầy đủ thông tin giao dịch (thời gian, số tiền, loại giao dịch và trạng thái giao dịch).
- Quản lý trạng thái giao dịch (Đang xử lý, hoàn thành, thất bại, hủy bỏ).
- Cho phép người dùng tra cứu lịch sử giao dịch của mình.
- Hệ thống phải kiểm tra số dư và tính hợp lệ trước khi thực hiện giao dịch.

1.2.2. Yêu cầu phi chức năng

1.2.2.1. Vận hành

- Hỗ trợ triển khai trên nền tảng thiết bị di động (Android).
- Hệ thống phải tích hợp với các cổng thanh toán và ngân hàng trong và ngoài nước như MoMo, ZaloPay, Visa, Mastercard và hệ thống ngân hàng nội địa.
- Giao diện hệ thống phải trực quan, thân thiện với người dùng, dễ sử dụng và tương tác.

1.2.2.2. Hiệu suất

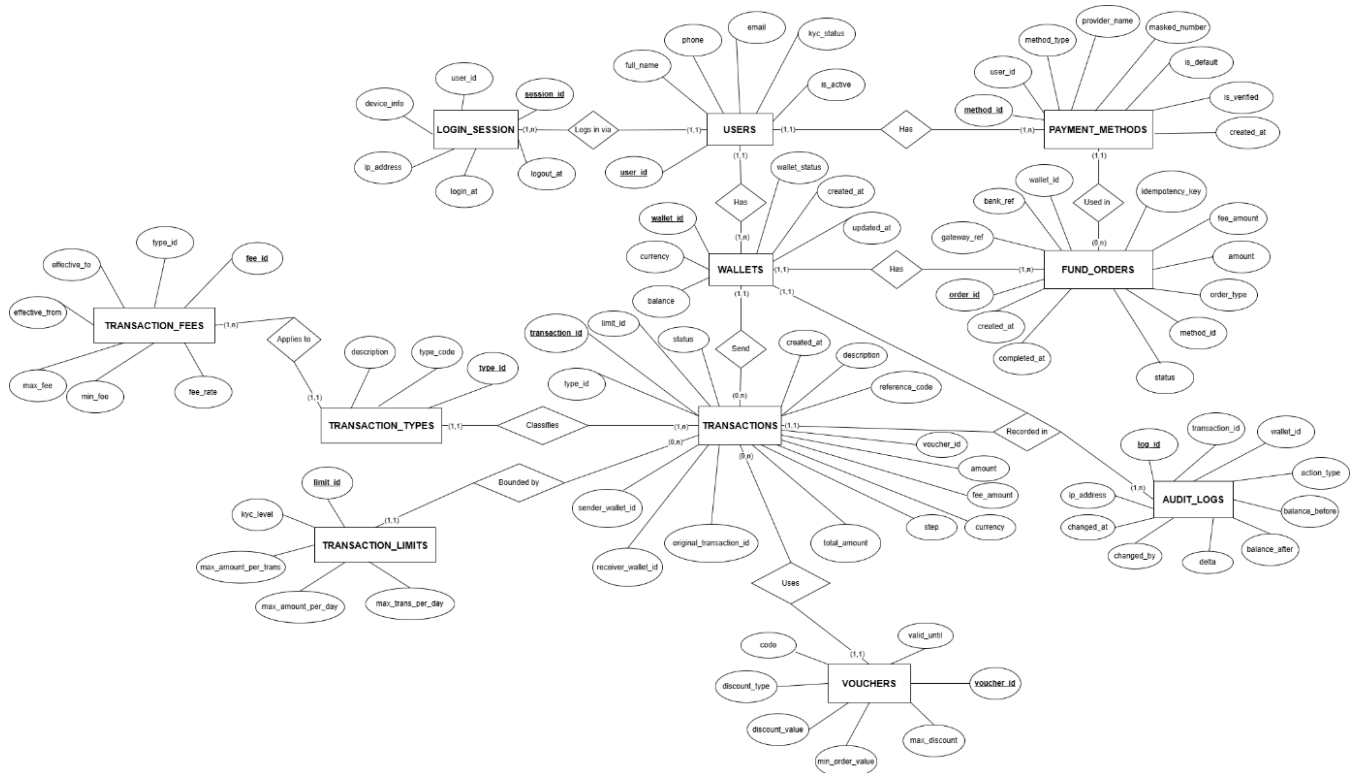
- Hệ thống phải đảm bảo xử lý giao dịch theo thời gian thực và cập nhật số dư ngay sau khi giao dịch hoàn tất.
- Hệ thống phải đảm bảo tính nhất quán dữ liệu trong các giao dịch tài chính.

1.2.2.3. Bảo mật

- Mỗi người dùng chỉ có thể truy cập và quản lý thông tin cá nhân của chính mình, không thể truy cập dữ liệu của người khác.
- Hệ thống phải ghi nhận log các hoạt động và giao dịch để phục vụ kiểm tra và truy vết.
- Áp dụng cơ chế kiểm soát truy cập và phân quyền theo vai trò (người dùng, quản trị viên, hệ thống).

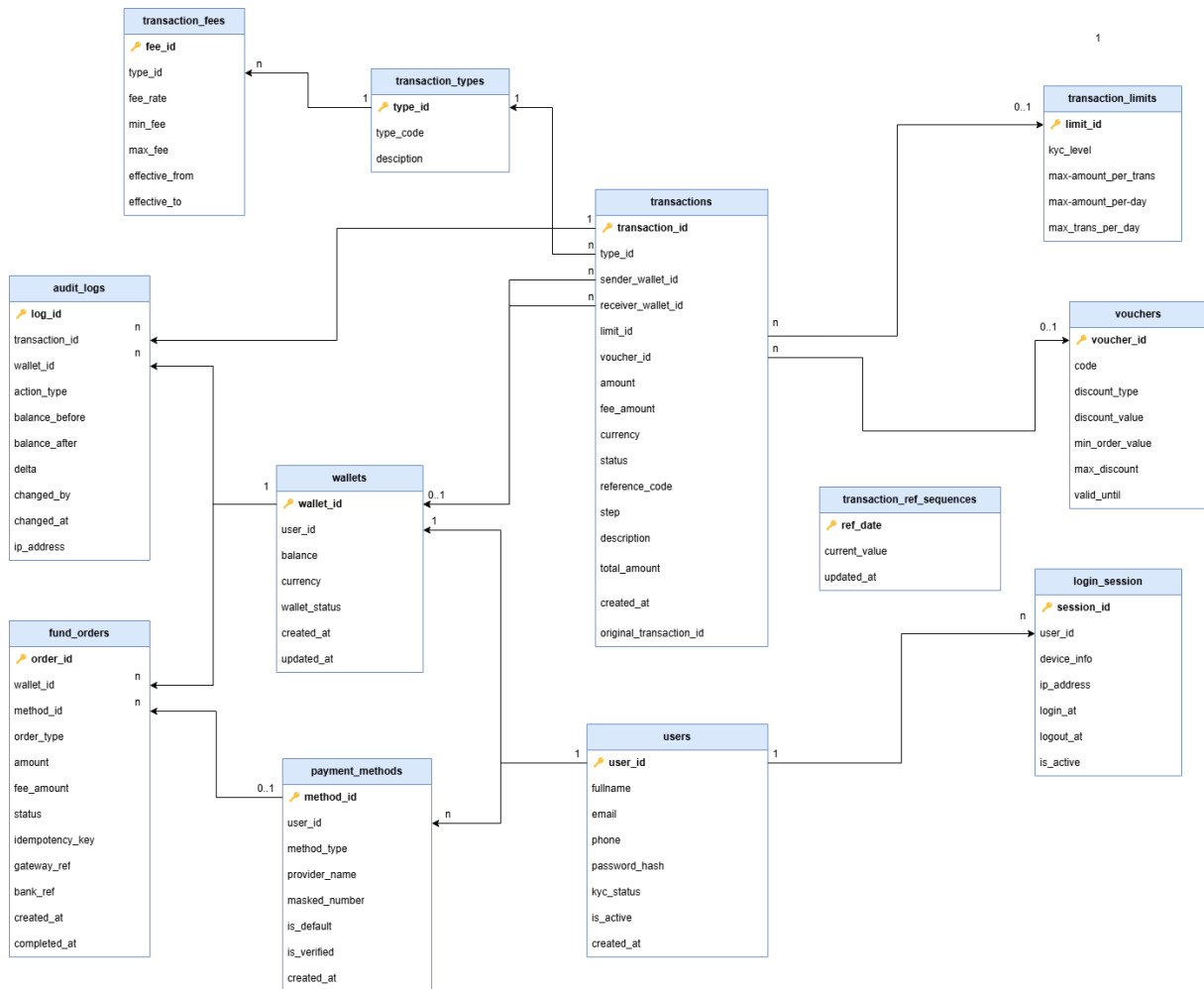
CHƯƠNG 2: THIẾT KẾ CƠ SỞ DỮ LIỆU

2.1. Mô hình ERD (Thực thể kết hợp)



Hình 2-1. Mô hình thực thể mối kết hợp

2.2. Mô hình quan hệ



Hình 2-2. Mô hình quan hệ

USERS (user id, full_name, email, phone, password_hash, kyc_status, is_active, created_at, updated_at)

WALLETS (wallet id, user_id, balance, currency, wallet_status, created_at, updated_at, pin_code)

PAYMENT_METHODS (method id, user_id, method_type, provider_name, masked_number, is_default, is_verified, created_at, updated_at)

LOGIN_SESSIONS (session id, user_id, device_info, ip_address, is_active)

TRANSACTION_TYPES (type id, type_code, description)

TRANSACTION_LIMITS (limit id, kyc_level, max_amount_per_trans, max_amount_per_day, max_trans_per_day)

TRANSACTION_FEES (fee id, type_id, fee_rate, fee_fixed, min_fee, max_fee, effective_from, effective_to)

TRANSACTIONS (transaction_id, type_id, sender_wallet_id, receiver_wallet_id, limit_id, voucher_id, original_transaction_id, amount, fee_amount, status, reference_code, step, total_amount, created_at, description)

FUND_ORDERS (order_id, wallet_id, method_id, order_type, amount, status, idempotency_key, gateway_ref, created_at, updated_at)

AUDIT_LOGS (log_id, transaction_id, wallet_id, action_type, balance_before, balance_after, delta)

VOUCHERS (voucher_id, code, discount_type, discount_value, valid_until, max_discount, min_order_value, amount_vouchers, is_active, created_at, updated_at)

TRANSACTION_REF_SEQUENCE (ref_date, current_value, updated_at)

2.3. Bảng thuyết minh quan hệ và thuộc tính

- **Bảng TRANSACTIONS**

STT	Tên thuộc tính	Kiểu dữ liệu	Quan hệ	Diễn giải
1	transaction_id	Number	Khóa chính	Mã định danh giao dịch.
2	type_id	Number (5)	FK (transaction_types)	Loại giao dịch thực hiện.
3	sender_wallet_id	Number	FK (wallets)	ID ví gửi tiền (có thể null nếu là nạp tiền).
4	receiver_wallet_id	Number	FK (wallets)	ID ví nhận tiền (có thể null nếu là rút tiền).
5	limit_id	Number (10)	FK (transaction_limits)	Liên kết với hạn mức áp dụng.
6	voucher_id	Number	FK (vouchers)	Mã giảm giá được áp dụng (nếu có).
7	amount	Number (18,4)	-	Số tiền gốc của giao dịch.
8	fee_amount	Number (18,4)	-	Số tiền phí thực tế đã thu.
9	status	Varchar2(20)	-	Trạng thái (PENDING / COMPLETED / FAILED...).
10	reference_code	Varchar(250)	Unique	Mã đối soát duy nhất cho giao dịch.

11	step	Number (10)	-	Giờ thực hiện trong ngày (phục vụ thống kê).
12	total_amount	Number(18,4)	-	Tổng tiền của giao dịch.
13	created_at	Timestamp	-	Thời gian tạo giao dịch.
14	original_transaction_id	Number	FK (transactions)	Giao dịch gốc, dùng cho giao dịch REVERSAL hoặc REFUND
15	description	Varchar2(500)		Mô tả nội dung giao dịch.

Bảng 2-1. Bảng TRANSACTIONS

- Bảng TRANSACTION_TYPES**

STT	Tên thuộc tính	Kiểu dữ liệu	Quan hệ	Diễn giải
1	type_id	Number 5)	Khóa chính	Mã loại giao dịch.
2	type_code	Varchar2(20)	Unique	Mã code (TOP_UP, TRANSFER, PAYMENT...).
3	description	Varchar2(255)	-	Mô tả chi tiết ý nghĩa loại giao dịch.

Bảng 2-2. Bảng TRANSACTION_TYPES

- Bảng TRANSACTION_FEES**

STT	Tên thuộc tính	Kiểu dữ liệu	Quan hệ	Diễn giải
1	fee_id	Number(10)	Khóa chính	Mã định danh quy tắc phí.
2	type_id	Number	FK (trans_types)	Áp dụng phí cho loại giao dịch nào.
3	fee_rate	Number (5,4)	-	Tỷ lệ phí theo phần trăm (ví dụ 0.005 = 0.5%).
4	fee_fixed	Number (18,4)	-	Mức phí cố định cộng thêm.
5	min_fee	Number 18,4)	-	Mức phí tối thiểu phải thu.
6	max_fee	Number (18,4)	-	Mức phí tối đa (trần phí).

7	effective_from	Date	-	Ngày bắt đầu áp dụng mức phí này.
8	effective_to	Date	-	Ngày kết thúc áp dụng (nếu có).

Bảng 2-3. Bảng TRANSACTION_FEES

- **Bảng TRANSACTION_LIMITS**

STT	Tên thuộc tính	Kiểu dữ liệu	Quan hệ	Diễn giải
1	limit_id	Number(10)	Khóa chính	Mã định danh hạn mức.
2	kyc_level	Varchar2(20)	-	Cấp độ KYC áp dụng (UNVERIFIED / VERIFIED).
3	max_amount_per_trans	Number (18,4)	-	Số tiền tối đa cho một giao dịch.
4	max_amount_per_day	Number (18,4)	-	Tổng số tiền tối đa trong một ngày.
5	max_trans_per_day	Number(10)	-	Số lượng giao dịch tối đa trong một ngày.

Bảng 2-4. Bảng TRANSACTION_LIMITS

- **Bảng USERS**

STT	Tên thuộc tính	Kiểu dữ liệu	Quan hệ	Diễn giải
1	user_id	Number	Khóa chính	Mã định danh duy nhất của người dùng.
2	full_name	Varchar2(100)	-	Họ và tên đầy đủ của người dùng.
3	email	Varchar2(150)	Unique	Địa chỉ email (duy nhất) để liên lạc/đăng nhập.
4	phone	Varchar2(15)	Unique	Số điện thoại (duy nhất) của người dùng.
5	password_hash	Varchar2(255)	-	Mật khẩu đã được mã hóa.
6	kyc_status	Varchar2(220)	-	Trạng thái định danh (PENDING / VERIFIED / REJECTED).

7	is_active	Number(1)	-	Trạng thái hoạt động của tài khoản.
8	created_at	timestamp	-	Thời điểm tạo tài khoản.
9	updated_at	timestamp	-	Thời điểm cập nhật gần nhất của tài khoản.

Bảng 2-5. Bảng USERS

- Bảng WALLETS**

STT	Tên thuộc tính	Kiểu dữ liệu	Quan hệ	Diễn giải
1	wallet_id	Number	Khóa chính	Mã định danh duy nhất của ví.
2	user_id	Number	FK (users)	Liên kết 1-1 với người dùng.
3	balance	Number (18,4)	-	Số dư hiện có trong ví.
4	currency	Varchar2(10)	-	Loại tiền tệ sử dụng (mặc định VND).
5	wallet_status	Varchar2(20)	-	Trạng thái ví (ACTIVE / FROZEN / CLOSED).
6	created_at	timestamp	-	Thời điểm khởi tạo ví.
7	updated_at	timestamp	-	Thời điểm cập nhật số dư gần nhất.
8	pin_code	Varchar(6)	-	Mã PIN giao dịch gồm 6 số

Bảng 2-6. Bảng WALLETS

- Bảng PAYMENT_METHODS**

STT	Tên thuộc tính	Kiểu dữ liệu	Quan hệ	Diễn giải
1	method_id	Number	Khóa chính	Mã phương thức thanh toán.
2	user_id	Number	FK (users)	Chủ sở hữu phương thức thanh toán.
3	method_type	Varchar2(20)	-	Loại (BANK_ACCOUNT / CREDIT_CARD...).

4	provider_name	Varchar2(100)	-	Tên ngân hàng hoặc tổ chức thẻ (VD: VCB, VISA).
5	masked_number	Varchar2(20)	-	Số tài khoản/thẻ đã che (chỉ hiện 4 số cuối).
6	is_default	Number(1)	-	Có phải phương thức mặc định không.
7	is_verified	Number(1)	-	Trạng thái đã xác thực với ngân hàng chưa.
8	created_at	Timestamp	-	Thời điểm tạo phương thức thanh toán.
9	updated_at	Timestamp	-	Thời điểm cập nhật gần nhất.

Bảng 2-7. Bảng PAYMENT_METHODS

- **Bảng FUND_ORDERS**

STT	Tên thuộc tính	Kiểu dữ liệu	Quan hệ	Diễn giải
1	order_id	Number	Khóa chính	Mã lệnh nạp/rút.
2	wallet_id	Number	FK (wallets)	Ví thực hiện lệnh.
3	method_id	Number	FK (pay methods)	Phương thức thanh toán sử dụng.
4	order_type	Varchar2(20)	-	Loại lệnh (TOP_UP / WITHDRAW).
5	amount	Number (18,4)	-	Số tiền thực hiện.
6	status	Varchar2(20)	-	Trạng thái (PENDING / SUCCESS / FAILED).
7	idempotency_key	Varchar2(100)	Unique	Khóa duy nhất để tránh nạp tiền trùng lặp.
8	gateway_ref	Varchar2(100)	-	Mã tham chiếu từ phía Cổng thanh toán.
8	created_at	Timestamp	-	Thời điểm tạo lệnh.
9	updated_at	Timestamp	-	Thời điểm cập nhật gần nhất của lệnh.

Bảng 2-8. Bảng FUND_ORDERS

- **Bảng AUDIT_LOGS**

STT	Tên thuộc tính	Kiểu dữ liệu	Quan hệ	Diễn giải
1	log_id	Number	Khóa chính	Mã nhật ký.
2	transaction_id	Number	FK (transactions)	Liên kết với giao dịch phát sinh.
3	wallet_id	Number	FK (wallets)	Ví bị biến động số dư.
4	action_type	Varchar2(10)	-	Loại biến động (DEBIT - Trừ tiền, CREDIT - Cộng tiền).
5	balance_before	Number (18,4)	-	Số dư trước khi thực hiện giao dịch.
6	balance_after	Number (18,4)	-	Số dư sau khi thực hiện giao dịch.
7	delta	Number 18,4)	-	Số tiền thay đổi thực tế.

Bảng 2-9. Bảng AUDIT_LOGS

- **Bảng LOGIN_SESSIONS**

STT	Tên thuộc tính	Kiểu dữ liệu	Quan hệ	Diễn giải
1	session_id	Number	Khóa chính	Mã phiên làm việc.
2	user_id	Number	FK (users)	Người dùng sở hữu phiên.
3	device_info	Varchar2(255)	-	Thông tin thiết bị (Trình duyệt, Hệ điều hành).
4	ip_address	Varchar2(45)	-	Địa chỉ IP đăng nhập.
5	is_active	Number(1)	-	Phiên còn hiệu lực hay không.

Bảng 2-10. Bảng LOGIN_SESSION

- **Bảng VOUCHERS**

STT	Tên thuộc tính	Kiểu dữ liệu	Quan hệ	Diễn giải
1	voucher_id	Number	Khóa chính	Mã định danh voucher.

2	code	varchar(20)	Unique	Mã code khách hàng nhập (VD: GIAM20K).
3	discount_type	varchar(20)	-	Loại giảm giá (FIXED - Số tiền, PERCENTAGE - %).
4	discount_value	Number (18,4)	-	Giá trị giảm giá.
5	valid_until	Timestamp	-	Thời hạn sử dụng mã.
6	max_discount	Number(10,4)	-	Mức giảm tối đa khi voucher tính theo phần trăm.
7	min_order_value	Number(18,4)	-	Giá trị giao dịch tối thiểu để áp dụng voucher.
8	amount_vouchers	Number(10)	-	Số lượng voucher còn lại.
9	is_active	Number(1)	-	Trạng thái voucher: 1 là còn hoạt động, 0 là ngừng hoạt động.
10	created_at	Timestamp	-	Thời điểm tạo voucher.
11	updated_at	Timestamp	-	Thời điểm cập nhật gần nhất của voucher.

Bảng 2-11. Bảng VOUCHERS

- **Bảng TRANSACTION_REF_SEQUENCE**

STT	Tên thuộc tính	Kiểu dữ liệu	Quan hệ	Diễn giải
1	ref_date	Timestamp	Khóa chính	Ngày sinh mã giao dịch
2	current_value	Varchar2(20)	Unique	Số thứ tự giao dịch hiện tại trong ngày
3	updated_at	Varchar2(20)	-	Ngày cập nhật giao dịch

Bảng 2-12. Bảng TRANSACTION_REF_SEQUENCE

2.4. Mô tả ràng buộc toàn vẹn

Dưới đây là mô tả chi tiết toàn bộ ràng buộc khóa chính, khóa ngoại và các ràng buộc UNIQUE của cơ sở dữ liệu hệ thống quản lý giao dịch ví điện tử. Các ràng buộc được thiết kế đảm bảo toàn vẹn thực thể, toàn vẹn tham chiếu và toàn vẹn miền theo yêu cầu của hệ quản trị cơ sở dữ liệu quan hệ.

2.4.1. Ràng buộc khóa chính

Ràng buộc 1: Thuộc tính `transaction_id` trong bảng `TRANSACTIONS` là duy nhất.

- Bối cảnh: `TRANSACTIONS`
- Nội dung: $\forall t1, t2 \in \text{TRANSACTIONS}: t1.transaction_id \neq t2.transaction_id$
- Bảng tầm ảnh hưởng:

	Thêm	Xóa	Sửa
TRANSACTIONS	+	-	+(transaction_id)

Ràng buộc 2: Thuộc tính `type_id` trong bảng `TRANSACTION_TYPES` là duy nhất.

- Bối cảnh: `TRANSACTION_TYPES`
- Nội dung: $\forall tt1, tt2 \in \text{TRANSACTION_TYPES}: tt1.type_id \neq tt2.type_id$
- Bảng tầm ảnh hưởng:

	Thêm	Xóa	Sửa
TRANSACTION_TYPES	+	-	+(type_id)

Ràng buộc 3: Thuộc tính `fee_id` trong bảng `TRANSACTION_FEES` là duy nhất.

- Bối cảnh: `TRANSACTION_FEES`
- Nội dung: $\forall f1, f2 \in \text{TRANSACTION_FEES}: f1.fee_id \neq f2.fee_id$
- Bảng tầm ảnh hưởng:

	Thêm	Xóa	Sửa
TRANSACTION_FEES	+	-	+(fee_id)

Ràng buộc 4: Thuộc tính `limit_id` trong bảng `TRANSACTION_LIMITS` là duy nhất.

- Bối cảnh: `TRANSACTION_LIMITS`
- Nội dung: $\forall l1, l2 \in \text{TRANSACTION_LIMITS}: l1.limit_id \neq l2.limit_id$
- Bảng tầm ảnh hưởng:

	Thêm	Xóa	Sửa
--	------	-----	-----

TRANSACTION_LIMITS	+	-	+(limit_id)
--------------------	---	---	-------------

Ràng buộc 5: Thuộc tính user_id trong bảng USERS là duy nhất.

- Bối cảnh: USERS
- Nội dung: $\forall u1, u2 \in \text{USERS}: u1.\text{user_id} \neq u2.\text{user_id}$
- Bảng tầm ảnh hưởng:

	Thêm	Xóa	Sửa
USERS	+	-	+(user_id)

Ràng buộc 6: Thuộc tính wallet_id trong bảng WALLETS là duy nhất.

- Bối cảnh: WALLETS
- Nội dung: $\forall w1, w2 \in \text{WALLETS}: w1.\text{wallet_id} \neq w2.\text{wallet_id}$
- Bảng tầm ảnh hưởng:

	Thêm	Xóa	Sửa
WALLETS	+	-	+(wallet_id)

Ràng buộc 7: Thuộc tính method_id trong bảng PAYMENT_METHODS là duy nhất.

- Bối cảnh: PAYMENT_METHODS
- Nội dung: $\forall pm1, pm2 \in \text{PAYMENT_METHODS}: pm1.\text{method_id} \neq pm2.\text{method_id}$
- Bảng tầm ảnh hưởng:

	Thêm	Xóa	Sửa
PAYMENT_METHODS	+	-	+(method_id)

Ràng buộc 8: Thuộc tính flag_id trong bảng FRAUD_FLAGS là duy nhất.

- Bối cảnh: FRAUD_FLAGS
- Nội dung: $\forall ff1, ff2 \in \text{FRAUD_FLAGS}: ff1.\text{flag_id} \neq ff2.\text{flag_id}$
- Bảng tầm ảnh hưởng:

	Thêm	Xóa	Sửa
FRAUD_FLAGS	+	-	+(flag_id)

Ràng buộc 9: Thuộc tính rule_id trong bảng FRAUD_RULES là duy nhất.

- Bối cảnh: FRAUD_RULES
- Nội dung: $\forall fr1, fr2 \in \text{FRAUD_RULES}: fr1.rule_id \neq fr2.rule_id$
- Bảng tầm ảnh hưởng:

	Thêm	Xóa	Sửa
FRAUD_RULES	+	-	+(rule_id)

Ràng buộc 10: Thuộc tính order_id trong bảng FUND_ORDERS là duy nhất.

- Bối cảnh: FUND_ORDERS
- Nội dung: $\forall fo1, fo2 \in \text{FUND_ORDERS}: fo1.order_id \neq fo2.order_id$
- Bảng tầm ảnh hưởng:

	Thêm	Xóa	Sửa
FUND_ORDERS	+	-	+(order_id)

Ràng buộc 11: Thuộc tính log_id trong bảng AUDIT_LOGS là duy nhất.

- Bối cảnh: AUDIT_LOGS
- Nội dung: $\forall al1, al2 \in \text{AUDIT_LOGS}: al1.log_id \neq al2.log_id$
- Bảng tầm ảnh hưởng:

	Thêm	Xóa	Sửa
AUDIT_LOGS	+	-	+(log_id)

Ràng buộc 12: Thuộc tính session_id trong bảng LOGIN_SESSIONS là duy nhất.

- Bối cảnh: LOGIN_SESSIONS
- Nội dung: $\forall ls1, ls2 \in \text{LOGIN_SESSIONS}: ls1.session_id \neq ls2.session_id$
- Bảng tầm ảnh hưởng:

	Thêm	Xóa	Sửa
LOGIN_SESSIONS	+	-	+(session_id)

Ràng buộc 13: Thuộc tính voucher_id trong bảng VOUCHERS là duy nhất.

- Bối cảnh: VOUCHERS
- Nội dung: $\forall v1, v2 \in \text{VOUCHERS}: v1.\text{voucher_id} \neq v2.\text{voucher_id}$
- Bảng tầm ảnh hưởng:

	Thêm	Xóa	Sửa
VOUCHERS	+	-	+(voucher_id)

2.4.2. Ràng buộc khóa ngoại

Ràng buộc 14: Mỗi giao dịch thuộc một loại giao dịch cụ thể.

- Bối cảnh: TRANSACTIONS
- Nội dung: $\forall t \in \text{TRANSACTIONS}, \exists tt \in \text{TRANSACTION_TYPES}: t.\text{type_id} = tt.\text{type_id}$
- Bảng tầm ảnh hưởng:

	Thêm	Xóa	Sửa
TRANSACTIONS	+	-	+(type_id)
TRANSACTION_TYPES	-	+	-

Ràng buộc 15: Mỗi giao dịch (nếu có) được liên kết với ví gửi.

- Bối cảnh: TRANSACTIONS
- Nội dung: $\forall t \in \text{TRANSACTIONS}, (\exists sw \in \text{WALLETS}: t.\text{sender_wallet_id} = sw.\text{wallet_id}) \vee (t.\text{sender_wallet_id} \text{ IS NULL})$
- Bảng tầm ảnh hưởng:

	Thêm	Xóa	Sửa
TRANSACTIONS	+	-	+(sender_wallet_id)

WALLETS	-	+	-

Ràng buộc 16: Mỗi giao dịch (nếu có) được liên kết với ví nhận.

- Bối cảnh: TRANSACTIONS
- Nội dung: $\forall t \in \text{TRANSACTIONS}, (\exists rw \in \text{WALLETS}: t.\text{receiver_wallet_id} = rw.\text{wallet_id}) \vee (t.\text{receiver_wallet_id} \text{ IS NULL})$
- Bảng tầm ảnh hưởng:

	Thêm	Xóa	Sửa
TRANSACTIONS	+	-	+(receiver_wallet_id)
WALLETS	-	+	-

Ràng buộc 17: Mỗi giao dịch thuộc một hạn mức áp dụng.

- Bối cảnh: TRANSACTIONS
- Nội dung: $\forall t \in \text{TRANSACTIONS}, \exists tl \in \text{TRANSACTION_LIMITS}: t.\text{limit_id} = tl.\text{limit_id}$
- Bảng tầm ảnh hưởng:

	Thêm	Xóa	Sửa
TRANSACTIONS	+	-	+(limit_id)
TRANSACTION_LIMITS	-	+	-

Ràng buộc 18: Mỗi giao dịch (nếu có) được áp dụng một voucher.

- Bối cảnh: TRANSACTIONS
- Nội dung: $\forall t \in \text{TRANSACTIONS}, (\exists v \in \text{VOUCHERS}: t.\text{voucher_id} = v.\text{voucher_id}) \vee (t.\text{voucher_id} \text{ IS NULL})$
- Bảng tầm ảnh hưởng:

	Thêm	Xóa	Sửa
TRANSACTIONS	+	-	+(voucher_id)
VOUCHERS	-	+	-

Ràng buộc 19: Mỗi quy tắc phí thuộc một loại giao dịch.

- Bối cảnh: TRANSACTION_FEES
- Nội dung: $\forall tf \in \text{TRANSACTION_FEES}, \exists tt \in \text{TRANSACTION_TYPES}: tf.type_id = tt.type_id$
- Bảng tầm ảnh hưởng:

	Thêm	Xóa	Sửa
TRANSACTION_FEES	+	-	+(type_id)
TRANSACTION_TYPES	-	+	-

Ràng buộc 20: Mỗi ví thuộc một người dùng.

- Bối cảnh: WALLETS
- Nội dung: $\forall w \in \text{WALLETS}, \exists u \in \text{USERS}: w.user_id = u.user_id$
- Bảng tầm ảnh hưởng:

	Thêm	Xóa	Sửa
WALLETS	+	-	+(user_id)
USERS	-	+	-

Ràng buộc 21: Mỗi phương thức thanh toán thuộc một người dùng.

- Bối cảnh: PAYMENT_METHODS
- Nội dung: $\forall pm \in \text{PAYMENT_METHODS}, \exists u \in \text{USERS}: pm.user_id = u.user_id$

- Bảng tầm ảnh hưởng:

	Thêm	Xóa	Sửa
PAYMENT_METHODS	+	-	+(user_id)
USERS	-	+	-

Ràng buộc 22: Mỗi cảnh báo gian lận thuộc một giao dịch.

- Bối cảnh: FRAUD_FLAGS
- Nội dung: $\forall ff \in \text{FRAUD_FLAGS}, \exists t \in \text{TRANSACTIONS}: ff.transaction_id = t.transaction_id$
- Bảng tầm ảnh hưởng:

	Thêm	Xóa	Sửa
FRAUD_FLAGS	+	-	+(transaction_id)
TRANSACTIONS	-	+	-

Ràng buộc 23: Mỗi cảnh báo gian lận thuộc một quy tắc.

- Bối cảnh: FRAUD_FLAGS
- Nội dung: $\forall ff \in \text{FRAUD_FLAGS}, \exists fr \in \text{FRAUD_RULES}: ff.rule_id = fr.rule_id$
- Bảng tầm ảnh hưởng:

	Thêm	Xóa	Sửa
FRAUD_FLAGS	+	-	+(rule_id)
FRAUD_RULES	-	+	-

Ràng buộc 24: Mỗi lệnh nạp/rút thuộc một ví.

- Bối cảnh: FUND_ORDERS

- Nội dung: $\forall fo \in \text{FUND_ORDERS}, \exists w \in \text{WALLETS}: fo.\text{wallet_id} = w.\text{wallet_id}$
- Bảng tầm ảnh hưởng:

	Thêm	Xóa	Sửa
FUND_ORDERS	+	-	+(wallet_id)
WALLETS	-	+	-

Ràng buộc 25: Mỗi lệnh nạp/rút thuộc một phương thức thanh toán.

- Bối cảnh: FUND_ORDERS
- Nội dung: $\forall fo \in \text{FUND_ORDERS}, \exists pm \in \text{PAYMENT_METHODS}: fo.\text{method_id} = pm.\text{method_id}$
- Bảng tầm ảnh hưởng:

	Thêm	Xóa	Sửa
FUND_ORDERS	+	-	+(method_id)
PAYMENT_METHODS	-	+	-

Ràng buộc 26: Mỗi bản ghi audit thuộc một giao dịch.

- Bối cảnh: AUDIT_LOGS
- Nội dung: $\forall al \in \text{AUDIT_LOGS}, \exists t \in \text{TRANSACTIONS}: al.\text{transaction_id} = t.\text{transaction_id}$
- Bảng tầm ảnh hưởng:

	Thêm	Xóa	Sửa
AUDIT_LOGS	+	-	+(transaction_id)
TRANSACTIONS	-	+	-

Ràng buộc 27: Mỗi bản ghi audit thuộc một ví.

- Bối cảnh: AUDIT_LOGS
- Nội dung: $\forall al \in \text{AUDIT_LOGS}, \exists w \in \text{WALLETS}: al.wallet_id = w.wallet_id$
- Bảng tầm ảnh hưởng:

	Thêm	Xóa	Sửa
AUDIT_LOGS	+	-	+(wallet_id)
WALLETS	-	+	-

Ràng buộc 28: Mỗi phiên đăng nhập thuộc một người dùng.

- Bối cảnh: LOGIN_SESSIONS
- Nội dung: $\forall ls \in \text{LOGIN_SESSIONS}, \exists u \in \text{USERS}: ls.user_id = u.user_id$
- Bảng tầm ảnh hưởng:

	Thêm	Xóa	Sửa
LOGIN_SESSIONS	+	-	+(user_id)
USERS	-	+	-

2.4.3. Ràng buộc khác

Ràng buộc 29: Số tiền giao dịch phải lớn hơn 0.

- Bối cảnh: TRANSACTIONS
- Nội dung: $\forall t \in \text{TRANSACTIONS}: t.amount > 0$
- Bảng tầm ảnh hưởng:

	Thêm	Xóa	Sửa
TRANSACTIONS	+	-	+(amount)

Ràng buộc 30: Số dư ví không được âm.

- Bối cảnh: WALLETS
- Nội dung: $\forall w \in \text{WALLETS}: w.balance \geq 0$
- Bảng tầm ảnh hưởng:

	Thêm	Xóa	Sửa
WALLETS	+	-	+(balance)

Ràng buộc 31: Tổng tiền giao dịch thực tế không được âm.

- Bối cảnh: TRANSACTIONS
- Nội dung: $\forall t \in \text{TRANSACTIONS}: t.\text{amount} + t.\text{fee_amount} \geq 0$
- Bảng tầm ảnh hưởng:

	Thêm	Xóa	Sửa
TRANSACTIONS	+	-	+(amount, fee_amount)

Ràng buộc 32: Thuộc tính phone trong bảng USERS là duy nhất.

- Bối cảnh: USERS
- Nội dung: $\forall u1, u2 \in \text{USERS}: u1.\text{phone} \neq u2.\text{phone}$
- Bảng tầm ảnh hưởng:

	Thêm	Xóa	Sửa
USERS	+	-	+(phone)

Ràng buộc 33: fee_rate nằm trong khoảng [0; 1].

- Bối cảnh: TRANSACTION_FEES
- Nội dung: $\forall tf \in \text{TRANSACTION_FEES}: 0 \leq tf.\text{fee_rate} \leq 1$
- Bảng tầm ảnh hưởng:

	Thêm	Xóa	Sửa
TRANSACTION_FEES	+	-	+(fee_rate)

Ràng buộc 34: Các giá trị phí trong TRANSACTION_FEES không được âm.

- Bối cảnh: TRANSACTION_FEES
- Nội dung: $\forall tf \in \text{TRANSACTION_FEES}:$

$$tf.fee_rate \geq 0 \wedge tf.fee_fixed \geq 0 \wedge tf.min_fee \geq 0 \wedge tf.max_fee \geq 0$$

- Bảng tầm ảnh hưởng:

	Thêm	Xóa	Sửa
TRANSACTION_FEES	+	-	+(fee_rate, fee_fixed, min_fee, max_fee)

Ràng buộc 35: Hạn mức giao dịch phải là số dương.

- Bối cảnh: TRANSACTIONS_LIMITS
- Nội dung: $\forall l \in \text{TRANSACTION_LIMITS}$:
 $l.max_amount_per_trans > 0 \wedge l.max_amount_per_day > 0 \wedge l.max_trans_per_day > 0$
- Bảng tầm ảnh hưởng:

	Thêm	Xóa	Sửa
TRANSACTIONS_LIMITS	+	-	+(max_amount_per_trans, max_amount_per_day, max_trans_per_day)

Ràng buộc 36: Hạn mức mỗi giao dịch không vượt quá hạn mức mỗi ngày.

- Bối cảnh: TRANSACTIONS_LIMITS
- Nội dung: $\forall l \in \text{TRANSACTION_LIMITS}$:
 $l.max_amount_per_trans \leq l.max_amount_per_day$
- Bảng tầm ảnh hưởng:

	Thêm	Xóa	Sửa
TRANSACTIONS_LIMITS	+	-	+(max_amount_per_trans, max_amount_per_day)

Ràng buộc 37: Ví gửi và ví nhận không được trùng nhau

- Bối cảnh: TRANSACTIONS
- Nội dung: $\forall t \in \text{TRANSACTIONS}$:
 $t.sender_wallet_id \text{ IS NOT NULL } \wedge t.receiver_wallet_id \text{ IS NOT NULL}$

$\Rightarrow t.sender_wallet_id \neq t.receiver_wallet_id$

- Bảng tầm ảnh hưởng:

	Thêm	Xóa	Sửa
TRANSACTIONS	+	-	+(sender_wallet_id, receiver_wallet_id)

Ràng buộc 38: Chỉ ví ACTIVE mới được gửi tiền.

- Bối cảnh: TRANSACTIONS, WALLETS
- Nội dung: $\forall t \in \text{TRANSACTIONS}, \exists w \in \text{WALLETS}:$
 $t.sender_wallet_id = w.wallet_id \Rightarrow w.wallet_status = 'ACTIVE'$
- Bảng tầm ảnh hưởng:

	Thêm	Xóa	Sửa
TRANSACTIONS	+	-	+(sender_wallet_id)
WALLETS	-	-	+(wallet_status)

Ràng buộc 39: Giao dịch TRANSFER phải có cả ví gửi và ví nhận.

- Bối cảnh: TRANSACTIONS, TRANSACTION_TYPES
- Nội dung: $\forall t \in \text{TRANSACTIONS}, \exists tt \in \text{TRANSACTION_TYPES}:$
 $t.type_id = tt.type_id \wedge tt.type_code = 'TRANSFER'$
 $\Rightarrow t.sender_wallet_id \text{ IS NOT NULL } \wedge t.receiver_wallet_id \text{ IS NOT NULL}$
- Bảng tầm ảnh hưởng:

	Thêm	Xóa	Sửa
TRANSACTIONS	+	-	+(type_id, sender_wallet_id, receiver_wallet_id)
TRANSACTION_TYPES	-	-	+(type_code)

Ràng buộc 40: Giao dịch TOP_UP không có ví gửi nhưng phải có ví nhận.

- Bối cảnh: TRANSACTIONS, TRANSACTION_TYPES
- Nội dung: $\forall t \in \text{TRANSACTIONS}, \exists tt \in \text{TRANSACTION_TYPES}$:
 $t.type_id = tt.type_id \wedge tt.type_code = 'TOP_UP'$
 $\Rightarrow t.sender_wallet_id \text{ IS NULL } \wedge t.receiver_wallet_id \text{ IS NOT NULL}$
- Bảng tầm ảnh hưởng:

	Thêm	Xóa	Sửa
TRANSACTIONS	+	-	+(type_id, sender_wallet_id, receiver_wallet_id)
TRANSACTION_TYPES	-	-	+(type_code)

Ràng buộc 41: Giao dịch WITHDRAW có ví gửi nhưng không có ví nhận.

- Bối cảnh: TRANSACTIONS, TRANSACTION_TYPES
- Nội dung: $\forall t \in \text{TRANSACTIONS}, \exists tt \in \text{TRANSACTION_TYPES}$:
 $t.type_id = tt.type_id \wedge tt.type_code = 'WITHDRAW'$
 $\Rightarrow t.sender_wallet_id \text{ IS NOT NULL } \wedge t.receiver_wallet_id \text{ IS NULL}$
- Bảng tầm ảnh hưởng:

	Thêm	Xóa	Sửa
TRANSACTIONS	+	-	+(type_id, sender_wallet_id, receiver_wallet_id)
TRANSACTION_TYPES	-	-	+(type_code)

Ràng buộc 42: Giao dịch không được vượt hạn mức KYC.

- Bối cảnh: TRANSACTIONS, WALLETS, USERS, TRANSACTION_LIMITS
- Nội dung: $\forall t \in \text{TRANSACTIONS}, \exists w \in \text{WALLETS}, \exists u \in \text{USERS}, \exists l \in \text{TRANSACTION_LIMITS}$:
 $t.sender_wallet_id = w.wallet_id \wedge w.user_id = u.user_id \wedge u.kyc_status =$
 $l.kyc_level \Rightarrow t.amount \leq l.max_amount_per_trans$
- Bảng tầm ảnh hưởng:

	Thêm	Xóa	Sửa
--	------	-----	-----

TRANSACTIONS	+	-	+(amount, sender_wallet_id)
WALLETS	-	-	+(user_id)
USERS	-	-	+(kyc_status)
TRANSACTION_LIMITS	-	-	+(kyc_level, max_amount_per_trans)

Ràng buộc 43: Tổng số tiền giao dịch trong ngày không vượt hạn mức ngày.

- Bối cảnh: TRANSACTIONS, WALLETS, USERS, TRANSACTION_LIMITS
- Nội dung: $\forall t \in \text{TRANSACTIONS}, \exists w \in \text{WALLETS}, \exists u \in \text{USERS}, \exists l \in \text{TRANSACTION_LIMITS}:$
 $t.\text{sender_wallet_id} = w.\text{wallet_id} \wedge w.\text{user_id} = u.\text{user_id} \wedge u.\text{kyc_status} = l.\text{kyc_level} \Rightarrow \text{SUM}(t.\text{amount trong ngày của } u) \leq l.\text{max_amount_per_day}$
- Bảng tầm ảnh hưởng:

	Thêm	Xóa	Sửa
TRANSACTIONS	+	-	+(amount, sender_wallet_id, status, created_at)
WALLETS	-	-	+(user_id)
USERS	-	-	+(kyc_status)
TRANSACTION_LIMITS	-	-	+(kyc_level, max_amount_per_day)

Ràng buộc 44: Số lượng giao dịch trong ngày không vượt số lần tối đa.

- Bối cảnh: TRANSACTIONS, WALLETS, USERS, TRANSACTION_LIMITS
- Nội dung: $\forall t \in \text{TRANSACTIONS}, \exists w \in \text{WALLETS}, \exists u \in \text{USERS}, \exists l \in \text{TRANSACTION_LIMITS}:$

$t.sender_wallet_id = w.wallet_id \wedge w.user_id = u.user_id \wedge u.kyc_status = l.kyc_level \Rightarrow COUNT(t \text{ trong ngày của } u) \leq l.max_trans_per_day$

- Bảng tầm ảnh hưởng:

	Thêm	Xóa	Sửa
TRANSACTIONS	+	-	+(sender_wallet_id, status, created_at)
WALLETS	-	-	+(user_id)
USERS	-	-	+(kyc_status)
TRANSACTION_LIMITS	-	-	+(kyc_level, max_trans_per_day)

Ràng buộc 45: Voucher chỉ được dùng khi còn hạn.

- Bối cảnh: TRANSACTIONS, VOUCHERS
- Nội dung: $\forall t \in \text{TRANSACTIONS}, \exists v \in \text{VOUCHERS}: t.voucher_id = v.voucher_id \Rightarrow v.valid_until \geq t.created_at$
- Bảng tầm ảnh hưởng:

	Thêm	Xóa	Sửa
TRANSACTIONS	+	-	+(voucher_id, created_at)
VOUCHERS	-	-	+(valid_until)

Ràng buộc 46: created_at của ví phải nhỏ hơn hoặc bằng updated_at.

- Bối cảnh: WALLETS
- Nội dung: $\forall w \in \text{WALLETS}: w.updated_at \text{ IS NULL } \vee w.created_at \leq w.updated_at$
- Bảng tầm ảnh hưởng:

	Thêm	Xóa	Sửa
WALLETS	+	-	+(created_at, updated_at)

Ràng buộc 47: Ngày giao dịch phải lớn hơn hoặc bằng ngày tạo ví liên quan.

- Bối cảnh: TRANSACTIONS, WALLETS
- Nội dung: $\forall t \in \text{TRANSACTIONS}, \exists w \in \text{WALLETS}$:
 $t.\text{sender_wallet_id} = w.\text{wallet_id} \Rightarrow t.\text{created_at} \geq w.\text{created_at}$
 Và:
 $\forall t \in \text{TRANSACTIONS}, \exists w \in \text{WALLETS}$:
 $t.\text{receiver_wallet_id} = w.\text{wallet_id} \Rightarrow t.\text{created_at} \geq w.\text{created_at}$
- Bảng tầm ảnh hưởng:

	Thêm	Xóa	Sửa
TRANSACTIONS	+	-	+(sender_wallet_id, receiver_wallet_id, created_at)
WALLETS	-	-	+(created_at)

Ràng buộc 48: Audit log không được sửa hoặc xóa sau khi tạo.

- Bối cảnh: AUDIT_LOGS
- Nội dung: Nếu $al \in \text{AUDIT_LOGS}$ đã tồn tại trong hệ thống, thì mọi thao tác UPDATE hoặc DELETE trên al đều không hợp lệ.
- Bảng tầm ảnh hưởng:

	Thêm	Xóa	Sửa
AUDIT_LOGS	+	-	+

CHƯƠNG 3: XÂY DỰNG VÀ QUẢN LÝ GIAO TÁC

3.1. Xây dựng và mô tả Trigger trong đồ án môn học

STT	TRIGGER	EVENT	TABLE
1.1	TRG_CHECK_ACTIVE_WALLET	INSERT, UPDATE	TRANSACTION
<u>Mã lệnh PL/SQL:</u> <pre> CREATE OR REPLACE TRIGGER trg_check_active_wallet BEFORE INSERT OR UPDATE ON transactions FOR EACH ROW WHEN (NEW.sender_wallet_id IS NOT NULL) DECLARE v_wallet_status wallets.wallet_status%TYPE; BEGIN -- Lấy trạng thái của ví gửi SELECT wallet_status INTO v_wallet_status FROM wallets WHERE wallet_id = :NEW.sender_wallet_id; -- Kiểm tra điều kiện ACTIVE IF v_wallet_status != 'ACTIVE' THEN RAISE_APPLICATION_ERROR(-20010, 'Giao dịch từ chối: Chỉ ví ở trạng thái ACTIVE mới được phép gửi hoặc rút tiền.');</pre> END IF; END;			
<u>Mô tả mã lệnh:</u> Mã lệnh này có chức năng kiểm tra trạng thái hoạt động của ví trước khi thêm mới hoặc cập nhật bản ghi trong bảng "TRANSACTIONS". Trigger sẽ đảm bảo chỉ những ví đang ở trạng thái hoạt động (ACTIVE) mới được phép thực hiện giao dịch (gửi hoặc rút tiền). Với mỗi dòng được thêm mới hoặc cập nhật trong bảng "TRANSACTIONS", trigger sẽ thực hiện các bước sau: Kiểm tra giao dịch có phát sinh từ một ví gửi hay không (tồn tại sender_wallet_id).			

	• Truy vấn bảng "WALLETS" để lấy trạng thái (wallet_status) của ví gửi tương ứng. • Kiểm tra xem trạng thái của ví có khác 'ACTIVE' hay không. • Nếu ví không ở trạng thái 'ACTIVE', trigger sẽ gây lỗi -20010 với thông báo 'Giao dịch từ chối: Chỉ ví ở trạng thái ACTIVE mới được phép gửi hoặc rút tiền.'.		
1.2	TRG_VALIDATE_TRANSFER	INSERT, UPDATE	TRANSACTION
	<u>Mã lệnh PL/SQL:</u> <pre> CREATE OR REPLACE TRIGGER trg_validate_transfer BEFORE INSERT OR UPDATE ON transactions FOR EACH ROW WHEN (NEW.type_id IS NOT NULL) DECLARE v_type_code transaction_types.type_code%TYPE; BEGIN -- Lấy mã type_code từ bảng transaction_types SELECT type_code INTO v_type_code FROM transaction_types WHERE type_id = :NEW.type_id; -- Kiểm tra điều kiện cho TRANSFER IF v_type_code = 'TRANSFER' THEN IF :NEW.sender_wallet_id IS NULL OR :NEW.receiver_wallet_id IS NULL THEN RAISE_APPLICATION_ERROR(-20011, 'Giao dịch TRANSFER yêu cầu phải có đầy đủ ví gửi và ví nhận.');</pre>		
	<u>Mô tả mã lệnh:</u> Mã lệnh này có chức năng kiểm tra tính hợp lệ về mặt cấu trúc dữ liệu của giao dịch chuyển tiền (TRANSFER) trước khi thêm mới hoặc cập nhật bản ghi trong bảng "TRANSACTIONS". Trigger đảm bảo rằng một giao dịch chuyển tiền bắt buộc phải xác định được cả người gửi và người nhận.		

	Với mỗi dòng được thêm mới hoặc cập nhật trong bảng "TRANSACTIONS", trigger sẽ thực hiện các bước sau: • Kiểm tra giao dịch có được gán loại giao dịch (type_id) hay không. • Truy vấn bảng "TRANSACTION_TYPES" để lấy mã loại giao dịch (type_code) tương ứng. • Kiểm tra nếu loại giao dịch là 'TRANSFER'. • Nếu đúng là 'TRANSFER', trigger tiếp tục kiểm tra thông tin ví gửi và ví nhận. • Nếu thiếu một trong hai (hoặc cả hai) thông tin ví, trigger sẽ gây lỗi - 20011 với thông báo 'Giao dịch TRANSFER yêu cầu phải có đầy đủ ví gửi và ví nhận.'		
1.3	TRG_VALIDATE_WITHDRAW	INSERT, UPDATE	TRANSACTION
<u>Mã lệnh PL/SQL:</u> <pre> CREATE OR REPLACE TRIGGER trg_validate_withdraw BEFORE INSERT OR UPDATE ON transactions FOR EACH ROW WHEN (NEW.type_id IS NOT NULL) DECLARE v_type_code transaction_types.type_code%TYPE; BEGIN -- Lấy mã type_code từ bảng transaction_types SELECT type_code INTO v_type_code FROM transaction_types WHERE type_id = :NEW.type_id; -- Kiểm tra điều kiện cho WITHDRAW IF v_type_code = 'WITHDRAW' THEN IF :NEW.sender_wallet_id IS NULL OR :NEW.receiver_wallet_id IS NOT NULL THEN RAISE_APPLICATION_ERROR(-20013, 'Giao dịch WITHDRAW yêu cầu phải có ví gửi và KHÔNG ĐƯỢC có ví nhận.');</pre> END IF; END IF; END;			

	<u>Mô tả mã lệnh:</u> Mã lệnh này có chức năng kiểm tra tính hợp lệ của giao dịch rút tiền (WITHDRAW) trước khi thêm mới hoặc cập nhật bản ghi trong bảng "TRANSACTIONS". Trigger đảm bảo rằng giao dịch rút tiền chỉ lấy tiền ra từ một ví (có ví gửi) và không được phép chuyển vào một ví khác trong hệ thống (không có ví nhận). Với mỗi dòng được thêm mới hoặc cập nhật trong bảng "TRANSACTIONS", trigger sẽ thực hiện các bước sau: • Kiểm tra giao dịch có được gán loại giao dịch (type_id) hay không. • Truy vấn bảng "TRANSACTION_TYPES" để lấy mã loại giao dịch (type_code) tương ứng. • Kiểm tra nếu loại giao dịch là 'WITHDRAW'. • Nếu đúng là 'WITHDRAW', trigger tiếp tục kiểm tra sự tồn tại của ví gửi và ví nhận. • Nếu giao dịch không có ví gửi HOẶC lại có chứa ví nhận, trigger sẽ gây lỗi -20013 với thông báo 'Giao dịch WITHDRAW yêu cầu phải có ví gửi và KHÔNG ĐƯỢC có ví nhận.'		
1.4	TRG_CHECK_DAILY_LIMITS	INSERT, UPDATE	TRANSACTIONS
	<u>Mã lệnh PL/SQL:</u> <pre> CREATE OR REPLACE TRIGGER trg_check_daily_limits BEFORE INSERT OR UPDATE ON transactions FOR EACH ROW DECLARE v_user_id NUMBER; v_kyc_status VARCHAR2(20); v_max_amount_day NUMBER; v_max_trans_day NUMBER; v_total_amount_today NUMBER(18,4); v_total_trans_today NUMBER; BEGIN -- Kiểm tra khi transaction chuẩn bị SUCCESS </pre>		

	<pre> IF :NEW.status = 'SUCCESS' THEN -- Tìm user từ sender wallet SELECT u.user_id, u.kyc_status INTO v_user_id, v_kyc_status FROM wallets w JOIN users u ON w.user_id = u.user_id WHERE w.wallet_id = :NEW.sender_wallet_id; -- Lấy hạn mức theo KYC SELECT max_amount_per_day, max_trans_per_day INTO v_max_amount_day, v_max_trans_day FROM transaction_limits WHERE kyc_level = v_kyc_status; -- ##### Tính tổng số tiền giao dịch SUCCESS trong ngày ##### SELECT NVL(SUM(t.amount), 0) INTO v_total_amount_today FROM transactions t JOIN wallets w ON t.sender_wallet_id = w.wallet_id WHERE w.user_id = v_user_id AND TRUNC(t.created_at) = TRUNC(NVL(:NEW.created_at, SYSTIMESTAMP)) AND t.status = 'SUCCESS' AND t.transaction_id != NVL(:NEW.transaction_id, -1); -- ##### Tính số lượng giao dịch SUCCESS trong ngày ##### SELECT COUNT(*) INTO v_total_trans_today FROM transactions t JOIN wallets w ON t.sender_wallet_id = w.wallet_id </pre>
--	--

	<pre> WHERE w.user_id = v_user_id AND TRUNC(t.created_at) = TRUNC(NVL(:NEW.created_at, SYSTIMESTAMP)) AND t.status = 'SUCCESS' AND t.transaction_id != NVL(:NEW.transaction_id, -1); -- Kiểm tra tổng số tiền giao dịch trong ngày IF (v_total_amount_today + :NEW.amount) > v_max_amount_day THEN RAISE_APPLICATION_ERROR(-20017, 'Vượt hạn mức tổng số tiền giao dịch tối đa trong ngày'); END IF; -- Kiểm tra số lượng giao dịch trong ngày IF (v_total_trans_today + 1) > v_max_trans_day THEN RAISE_APPLICATION_ERROR(-20018, 'Vượt số lượng giao dịch tối đa trong ngày'); END IF; END IF; END; </pre>
	<u>Mô tả mã lệnh:</u> Mã lệnh này có chức năng kiểm tra hạn mức giao dịch trong ngày trước khi thêm mới hoặc cập nhật bản ghi trong bảng "TRANSACTIONS". Trigger sẽ lấy thông tin người dùng và hạn mức giao dịch dựa trên trạng thái KYC, sau đó kiểm tra giao dịch mới có vượt quá giới hạn cho phép hay không. Với mỗi dòng được thêm mới hoặc cập nhật trong bảng "TRANSACTIONS", trigger sẽ thực hiện các bước sau: • Chỉ kiểm tra khi giao dịch có trạng thái 'SUCCESS'. • Truy vấn USER_ID và KYC_STATUS từ ví gửi tiền. • Lấy hạn mức số tiền và số lượng giao dịch tối đa trong ngày từ bảng "TRANSACTION_LIMITS". • Tính tổng số tiền và số lượng giao dịch thành công trong ngày của người dùng. • Tiến hành kiểm tra:

	- Nếu tổng số tiền giao dịch vượt hạn mức cho phép, trigger gây lỗi -20017 với thông báo 'Vượt hạn mức tổng số tiền giao dịch tối đa trong ngày'. - Nếu số lượng giao dịch vượt số lượng cho phép, trigger gây lỗi -20018 với thông báo 'Vượt số lượng giao dịch tối đa trong ngày'.		
1.5	TRG__CHECK_VOUCHER_VALID	INSERT, UPDATE	TRANSACTIONS
<u>Mã lệnh PL/SQL:</u> <pre> CREATE OR REPLACE TRIGGER trg_check_voucher_valid BEFORE INSERT OR UPDATE ON transactions FOR EACH ROW DECLARE v_valid_until TIMESTAMP; BEGIN -- Kiểm tra transaction có voucher không IF :NEW.voucher_id IS NOT NULL THEN -- Lấy thời gian hết hạn voucher SELECT valid_until INTO v_valid_until FROM vouchers WHERE voucher_id = :NEW.voucher_id; -- ##### Kiểm tra voucher còn hạn ##### IF v_valid_until < NVL(:NEW.created_at, SYSTIMESTAMP) THEN RAISE_APPLICATION_ERROR(-20019, 'Voucher đã hết hạn'); END IF; END IF; END;</pre>			
<u>Mô tả mã lệnh:</u> Mã lệnh này có chức năng kiểm tra tính hợp lệ của voucher trước khi thêm mới hoặc cập nhật bản ghi trong bảng "TRANSACTIONS". Trigger sẽ kiểm tra			

	voucher được sử dụng còn hạn hay không, nếu voucher đã hết hạn thì sẽ gây ra lỗi và thông báo cho người dùng. Với mỗi dòng được thêm mới hoặc cập nhật trong bảng "TRANSACTIONS", trigger sẽ thực hiện các bước sau: • Kiểm tra giao dịch có sử dụng voucher hay không. • Nếu có voucher, trigger sẽ truy vấn bảng "VOUCHERS" để lấy thời gian hết hạn VALID_UNTIL. • So sánh thời gian hết hạn voucher với thời gian tạo giao dịch. • Nếu voucher đã hết hạn, trigger sẽ gây lỗi -20019 với thông báo 'Voucher đã hết hạn'.		
1.6	TRG_CHECK_TRANSACTION_DATE	INSERT, UPDATE	TRANSACTIONS
	<u>Mã lệnh PL/SQL:</u> <pre> CREATE OR REPLACE TRIGGER trg_check_transaction_date BEFORE INSERT OR UPDATE ON transactions FOR EACH ROW DECLARE v_sender_created_at TIMESTAMP; v_receiver_created_at TIMESTAMP; BEGIN -- ##### Kiểm tra ví gửi ##### IF :NEW.sender_wallet_id IS NOT NULL THEN SELECT created_at INTO v_sender_created_at FROM wallets WHERE wallet_id = :NEW.sender_wallet_id; IF NVL(:NEW.created_at, SYSTIMESTAMP) < v_sender_created_at THEN RAISE_APPLICATION_ERROR(-20020, 'Ngày giao dịch nhỏ hơn ngày tạo ví gửi'); END IF; END IF; </pre>		

	<pre> -- ##### Kiểm tra ví nhận ##### IF :NEW.receiver_wallet_id IS NOT NULL THEN SELECT created_at INTO v_receiver_created_at FROM wallets WHERE wallet_id = :NEW.receiver_wallet_id; IF NVL(:NEW.created_at, SYSTIMESTAMP) < v_receiver_created_at THEN RAISE_APPLICATION_ERROR(-20021, 'Ngày giao dịch nhỏ hơn ngày tạo ví nhận'); END IF; END IF; END; </pre>		
	<u>Mô tả mã lệnh:</u> Mã lệnh này có chức năng kiểm tra tính hợp lệ của ngày giao dịch trước khi thêm mới hoặc cập nhật bản ghi trong bảng "TRANSACTIONS". Trigger sẽ kiểm tra ngày giao dịch có nhỏ hơn ngày tạo ví gửi hoặc ví nhận hay không. Nếu không hợp lệ, trigger sẽ gây ra lỗi và thông báo cho người dùng. Với mỗi dòng được thêm mới hoặc cập nhật trong bảng "TRANSACTIONS", trigger sẽ thực hiện các bước sau: • Nếu có SENDER_WALLET_ID, trigger sẽ truy vấn ngày tạo ví gửi CREATED_AT trong bảng "WALLETS". • So sánh ngày giao dịch với ngày tạo ví gửi. Nếu ngày giao dịch nhỏ hơn ngày tạo ví gửi, trigger sẽ gây lỗi -20020 với thông báo 'Ngày giao dịch nhỏ hơn ngày tạo ví gửi'. • Nếu có RECEIVER_WALLET_ID, trigger sẽ truy vấn ngày tạo ví nhận CREATED_AT trong bảng "WALLETS". • So sánh ngày giao dịch với ngày tạo ví nhận. Nếu ngày giao dịch nhỏ hơn ngày tạo ví nhận, trigger sẽ gây lỗi -20021 với thông báo 'Ngày giao dịch nhỏ hơn ngày tạo ví nhận'.		
1.7	TRG_CHECK_KYC_LIMIT	INSERT, UPDATE	TRANSACTION
	<u>Mã lệnh PL/SQL:</u>		

```

CREATE OR REPLACE TRIGGER trg_check_kyc_limit
BEFORE INSERT OR UPDATE ON transactions
FOR EACH ROW
WHEN (NEW.sender_wallet_id IS NOT NULL)
DECLARE
    v_kyc_status users.kyc_status%TYPE;
    v_max_amount transaction_limits.max_amount_per_trans%TYPE;
BEGIN
    -- Lấy cấp độ KYC của người dùng sở hữu ví gửi
    SELECT u.kyc_status
    INTO v_kyc_status
    FROM wallets w
    JOIN users u ON w.user_id = u.user_id
    WHERE w.wallet_id = :NEW.sender_wallet_id;

    -- Kiểm tra hạn mức giao dịch dựa trên cấp độ KYC
    BEGIN
        SELECT max_amount_per_trans
        INTO v_max_amount
        FROM transaction_limits
        WHERE kyc_level = v_kyc_status;

        IF :NEW.amount > v_max_amount THEN
            RAISE_APPLICATION_ERROR(-20015,
                'Số tiền giao dịch (' || :NEW.amount ||
                ') vượt quá hạn mức tối đa cho phép/giao dịch đối với cấp độ KYC '
                ||
                v_kyc_status || ' (Giới hạn: ' || v_max_amount || ').');
        END IF;
    END IF;

    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            RAISE_APPLICATION_ERROR(-20016, 'Hệ thống chưa cấu hình
            hạn mức giao dịch cho cấp độ KYC: ' || v_kyc_status);

```

	END; END;
	<u>Mô tả mã lệnh:</u> Mã lệnh này có chức năng kiểm soát rủi ro, kiểm tra xem số tiền của một giao dịch có vượt quá hạn mức cho phép dựa trên cấp độ định danh (KYC) của người dùng hay không, trước khi bản ghi được thêm mới hoặc cập nhật vào bảng "TRANSACTIONS". Với mỗi dòng được thêm mới hoặc cập nhật trong bảng "TRANSACTIONS", trigger sẽ thực hiện các bước sau: • Kiểm tra giao dịch có phát sinh từ một ví gửi hay không (tồn tại sender_wallet_id). • Thực hiện truy vấn kết hợp (JOIN) giữa bảng "WALLETS" và "USERS" để lấy cấp độ định danh (kyc_status) của người dùng sở hữu ví gửi đó. • Tiếp tục truy vấn bảng "TRANSACTION_LIMITS" để lấy hạn mức giao dịch tối đa (max_amount_per_trans) tương ứng với cấp độ KYC vừa tìm được (nếu không tìm thấy cấu hình KYC này, sẽ vắng lỗi - 20016). • So sánh số tiền của giao dịch hiện tại (amount) với hạn mức tối đa vừa lấy ra. • Nếu số tiền giao dịch lớn hơn hạn mức cho phép, trigger sẽ gây lỗi - 20015 với thông báo chi tiết bao gồm số tiền định giao dịch, cấp độ KYC và hạn mức tối đa của cấp độ đó.

3.2. Xây dựng và mô tả Procedure, Function trong đồ án môn học

3.2.1. Procedure

STT	PROCEDURE
2.1	PR_CREATE_USER_WITH_WALLET
	<pre>CREATE OR REPLACE PROCEDURE pr_create_user_with_wallet (p_full_name IN users.full_name%TYPE, p_email IN users.email%TYPE, p_phone IN users.phone%TYPE, p_password_hash IN users.password_hash%TYPE, -- Nhận chuỗi hash từ Back-end p_pin_code IN wallets.pin_code%TYPE,</pre>

<pre> p_currency IN wallets.currency%TYPE DEFAULT 'VND') AS v_user_id users.user_id%TYPE; BEGIN -- 1. Thêm mới thông tin người dùng vào bảng USERS -- Các trường kyc_status, is_active, created_at sẽ lấy giá trị DEFAULT đã cấu hình ở bảng INSERT INTO users (full_name, email, phone, password_hash) VALUES (p_full_name, p_email, p_phone, p_password_hash) RETURNING user_id INTO v_user_id; -- Lấy ID tự động sinh ra để dùng cho bảng WALLETS INSERT INTO wallets (user_id, balance, currency, wallet_status, pin_code) VALUES (v_user_id, 0, p_currency, 'ACTIVE', p_pin_code); COMMIT; EXCEPTION WHEN OTHERS THEN ROLLBACK; RAISE_APPLICATION_ERROR(-20101, 'Lỗi hệ thống khi tạo tài khoản và ví: ' SQLERRM); END; </pre>	<u>Mô tả mã lệnh:</u> Mã lệnh này có chức năng xử lý nghiệp vụ đăng ký người dùng mới đồng thời khởi tạo ví điện tử mặc định cho người dùng đó. Procedure nhận vào các thông tin cá nhân bao gồm họ tên, email, số điện thoại, chuỗi mật khẩu đã được băm (hash) từ hệ thống Back-end, mã PIN bảo mật của ví và loại tiền tệ (mặc định là 'VND'). Khi được gọi, procedure sẽ thực hiện thêm mới thông tin vào bảng "USERS" (các trường như trạng thái định danh KYC, trạng thái hoạt động và thời gian tạo sẽ tự động lấy giá trị mặc định) và lấy ra ID của người dùng vừa được tạo. Sau đó, hệ thống sử dụng ID này để tiếp tục tạo một ví điện tử trong bảng "WALLETS" với số dư ban đầu bằng 0, trạng thái 'ACTIVE' và mã PIN tương ứng. Nếu mọi thao tác diễn ra thành công, hệ thống sẽ lưu lại toàn bộ thay đổi
---	--

	(COMMIT). Trong trường hợp có bất kỳ lỗi nào xảy ra trong suốt quá trình, procedure sẽ lập tức hoàn tác toàn bộ thao tác (ROLLBACK) và ném ra thông báo lỗi hệ thống. Procedure này giúp đảm bảo tính toàn vẹn dữ liệu, chắc chắn rằng mỗi khi một tài khoản người dùng được tạo mới thì sẽ luôn có một ví điện tử hợp lệ đi kèm mà không gặp tình trạng tạo thiếu hay gián đoạn.
2.2	PR_UPDATE_USER_INFO <u>Mã lệnh PL/SQL:</u> <pre> CREATE OR REPLACE PROCEDURE pr_update_user_info (p_user_id IN users.user_id%TYPE, p_full_name IN users.full_name%TYPE, p_email IN users.email%TYPE, p_phone IN users.phone%TYPE) AS BEGIN -- Thực hiện cập nhật các trường thông tin cơ bản UPDATE users SET full_name = p_full_name, email = p_email, phone = p_phone WHERE user_id = p_user_id; -- Kiểm tra xem ID người dùng truyền vào có tồn tại hay không IF SQL%ROWCOUNT = 0 THEN RAISE_APPLICATION_ERROR(-20102, 'Không tìm thấy người dùng với ID đã cung cấp: ' p_user_id); END IF; COMMIT; EXCEPTION WHEN OTHERS THEN ROLLBACK; RAISE; END; </pre> <u>Mô tả mã lệnh:</u>

	Mã lệnh này có chức năng xử lý nghiệp vụ cập nhật thông tin cá nhân của người dùng. Procedure nhận vào các tham số bao gồm mã định danh người dùng (ID) cùng với các dữ liệu mới cần thay đổi: họ và tên, email và số điện thoại. Khi được gọi, procedure sẽ thực hiện câu lệnh UPDATE để ghi đè các thông tin mới (họ tên, email, số điện thoại) vào bảng "USERS" ứng với mã người dùng được truyền vào. Ngay sau đó, hệ thống sẽ kiểm tra số lượng bản ghi bị ảnh hưởng thông qua biến hệ thống (SQL%ROWCOUNT). Nếu không có bản ghi nào được cập nhật (đồng nghĩa với việc ID người dùng cung cấp không tồn tại), procedure sẽ lập tức gây lỗi -20102 với thông báo không tìm thấy người dùng. Nếu thao tác cập nhật thành công, hệ thống sẽ lưu lại toàn bộ thay đổi (COMMIT). Trong trường hợp có bất kỳ lỗi bất thường nào xảy ra trong quá trình thực thi, procedure sẽ hoàn tác toàn bộ giao dịch (ROLLBACK) và ném lỗi (RAISE) lên để đảm bảo tính an toàn và toàn vẹn của dữ liệu hệ thống.
2.3	PR_CHANGE_USER_PASSWORD <u>Mã lệnh PL/SQL:</u> <pre> CREATE OR REPLACE PROCEDURE pr_change_user_password (p_user_id IN users.user_id%TYPE, p_new_password_hash IN users.password_hash%TYPE BEGIN -- Cập nhật mật khẩu mới UPDATE users SET password_hash = p_new_password_hash WHERE user_id = p_user_id; -- Kiểm tra sự tồn tại của người dùng IF SQL%ROWCOUNT = 0 THEN RAISE_APPLICATION_ERROR(-20103, 'Không thể đổi mật khẩu. Không tìm thấy người dùng có ID: ' p_user_id); END IF; COMMIT; EXCEPTION WHEN OTHERS THEN ROLLBACK; </pre>

	RAISE; END; <u>Mô tả mã lệnh:</u> Mã lệnh này có chức năng xử lý nghiệp vụ thay đổi mật khẩu của người dùng trong hệ thống. Procedure nhận vào các tham số bao gồm mã định danh người dùng (ID) và chuỗi mật khẩu mới đã được băm (hash) an toàn từ hệ thống Back-end. Khi được gọi, procedure sẽ thực hiện câu lệnh UPDATE để ghi đè mật khẩu mới (password_hash) vào bảng "USERS" ứng với mã người dùng được truyền vào. Ngay sau đó, hệ thống sẽ kiểm tra số lượng bản ghi bị ảnh hưởng thông qua biến hệ thống (SQL%ROWCOUNT). Nếu không có bản ghi nào được cập nhật (đồng nghĩa với việc ID người dùng cung cấp không tồn tại trong cơ sở dữ liệu), procedure sẽ lập tức gây lỗi -20103 với thông báo không thể đổi mật khẩu do không tìm thấy người dùng. Nếu thao tác cập nhật thành công, hệ thống sẽ lưu lại toàn bộ thay đổi (COMMIT). Trong trường hợp có bất kỳ lỗi bất thường nào xảy ra trong quá trình thực thi, procedure sẽ hoàn tác toàn bộ giao dịch (ROLLBACK) và ném lỗi (RAISE) lên hệ thống để đảm bảo tính an toàn và bảo mật của dữ liệu.
2.4	PR_LINK_BANK_ACCOUNT <u>Mã lệnh PL/SQL:</u> <pre> CREATE OR REPLACE PROCEDURE pr_link_bank_account (p_wallet_id IN wallets.wallet_id%TYPE, p_provider_name IN payment_methods.provider_name%TYPE, p_masked_number IN payment_methods.masked_number%TYPE, p_is_default IN payment_methods.is_default%TYPE DEFAULT 0) AS v_user_id wallets.user_id%TYPE; v_wallet_status wallets.wallet_status%TYPE; BEGIN -- Kiểm tra sự tồn tại của ví và lấy thông tin user_id cùng trạng thái ví BEGIN SELECT user_id, wallet_status INTO v_user_id, v_wallet_status FROM wallets WHERE wallet_id = p_wallet_id; </pre>

	<pre> EXCEPTION WHEN NO_DATA_FOUND THEN RAISE_APPLICATION_ERROR(-20110, 'Lỗi: Không tìm thấy ví điện tử với ID đã cung cấp.');</pre> END; -- Kiểm tra điều kiện trạng thái ví <pre> IF v_wallet_status != 'ACTIVE' THEN RAISE_APPLICATION_ERROR(-20111, 'Lỗi: Ví điện tử đang bị khóa hoặc đóng, không thể liên kết ngân hàng.');</pre> END IF; -- Xử lý logic đặt làm mặc định -- Nếu người dùng chọn phương thức này làm mặc định (p_is_default = 1), -- hệ thống sẽ tự động gỡ bỏ trạng thái mặc định của các tài khoản/thẻ cũ trước đó để không vi phạm ràng buộc UNIQUE INDEX (uq_pm_default). <pre> IF p_is_default = 1 THEN UPDATE payment_methods SET is_default = 0 WHERE user_id = v_user_id AND is_default = 1; END IF;</pre> -- Thêm mới bản ghi liên kết ngân hàng vào bảng PAYMENT_METHODS <pre> INSERT INTO payment_methods (user_id, method_type, provider_name, masked_number, is_default, is_verified, is_active) VALUES (v_user_id, 'BANK_ACCOUNT', -- Cố định loại phương thức là tài khoản ngân hàng</pre>
--	--

	<pre> p_provider_name, p_masked_number, p_is_default, 1, -- Mặc định đặt bằng 1 (Đã xác thực thành công với phía ngân hàng) 1 -- Trạng thái hoạt động kích hoạt ngay); COMMIT; EXCEPTION WHEN OTHERS THEN -- Hoàn tác nếu gặp bất kỳ lỗi xung đột dữ liệu nào khác ROLLBACK; RAISE_APPLICATION_ERROR(-20112, 'Lỗi hệ thống khi liên kết tài khoản ngân hàng: ' SQLERRM); END; </pre> <u>Mô tả mã lệnh:</u> Mã lệnh này có chức năng xử lý nghiệp vụ liên kết tài khoản ngân hàng vào ví điện tử của người dùng. Procedure nhận vào các tham số bao gồm mã định danh ví điện tử, tên ngân hàng hoặc nhà cung cấp, số tài khoản đã được ẩn một phần (masked number) và cờ đánh dấu phương thức này có được đặt làm mặc định hay không. Khi được gọi, procedure sẽ truy vấn bảng "WALLETS" để xác thực sự tồn tại của ví và lấy ra mã định danh người dùng cùng trạng thái ví; nếu không tìm thấy ví, hệ thống sẽ gây lỗi -20110. Tiếp theo, procedure kiểm tra điều kiện trạng thái; nếu ví không ở trạng thái 'ACTIVE', thao tác sẽ bị chặn lại kèm lỗi -20111. Nếu phương thức mới được chọn làm mặc định, hệ thống sẽ tự động gỡ bỏ trạng thái mặc định của các tài khoản hoặc thẻ cũ của người dùng trong bảng "PAYMENT_METHODS" để tránh vi phạm ràng buộc dữ liệu. Sau đó, hệ thống tiến hành thêm mới bản ghi liên kết ngân hàng vào bảng "PAYMENT_METHODS" với trạng thái đã xác thực và kích hoạt thành công. Nếu không có vấn đề gì, hệ thống sẽ lưu lại toàn bộ thay đổi (COMMIT). Trong trường hợp xảy ra bất kỳ lỗi hệ thống hay xung đột dữ liệu nào, procedure sẽ lập tức hoàn tác toàn bộ giao dịch (ROLLBACK) và ném ra thông báo lỗi -20112. Procedure này giúp đảm bảo thao tác liên kết ngân hàng diễn ra an toàn, tuân thủ chặt chẽ các điều kiện về trạng thái ví và tính duy nhất của phương thức thanh toán mặc định.
--	---

2.5	PR_SET_DEFAULT_PAYMENT_METHOD <u>Mã lệnh PL/SQL:</u> <pre data-bbox="320 320 1396 2002"> CREATE OR REPLACE PROCEDURE pr_set_default_payment_method (p_user_id IN NUMBER, p_provider_name IN VARCHAR2, p_method_type IN VARCHAR2) AS v_count NUMBER; BEGIN -- Kiểm tra phương thức thanh toán có tồn tại theo đúng thông tin cung -- cấp, đã xác thực và đang hoạt động không SELECT COUNT(*) INTO v_count FROM payment_methods WHERE user_id = p_user_id AND provider_name = p_provider_name AND method_type = p_method_type AND is_verified = 1 AND is_active = 1; IF v_count = 0 THEN RAISE_APPLICATION_ERROR(-20113, 'Phương thức thanh toán không hợp lệ, chưa xác thực hoặc đã bị vô hiệu hóa.');</pre> <pre data-bbox="320 1487 767 1845"> END IF; -- Bỏ mặc định cũ UPDATE payment_methods SET is_default = 0 WHERE user_id = p_user_id AND is_default = 1; -- Đặt mặc định mới dựa trên ngân hàng và loại phương thức được chọn UPDATE payment_methods</pre>
-----	--

	<pre> SET is_default = 1 WHERE user_id = p_user_id AND provider_name = p_provider_name AND method_type = p_method_type; COMMIT; EXCEPTION WHEN OTHERS THEN -- Hoàn tác nếu có lỗi bất ngờ xảy ra ROLLBACK; RAISE; END; </pre>
	<u>Mô tả mã lệnh:</u> Mã lệnh này có chức năng thiết lập phương thức thanh toán mặc định cho người dùng trong bảng "PAYMENT_METHODS". Procedure thực hiện các bước sau: • Kiểm tra phương thức thanh toán theo USER_ID, PROVIDER_NAME và METHOD_TYPE có tồn tại, đã được xác thực và đang hoạt động hay không. • Nếu không hợp lệ, procedure sẽ gây lỗi với mã -20113 và thông báo: 'Phương thức thanh toán không hợp lệ, chưa xác thực hoặc đã bị vô hiệu hóa'. • Nếu hợp lệ, procedure sẽ cập nhật phương thức thanh toán mặc định cũ của người dùng thành "IS_DEFAULT" = 0. • Sau đó, procedure thiết lập phương thức thanh toán được chọn mới thành mặc định bằng cách cập nhật "IS_DEFAULT" = 1. • Cuối cùng, procedure thực hiện "COMMIT" để lưu thay đổi. Nếu xảy ra lỗi trong quá trình xử lý, procedure sẽ "ROLLBACK" để hoàn tác dữ liệu.
2.6	PR_UPDATE_TRANSACTION_LIMIT <u>Mã lệnh PL/SQL:</u> <pre> CREATE OR REPLACE PROCEDURE pr_update_transaction_limit (p_kyc_level IN VARCHAR2, p_max_amount_per_trans IN NUMBER, </pre>

	<pre> p_max_amount_per_day IN NUMBER, p_max_trans_per_day IN NUMBER) AS v_count NUMBER; BEGIN -- Kiểm tra kyc level tồn tại SELECT COUNT(*) INTO v_count FROM transaction_limits WHERE kyc_level = p_kyc_level; IF v_count = 0 THEN RAISE_APPLICATION_ERROR(-20114, 'KYC level không tồn tại'); END IF; -- Update hạn mức UPDATE transaction_limits SET max_amount_per_trans = p_max_amount_per_trans, max_amount_per_day = p_max_amount_per_day, max_trans_per_day = p_max_trans_per_day WHERE kyc_level = p_kyc_level; COMMIT; END;</pre>
	<u>Mô tả mã lệnh:</u> Mã lệnh này có chức năng cập nhật hạn mức giao dịch theo từng mức xác thực KYC trong bảng "TRANSACTION_LIMITS". Procedure thực hiện các bước sau: • Kiểm tra KYC_LEVEL được cung cấp có tồn tại trong bảng "TRANSACTION_LIMITS" hay không.

	- Nếu không tồn tại, procedure sẽ gây lỗi với mã lỗi -20114 và thông báo: 'KYC level không tồn tại' - Nếu hợp lệ, procedure sẽ cập nhật: - MAX_AMOUNT_PER_TRANS: hạn mức giao dịch tối đa. - MAX_AMOUNT_PER_DAY: hạn mức tổng giao dịch trong ngày. - MAX_TRANS_PER_DAY: số lượng giao dịch tối đa trong ngày. - Cuối cùng, procedure thực hiện "COMMIT" để lưu thay đổi vào cơ sở dữ liệu.
2.7	PR_CREATE_LOGIN_SESSION <u>Mã lệnh PL/SQL:</u> <pre> CREATE OR REPLACE PROCEDURE pr_create_login_session (p_user_id IN NUMBER, p_device_info IN VARCHAR2, p_ip_address IN VARCHAR2) AS -- Khai báo ngoại lệ cho lỗi vi phạm khóa ngoại (Foreign Key Violation) e_fk_violation EXCEPTION; PRAGMA EXCEPTION_INIT(e_fk_violation, -2291); BEGIN INSERT INTO login_sessions (user_id, device_info, ip_address, is_active) VALUES (p_user_id, p_device_info, p_ip_address, 1); COMMIT; EXCEPTION WHEN e_fk_violation THEN ROLLBACK; RAISE_APPLICATION_ERROR(-20115, 'User không tồn tại.');</pre> <pre> WHEN OTHERS THEN ROLLBACK; RAISE; END;</pre>
	<u>Mô tả mã lệnh:</u>

	Mã lệnh này có chức năng tạo phiên đăng nhập mới cho người dùng trong bảng "LOGIN_SESSIONS". Procedure thực hiện các bước sau: • Thêm một bản ghi mới vào bảng "LOGIN_SESSIONS" với các thuộc tính: USER_ID, DEVICE_INFO, IP_ADDRESS, IS_ACTIVE. • Sau khi thêm dữ liệu thành công, procedure thực hiện "COMMIT" để lưu thay đổi. • Procedure có xử lý ngoại lệ vi phạm khóa ngoại. Nếu USER_ID không tồn tại trong bảng liên kết, procedure sẽ ROLLBACK và gây lỗi với mã lỗi -20115 với thông báo: 'User không tồn tại'. • Nếu xảy ra lỗi khác trong quá trình xử lý, procedure sẽ ROLLBACK và phát sinh lại lỗi.
2.8	PR_TOP_UP_WALLET <u>Mã lệnh PL/SQL:</u> <pre> CREATE OR REPLACE PROCEDURE pr_top_up_wallet(p_wallet_id IN NUMBER, p_method_id IN NUMBER, p_amount IN NUMBER, p_idempotency_key IN VARCHAR2, p_gateway_success IN NUMBER, p_gateway_ref IN VARCHAR2 DEFAULT NULL, p_description IN VARCHAR2 DEFAULT NULL, p_out_order_id OUT NUMBER, p_out_transaction_id OUT NUMBER, p_out_status OUT VARCHAR2) IS -- Thông tin ví v_wallet_status wallets.wallet_status%TYPE; v_wallet_balance wallets.balance%TYPE; v_wallet_user_id wallets.user_id%TYPE; -- Thông tin payment method v_method_user_id payment_methods.user_id%TYPE; </pre>

	<pre> v_method_verified payment_methods.is_verified%TYPE; v_method_active payment_methods.is_active%TYPE; -- Idempotency v_existing_order_id fund_orders.order_id%TYPE; v_existing_status fund_orders.status%TYPE; -- Giao dịch v_order_id fund_orders.order_id%TYPE; v_type_id transaction_types.type_id%TYPE; v_transaction_id transactions.transaction_id%TYPE; v_reference_code transactions.reference_code%TYPE; v_balance_before wallets.balance%TYPE; v_balance_after wallets.balance%TYPE; v_description transactions.description%TYPE; BEGIN -- 1. Kiểm tra input IF p_amount IS NULL OR p_amount <= 0 THEN RAISE_APPLICATION_ERROR(-20100, 'Số tiền nạp phải lớn hơn 0.');</pre>
	<pre> END IF; IF p_idempotency_key IS NULL THEN RAISE_APPLICATION_ERROR(-20101, 'idempotency_key không được để trống khi nạp tiền.');</pre>
	<pre> END IF; -- 2. Kiểm tra idempotency_key đã tồn tại chưa. BEGIN SELECT order_id, status INTO v_existing_order_id, v_existing_status FROM fund_orders WHERE idempotency_key = p_idempotency_key;</pre>

```

    p_out_order_id := v_existing_order_id;
    p_out_status  := v_existing_status;

    -- Vì fund_orders hiện chưa có transaction_id, tìm lại transaction qua mô
    tả kỹ thuật.
    BEGIN
        SELECT transaction_id
        INTO  p_out_transaction_id
        FROM  transactions
        WHERE receiver_wallet_id = p_wallet_id
              AND      description  LIKE  '%TOP_UP_ORDER_ID='  ||
v_existing_order_id || '%'
              AND ROWNUM = 1;
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            p_out_transaction_id := NULL;
    END;

    RETURN;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        NULL;
END;

-- 3. Kiểm tra wallet tồn tại và ACTIVE.
SELECT wallet_status, balance, user_id
INTO  v_wallet_status, v_wallet_balance, v_wallet_user_id
FROM  wallets
WHERE wallet_id = p_wallet_id;

IF v_wallet_status != 'ACTIVE' THEN
    RAISE_APPLICATION_ERROR(-20102, 'Vị nhận tiền không ở trạng
    thái ACTIVE.');
```

	<pre> END IF; -- 4. Kiểm tra payment_method tồn tại, cùng user, đã verified và active. SELECT user_id, is_verified, is_active INTO v_method_user_id, v_method_verified, v_method_active FROM payment_methods WHERE method_id = p_method_id; IF v_method_user_id != v_wallet_user_id THEN RAISE_APPLICATION_ERROR(-20103, 'Payment method không thuộc cùng user với wallet.');</pre> <pre> END IF; IF v_method_verified != 1 THEN RAISE_APPLICATION_ERROR(-20104, 'Payment method chưa được xác thực.');</pre> <pre> END IF; IF v_method_active != 1 THEN RAISE_APPLICATION_ERROR(-20105, 'Payment method đang không active.');</pre> <pre> END IF; -- 5. Tạo fund_order status PENDING. INSERT INTO fund_orders (wallet_id, method_id, order_type, amount, status, idempotency_key, gateway_ref, created_at) VALUES (p_wallet_id, p_method_id, 'TOP_UP', p_amount, 'PENDING', p_idempotency_key, p_gateway_ref, SYSTIMESTAMP) RETURNING order_id INTO v_order_id; -- 6. Nếu gateway thất bại thì cập nhật fund_order FAILED và kết thúc. IF NVL(p_gateway_success, 0) != 1 THEN </pre>
--	--

	<pre> UPDATE fund_orders SET status = 'FAILED', gateway_ref = p_gateway_ref WHERE order_id = v_order_id; p_out_order_id := v_order_id; p_out_transaction_id := NULL; p_out_status := 'FAILED'; RETURN; END IF; -- 7. Gateway thành công: lock wallet FOR UPDATE. SELECT balance, wallet_status INTO v_balance_before, v_wallet_status FROM wallets WHERE wallet_id = p_wallet_id FOR UPDATE; IF v_wallet_status != 'ACTIVE' THEN RAISE_APPLICATION_ERROR(-20106, 'Ví bị đổi trạng thái, không thể cộng tiền.');</pre> END IF; <pre> -- 8. Sinh mã giao dịch và lấy type TOP_UP. SELECT type_id INTO v_type_id FROM transaction_types WHERE type_code = 'TOP_UP'; v_reference_code := fn_generate_code('TOPUP'); v_description := NVL(p_description, 'Nạp tiền vào ví') ' TOP_UP_ORDER_ID=' v_order_id; -- 9. Cộng tiền vào ví.</pre>
--	--

	<pre> UPDATE wallets SET balance = balance + p_amount WHERE wallet_id = p_wallet_id; v_balance_after := v_balance_before + p_amount; -- 10. Tạo transaction TOP_UP. INSERT INTO transactions (type_id, sender_wallet_id, receiver_wallet_id, amount, fee_amount, status, reference_code, step, description, created_at) VALUES (v_type_id, NULL, p_wallet_id, p_amount, 0, 'COMPLETED', v_reference_code, TO_NUMBER(TO_CHAR(SYSTIMESTAMP, 'HH24')), v_description, SYSTIMESTAMP) RETURNING transaction_id INTO v_transaction_id; -- 11. Ghi audit_logs CREDIT. INSERT INTO audit_logs (transaction_id, wallet_id, action_type, </pre>
--	---

	<pre> balance_before, balance_after, delta) VALUES (v_transaction_id, p_wallet_id, 'CREDIT', v_balance_before, v_balance_after, p_amount); -- 12. Update fund_order SUCCESS. UPDATE fund_orders SET status = 'SUCCESS', gateway_ref = p_gateway_ref WHERE order_id = v_order_id; -- 13. Trả kết quả. p_out_order_id := v_order_id; p_out_transaction_id := v_transaction_id; p_out_status := 'SUCCESS'; EXCEPTION WHEN NO_DATA_FOUND THEN RAISE_APPLICATION_ERROR(-20109, 'Không tìm thấy wallet/payment method/transaction type phù hợp. '); WHEN OTHERS THEN RAISE; END pr_top_up_wallet; / </pre>
	<u>Mô tả mã lệnh:</u> Mã lệnh này có chức năng xử lý nghiệp vụ nạp tiền vào ví. Procedure nhận vào mã ví, phương thức thanh toán, số tiền nạp, khóa chống xử lý trùng idempotency_key, kết quả trả về từ gateway và mô tả giao dịch. Khi được gọi, procedure sẽ kiểm tra số tiền nạp, kiểm tra idempotency_key để tránh xử lý trùng, sau đó xác thực ví và phương thức thanh toán. Nếu gateway trả về thất bại, hệ thống cập nhật lệnh nạp tiền trong bảng fund_orders sang trạng thái FAILED. Nếu gateway trả về thành công, procedure sẽ khóa ví bằng SELECT ... FOR UPDATE, cộng tiền vào bảng

	wallets, tạo bản ghi giao dịch trong bảng transactions, ghi log biến động số dư vào bảng audit_logs, sau đó cập nhật fund_orders sang trạng thái SUCCESS. Procedure này giúp đảm bảo thao tác nạp tiền được xử lý đầy đủ, không bị trùng lặp và có lưu lịch sử biến động số dư.
2.9	PR_WITHDRAW_REQUEST <u>Mã lệnh PL/SQL:</u> <pre> CREATE OR REPLACE PROCEDURE pr_withdraw_request(p_wallet_id IN NUMBER, p_method_id IN NUMBER, p_amount IN NUMBER, p_pin_code IN VARCHAR2, p_idempotency_key IN VARCHAR2 DEFAULT NULL, p_description IN VARCHAR2 DEFAULT NULL, p_out_order_id OUT NUMBER, p_out_transaction_id OUT NUMBER, p_out_reference_code OUT VARCHAR2, p_out_status OUT VARCHAR2) IS -- Thông tin ví v_wallet_status wallets.wallet_status%TYPE; v_wallet_balance wallets.balance%TYPE; v_wallet_user_id wallets.user_id%TYPE; v_wallet_pin wallets.pin_code%TYPE; -- Thông tin user v_user_is_active users.is_active%TYPE; v_kyc_status users.kyc_status%TYPE; v_kyc_level VARCHAR2(20); -- Thông tin payment method v_method_user_id payment_methods.user_id%TYPE; v_method_verified payment_methods.is_verified%TYPE; </pre>

	<pre> v_method_active payment_methods.is_active%TYPE; -- Hạn mức v_limit_id transaction_limits.limit_id%TYPE; -- Phí và số tiền v_fee_amount NUMBER(18,4) := 0; v_total_amount NUMBER(18,4) := 0; v_risk_score NUMBER(4,2) := 0; -- Idempotency v_existing_order_id fund_orders.order_id%TYPE; v_existing_status fund_orders.status%TYPE; -- Kết quả v_type_id transaction_types.type_id%TYPE; v_order_id fund_orders.order_id%TYPE; v_transaction_id transactions.transaction_id%TYPE; v_reference_code transactions.reference_code%TYPE; v_description transactions.description%TYPE; BEGIN -- 1. Kiểm tra input. IF p_amount IS NULL OR p_amount <= 0 THEN RAISE_APPLICATION_ERROR(-20200, 'Số tiền rút phải lớn hơn 0.');</pre>
	<pre> END IF; -- 2. Nếu có idempotency_key thì kiểm tra trước. IF p_idempotency_key IS NOT NULL THEN BEGIN SELECT order_id, status INTO v_existing_order_id, v_existing_status FROM fund_orders WHERE idempotency_key = p_idempotency_key;</pre>

```

        p_out_order_id := v_existing_order_id;
        p_out_status  := v_existing_status;

BEGIN
    SELECT transaction_id, reference_code
    INTO  p_out_transaction_id, p_out_reference_code
    FROM  transactions
    WHERE sender_wallet_id = p_wallet_id
        AND  description LIKE '%WITHDRAW_ORDER_ID=' ||
v_existing_order_id || '%'
        AND ROWNUM = 1;
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            p_out_transaction_id := NULL;
            p_out_reference_code := NULL;
    END;

    RETURN;
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            NULL;
    END;
END IF;

-- 3. Lock wallet FOR UPDATE.
SELECT balance, wallet_status, user_id, pin_code
INTO  v_wallet_balance, v_wallet_status, v_wallet_user_id, v_wallet_pin
FROM  wallets
WHERE wallet_id = p_wallet_id
FOR UPDATE;

-- 4. Kiểm tra wallet ACTIVE.
IF v_wallet_status != 'ACTIVE' THEN

```

	<pre> RAISE_APPLICATION_ERROR(-20201, 'Vị rút tiền không ở trạng thái ACTIVE.');</pre> <pre> END IF;</pre> -- 5. Kiểm tra PIN giao dịch. <pre> IF v_wallet_pin IS NULL OR v_wallet_pin != p_pin_code THEN RAISE_APPLICATION_ERROR(-20202, 'Mã PIN giao dịch không đúng.');</pre> <pre> END IF;</pre> -- 6. Kiểm tra user active và KYC. <pre> SELECT is_active, kyc_status INTO v_user_is_active, v_kyc_status FROM users WHERE user_id = v_wallet_user_id;</pre> <pre> IF v_user_is_active != 1 THEN RAISE_APPLICATION_ERROR(-20203, 'Tài khoản user đang không active.');</pre> <pre> END IF;</pre> <pre> v_kyc_level := CASE WHEN v_kyc_status = 'VERIFIED' THEN 'VERIFIED' ELSE 'UNVERIFIED' END;</pre> -- 7. Kiểm tra payment_method thuộc cùng user, verified và active. <pre> SELECT user_id, is_verified, is_active INTO v_method_user_id, v_method_verified, v_method_active FROM payment_methods WHERE method_id = p_method_id;</pre> <pre> IF v_method_user_id != v_wallet_user_id THEN RAISE_APPLICATION_ERROR(-20204, 'Payment method không thuộc cùng user với wallet.');</pre>
--	--

	<pre> END IF; IF v_method_verified != 1 THEN RAISE_APPLICATION_ERROR(-20205, 'Payment method chưa được xác thực.');</pre> <pre> END IF; IF v_method_active != 1 THEN RAISE_APPLICATION_ERROR(-20206, 'Payment method đang không active.');</pre> <pre> END IF; -- 8. Lấy type WITHDRAW và limit_id. SELECT type_id INTO v_type_id FROM transaction_types WHERE type_code = 'WITHDRAW'; SELECT limit_id INTO v_limit_id FROM transaction_limits WHERE kyc_level = v_kyc_level AND ROWNUM = 1; -- 9. Tính phí và total_amount. total_amount chỉ là biến xử lý, không lưu DB. v_fee_amount := fn_real_fee(v_type_id, p_amount); v_total_amount := p_amount + v_fee_amount; -- 10. Kiểm tra balance >= total_amount. IF v_wallet_balance < v_total_amount THEN RAISE_APPLICATION_ERROR(-20207, 'Số dư ví không đủ để rút tiền. Số dư: ' v_wallet_balance </pre>
--	---

	<pre> ', cần: ' v_total_amount); END IF; -- 11. Kiểm tra hạn mức KYC. IF fn_check_transaction_limit(p_wallet_id, v_type_id, p_amount) = 0 THEN RAISE_APPLICATION_ERROR(-20208, 'Giao dịch rút tiền vượt hạn mức KYC/ngày.');</pre> <pre> END IF; -- 12. Tính risk_score heuristic để ghi vào mô tả xử lý. v_risk_score := fn_risk_score(p_wallet_id, v_type_id, p_amount); -- 13. Tạo fund_order PENDING. INSERT INTO fund_orders (wallet_id, method_id, order_type, amount, status, idempotency_key, created_at) VALUES (p_wallet_id, p_method_id, 'WITHDRAW', p_amount, 'PENDING', p_idempotency_key, SYSTIMESTAMP) RETURNING order_id INTO v_order_id; -- 14. Sinh mã giao dịch. v_reference_code := fn_generate_code('WDR'); v_description := NVL(p_description, 'Yêu cầu rút tiền') ' WITHDRAW_ORDER_ID=' v_order_id ' RISK_SCORE=' TO_CHAR(v_risk_score); -- 15. Tạo transaction WITHDRAW status PENDING. INSERT INTO transactions (type_id, sender_wallet_id, receiver_wallet_id,</pre>
--	--

	<pre> limit_id, amount, fee_amount, status, reference_code, step, description, created_at) VALUES (v_type_id, p_wallet_id, NULL, v_limit_id, p_amount, v_fee_amount, 'PENDING', v_reference_code, TO_NUMBER(TO_CHAR(SYSTIMESTAMP, 'HH24')), v_description, SYSTIMESTAMP) RETURNING transaction_id INTO v_transaction_id; -- 16. Trừ tiền ví, bao gồm amount + fee_amount. UPDATE wallets SET balance = balance - v_total_amount WHERE wallet_id = p_wallet_id; -- 17. Ghi audit_logs DEBIT. INSERT INTO audit_logs (transaction_id, wallet_id, action_type, balance_before, balance_after, delta) VALUES (v_transaction_id, p_wallet_id, 'DEBIT', v_wallet_balance,</pre>
--	--

	<pre> v_wallet_balance - v_total_amount, v_total_amount); -- 18. Trả kết quả. p_out_order_id := v_order_id; p_out_transaction_id := v_transaction_id; p_out_reference_code := v_reference_code; p_out_status := 'PENDING'; EXCEPTION WHEN NO_DATA_FOUND THEN RAISE_APPLICATION_ERROR(-20209, 'Không tìm thấy dữ liệu cần thiết cho yêu cầu rút tiền.');</pre> WHEN OTHERS THEN RAISE; END pr_withdraw_request; /
	<u>Mô tả mã lệnh:</u> Mã lệnh này có chức năng tạo yêu cầu rút tiền từ ví về phương thức thanh toán đã liên kết. Procedure nhận vào mã ví, mã phương thức thanh toán, số tiền rút, mã PIN, idempotency_key và mô tả giao dịch. Procedure sẽ kiểm tra số tiền rút, kiểm tra chống xử lý trùng, sau đó khóa ví để đảm bảo số dư không bị thay đổi đồng thời. Tiếp theo, mã lệnh kiểm tra trạng thái ví, mã PIN, trạng thái tài khoản người dùng, phương thức thanh toán, hạn mức giao dịch và số dư hiện tại. Sau khi hợp lệ, procedure tính phí rút tiền, tính điểm rủi ro, tạo bản ghi trong fund_orders, tạo giao dịch WITHDRAW trong transactions, trừ tiền trong bảng wallets và ghi một dòng DEBIT vào audit_logs. Kết quả của procedure là một yêu cầu rút tiền ở trạng thái PENDING, chờ bước xác nhận từ gateway hoặc ngân hàng.
2.10	PR_CONFIRM_WITHDRAW <u>Mã lệnh PL/SQL:</u> CREATE OR REPLACE PROCEDURE pr_confirm_withdraw(

	<pre> p_order_id IN NUMBER, p_transaction_id IN NUMBER, p_gateway_success IN NUMBER, p_gateway_ref IN VARCHAR2 DEFAULT NULL, p_out_status OUT VARCHAR2) IS -- Fund order v_order_wallet_id fund_orders.wallet_id%TYPE; v_order_amount fund_orders.amount%TYPE; v_order_status fund_orders.status%TYPE; -- Transaction v_trans_wallet_id transactions.sender_wallet_id%TYPE; v_trans_amount transactions.amount%TYPE; v_trans_fee transactions.fee_amount%TYPE; v_trans_status transactions.status%TYPE; v_refund_amount NUMBER(18,4) := 0; -- Wallet v_wallet_balance wallets.balance%TYPE; v_balance_after wallets.balance%TYPE; BEGIN -- 1. Lock fund_order. SELECT wallet_id, amount, status INTO v_order_wallet_id, v_order_amount, v_order_status FROM fund_orders WHERE order_id = p_order_id FOR UPDATE; -- 2. Lock transaction. SELECT sender_wallet_id, amount, fee_amount, status INTO v_trans_wallet_id, v_trans_amount, v_trans_fee, v_trans_status </pre>
--	---

	<pre> FROM transactions WHERE transaction_id = p_transaction_id FOR UPDATE; -- 3. Kiểm tra order và transaction khớp nhau. IF v_order_wallet_id != v_trans_wallet_id THEN RAISE_APPLICATION_ERROR(-20300, 'fund_order và transaction không cùng wallet.');</pre> END IF; <pre> IF v_order_amount != v_trans_amount THEN RAISE_APPLICATION_ERROR(-20301, 'Số tiền fund_order và transaction không khớp.');</pre> END IF; -- 4. Nếu đã được xử lý trước đó thì trả trạng thái hiện tại, tránh hoàn tiền/cập nhật trùng. <pre> IF v_order_status != 'PENDING' OR v_trans_status != 'PENDING' THEN p_out_status := v_trans_status; RETURN; END IF;</pre> -- 5. Gateway/bank thành công. <pre> IF NVL(p_gateway_success, 0) = 1 THEN UPDATE fund_orders SET status = 'SUCCESS', gateway_ref = p_gateway_ref WHERE order_id = p_order_id; UPDATE transactions SET status = 'COMPLETED' WHERE transaction_id = p_transaction_id; p_out_status := 'COMPLETED';</pre>
--	---

```

RETURN;
END IF;

-- 6. Gateway/bank thất bại: hoàn lại amount + fee_amount.
v_refund_amount := v_trans_amount + NVL(v_trans_fee, 0);

SELECT balance
INTO v_wallet_balance
FROM wallets
WHERE wallet_id = v_trans_wallet_id
FOR UPDATE;

UPDATE wallets
SET balance = balance + v_refund_amount
WHERE wallet_id = v_trans_wallet_id;

v_balance_after := v_wallet_balance + v_refund_amount;

-- 7. Ghi audit_logs CREDIT cho phần hoàn lại.
INSERT INTO audit_logs (
    transaction_id, wallet_id, action_type,
    balance_before, balance_after, delta
) VALUES (
    p_transaction_id, v_trans_wallet_id, 'CREDIT',
    v_wallet_balance, v_balance_after, v_refund_amount
);

-- 8. Update status FAILED.
UPDATE fund_orders
SET status = 'FAILED',
    gateway_ref = p_gateway_ref
WHERE order_id = p_order_id;

UPDATE transactions

```

	<pre> SET status = 'FAILED' WHERE transaction_id = p_transaction_id; p_out_status := 'FAILED'; EXCEPTION WHEN NO_DATA_FOUND THEN RAISE_APPLICATION_ERROR(-20309, 'Không tìm thấy fund_order/transaction/wallet khi xác nhận rút tiền.');</pre> WHEN OTHERS THEN <pre> RAISE;</pre> END pr_confirm_withdraw; /
	<u>Mô tả mã lệnh:</u> Mã lệnh này có chức năng xác nhận kết quả rút tiền sau khi có phản hồi từ gateway hoặc ngân hàng. Procedure nhận vào mã order, mã transaction, kết quả xử lý từ gateway và mã tham chiếu gateway. Khi thực hiện, procedure khóa bản ghi trong fund_orders và transactions để tránh xác nhận trùng. Sau đó, procedure kiểm tra order và transaction có khớp nhau về ví, số tiền và trạng thái hay không. Nếu giao dịch đã được xử lý trước đó, procedure trả về trạng thái hiện tại và không xử lý lại. Nếu gateway xác nhận thành công, procedure cập nhật fund_orders thành SUCCESS và transactions thành COMPLETED. Nếu gateway xác nhận thất bại, procedure hoàn lại số tiền đã trừ vào ví, ghi một dòng CREDIT vào audit_logs, đồng thời cập nhật fund_orders và transactions sang trạng thái FAILED. Procedure này giúp đảm bảo yêu cầu rút tiền được xác nhận đúng trạng thái và tránh lỗi hoàn tiền hoặc xử lý lặp nhiều lần.
2.11	PR_TRANSFER_MONEY <u>Mã lệnh PL/SQL:</u> <pre> CREATE OR REPLACE PROCEDURE pr_transfer_money(p_sender_wallet_id IN NUMBER, p_receiver_wallet_id IN NUMBER, p_amount IN NUMBER,</pre>

<pre> p_pin_code IN VARCHAR2, p_voucher_code IN VARCHAR2 DEFAULT NULL, p_description IN VARCHAR2 DEFAULT NULL, p_out_transaction_id OUT NUMBER, p_out_reference_code OUT VARCHAR2, p_out_status OUT VARCHAR2) IS -- Thông tin ví v_sender_status wallets.wallet_status%TYPE; v_sender_balance wallets.balance%TYPE; v_sender_user_id wallets.user_id%TYPE; v_sender_pin wallets.pin_code%TYPE; v_receiver_status wallets.wallet_status%TYPE; v_receiver_balance wallets.balance%TYPE; v_receiver_user_id wallets.user_id%TYPE; -- Thông tin user v_sender_is_active users.is_active%TYPE; v_kyc_status users.kyc_status%TYPE; v_kyc_level VARCHAR2(20); -- Phí và số tiền v_fee_amount NUMBER(18,4) := 0; v_net_fee NUMBER(18,4) := 0; -- Phí sau khi trừ voucher giảm giá v_discount_amount NUMBER(18,4) := 0; v_total_deduct NUMBER(18,4); -- Voucher v_voucher_id vouchers.voucher_id%TYPE; v_discount_type vouchers.discount_type%TYPE; v_discount_value vouchers.discount_value%TYPE; v_min_order_value vouchers.min_order_value%TYPE; </pre>
--

	<pre> v_max_discount vouchers.max_discount%TYPE; -- Hạn mức KYC v_limit_id transaction_limits.limit_id%TYPE; v_max_per_trans transaction_limits.max_amount_per_trans%TYPE; v_max_per_day transaction_limits.max_amount_per_day%TYPE; v_max_trans_day transaction_limits.max_trans_per_day%TYPE; v_total_today NUMBER(18,4) := 0; v_count_today NUMBER(10) := 0; -- Kết quả v_transaction_id transactions.transaction_id%TYPE; v_reference_code transactions.reference_code%TYPE; -- Hằng số TRANSFER C_TYPE_TRANSFER CONSTANT NUMBER(5) := 3; BEGIN --1. Lấy thông tin ví gửi và ví nhận SELECT balance, wallet_status, user_id, pin_code INTO v_sender_balance, v_sender_status, v_sender_user_id, v_sender_pin FROM wallets WHERE wallet_id = p_sender_wallet_id; SELECT balance, wallet_status, user_id INTO v_receiver_balance, v_receiver_status, v_receiver_user_id FROM wallets WHERE wallet_id = p_receiver_wallet_id; -- 2-3. SELECT ví gửi và ví nhận FOR UPDATE IF p_sender_wallet_id < p_receiver_wallet_id THEN -- Lock ví gửi trước </pre>
--	--

	<pre> SELECT balance, wallet_status INTO v_sender_balance, v_sender_status FROM wallets WHERE wallet_id = p_sender_wallet_id FOR UPDATE; -- Lock ví nhận sau SELECT balance, wallet_status INTO v_receiver_balance, v_receiver_status FROM wallets WHERE wallet_id = p_receiver_wallet_id FOR UPDATE; ELSE -- Lock ví nhận trước SELECT balance, wallet_status INTO v_receiver_balance, v_receiver_status FROM wallets WHERE wallet_id = p_receiver_wallet_id FOR UPDATE; -- Lock ví gửi sau SELECT balance, wallet_status INTO v_sender_balance, v_sender_status FROM wallets WHERE wallet_id = p_sender_wallet_id FOR UPDATE; END IF; -- 4. Kiểm tra ví gửi ACTIVE IF v_sender_status != 'ACTIVE' THEN RAISE_APPLICATION_ERROR(-20010, 'Ví người gửi không ở trạng thái ACTIVE nha bạn. Trạng thái hiện tại: ' v_sender_status </pre>
--	--

	<pre>); END IF; -- 5. Kiểm tra ví nhận ACTIVE IF v_receiver_status != 'ACTIVE' THEN RAISE_APPLICATION_ERROR(-20011, 'Ví người nhận không ở trạng thái ACTIVE nha bạn. Trạng thái hiện tại: ' v_receiver_status); END IF; -- 6. Kiểm tra sender_wallet_id <> receiver_wallet_id IF p_sender_wallet_id = p_receiver_wallet_id THEN RAISE_APPLICATION_ERROR(-20012, 'Ví gửi và ví nhận không được trùng nhau nha bạn.');</pre> <pre> END IF; -- 7. Kiểm tra PIN giao dịch IF v_sender_pin IS NULL OR v_sender_pin != p_pin_code THEN RAISE_APPLICATION_ERROR(-20013, 'Mã PIN không đúng nha bạn.');</pre> <pre> END IF; -- 8. Kiểm tra user active SELECT is_active, kyc_status INTO v_sender_is_active, v_kyc_status FROM users WHERE user_id = v_sender_user_id; IF v_sender_is_active = 0 THEN RAISE_APPLICATION_ERROR(-20014, 'Tài khoản người gửi đã bị khóa rồi nha bạn.');</pre> <pre> END IF; </pre>
--	--

```

-- 9. Tính fee_amount dựa trên transaction_fees
v_fee_amount := fn_real_fee(C_TYPE_TRANSFER, p_amount);

-- 10. Kiểm tra voucher
v_discount_amount := 0;
v_voucher_id      := NULL;

IF p_voucher_code IS NOT NULL THEN
    -- Thông tin voucher hợp lệ là còn hạn, còn lượt và active
    BEGIN
        SELECT voucher_id, discount_type, discount_value,
               min_order_value, max_discount
        INTO   v_voucher_id, v_discount_type, v_discount_value,
               v_min_order_value, v_max_discount
        FROM   vouchers
        WHERE  code          = p_voucher_code
               AND is_active = 1
               AND amount_vouchers > 0
               AND (valid_until IS NULL OR valid_until >= SYSTIMESTAMP);
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            RAISE_APPLICATION_ERROR(-20015, 'Voucher không hợp lệ
hoặc đã hết hạn hoặc đã hết lượt sử dụng nha bạn.');
```

END;

```

    -- Kiểm tra số đơn hàng tối thiểu để áp dụng voucher
    IF v_min_order_value IS NOT NULL AND p_amount <
v_min_order_value THEN
        RAISE_APPLICATION_ERROR(
            -20016,
            'Số tiền giao dịch (' || p_amount || ') chưa đạt giá trị tối thiểu để dùng
voucher nha bạn (' || v_min_order_value || ').'
```

);

```

    END IF;

```

	<pre> -- Tính giảm giá theo loại voucher IF v_discount_type = 'PERCENTAGE' THEN v_discount_amount := p_amount * v_discount_value; ELSE v_discount_amount := v_discount_value; END IF; -- Giới hạn giảm giá tối đa IF v_max_discount IS NOT NULL AND v_discount_amount > v_max_discount THEN v_discount_amount := v_max_discount; END IF; IF v_discount_amount > v_fee_amount THEN v_discount_amount := v_fee_amount; END IF; END IF; -- 11. Tính total_amount = amount + fee_amount - discount_amount (voucher). v_net_fee := v_fee_amount - v_discount_amount; v_total_deduct := p_amount + v_net_fee; -- 12. Kiểm tra balance ví gửi >= total_amount IF v_sender_balance < v_total_deduct THEN RAISE_APPLICATION_ERROR(-20017, 'Số dư ví không đủ nha bạn. Số dư hiện tại: ' v_sender_balance ', cần thanh toán nha bạn: ' v_total_deduct); END IF; -- 13. Kiểm tra hạn mức KYC </pre>
--	--

<pre> v_kyc_level := CASE WHEN v_kyc_status = 'VERIFIED' THEN 'VERIFIED' ELSE 'UNVERIFIED' END; SELECT limit_id, max_amount_per_trans, max_amount_per_day, max_trans_per_day INTO v_limit_id, v_max_per_trans, v_max_per_day, v_max_trans_day FROM transaction_limits WHERE kyc_level = v_kyc_level AND ROWNUM = 1; IF p_amount > v_max_per_trans THEN RAISE_APPLICATION_ERROR(-20018, 'Số tiền vượt hạn mức tối đa mỗi giao dịch nha bạn: ' v_max_per_trans); END IF; -- Tổng giao dịch TRANSFER trong ngày hôm nay của ví gửi SELECT NVL(SUM(amount), 0), COUNT(*) INTO v_total_today, v_count_today FROM transactions WHERE sender_wallet_id = p_sender_wallet_id AND type_id = C_TYPE_TRANSFER AND status IN ('COMPLETED', 'PENDING') AND TRUNC(created_at) = TRUNC(SYSDATE); -- Kiểm tra tổng giao dịch ngày + amount <= max_amount_per_day IF v_total_today + p_amount > v_max_per_day THEN RAISE_APPLICATION_ERROR(-20019, 'Vượt hạn mức giao dịch ngày nha bạn. Còn có thể giao dịch: ' GREATEST(v_max_per_day - v_total_today, 0)); </pre>

	<pre> END IF; -- Kiểm tra số giao dịch ngày + 1 <= max_trans_per_day IF v_count_today + 1 > v_max_trans_day THEN RAISE_APPLICATION_ERROR(-20020, 'Đã đạt giới hạn số lượng giao dịch trong ngày nha bạn (' v_max_trans_day ' lần).'); END IF; -- 14. Sinh reference_code v_reference_code := fn_generate_code('TXN'); -- 15. Tạo transaction status PROCESSING hoặc COMPLETED. INSERT INTO transactions (type_id, sender_wallet_id, receiver_wallet_id, limit_id, voucher_id, amount, fee_amount, status, reference_code, description, created_at) VALUES (C_TYPE_TRANSFER, p_sender_wallet_id, p_receiver_wallet_id, v_limit_id, v_voucher_id, p_amount, </pre>
--	--

	<pre> v_net_fee, 'COMPLETED', v_reference_code, p_description, SYSTIMESTAMP) RETURNING transaction_id INTO v_transaction_id; -- 16. Trừ tiền ví gửi UPDATE wallets SET balance = balance - v_total_deduct WHERE wallet_id = p_sender_wallet_id; -- 17. Cộng tiền ví nhận UPDATE wallets SET balance = balance + p_amount WHERE wallet_id = p_receiver_wallet_id; -- 18. Ghi audit log DEBIT cho ví gửi INSERT INTO audit_logs (transaction_id, wallet_id, action_type, balance_before, balance_after, delta) VALUES (v_transaction_id, p_sender_wallet_id, 'DEBIT', v_sender_balance, v_sender_balance - v_total_deduct, v_total_deduct); -- 19. Ghi audit log CREDIT cho ví nhận INSERT INTO audit_logs (transaction_id, wallet_id, action_type, balance_before, balance_after, delta) VALUES (v_transaction_id, p_receiver_wallet_id, 'CREDIT', </pre>
--	--

	<pre> v_receiver_balance, v_receiver_balance + p_amount, p_amount); IF v_voucher_id IS NOT NULL THEN UPDATE vouchers SET amount_vouchers = amount_vouchers - 1 WHERE voucher_id = v_voucher_id; END IF; -- 20. Kiểm tra fraud rule p_out_transaction_id := v_transaction_id; p_out_reference_code := v_reference_code; p_out_status := 'COMPLETED'; EXCEPTION WHEN NO_DATA_FOUND THEN RAISE_APPLICATION_ERROR(-20009, 'Không tìm thấy ví nha bạn.');</pre>
	<u>Mô tả mã lệnh:</u> Mã lệnh này có chức năng xử lý nghiệp vụ chuyển tiền giữa hai ví trong hệ thống, bao gồm kiểm tra số dư, kiểm tra trạng thái ví, kiểm tra PIN, áp dụng phí giao dịch, áp dụng voucher giảm phí, kiểm tra hạn mức giao dịch và ghi nhận lịch sử giao dịch. Khi được gọi, procedure sẽ lần lượt lấy thông tin ví gửi và ví nhận, sau đó khóa hai ví bằng SELECT ... FOR UPDATE theo thứ tự ID để tránh xung đột đồng thời. Tiếp theo, hệ thống kiểm tra ví gửi và ví nhận có đang ở trạng thái ACTIVE hay không, kiểm tra hai ví có bị trùng nhau không, kiểm tra mã PIN giao dịch và xác nhận tài khoản người gửi còn hoạt động.

	Sau đó, procedure tính phí giao dịch bằng function fn_real_fee, rồi kiểm tra voucher nếu người dùng nhập mã voucher. Nếu voucher hợp lệ, hệ thống sẽ xét điều kiện áp dụng, tính mức giảm giá, giới hạn giảm tối đa và đảm bảo số tiền giảm không vượt quá phí giao dịch. Tiếp theo, procedure tính tổng số tiền cần trừ từ ví gửi, gồm số tiền chuyển cộng với phí sau giảm. Nếu số dư không đủ thì hệ thống báo lỗi. Procedure cũng kiểm tra hạn mức giao dịch theo KYC, bao gồm giới hạn cho mỗi giao dịch, tổng số tiền trong ngày và số lượng giao dịch trong ngày. Sau khi tất cả điều kiện hợp lệ, procedure sinh mã tham chiếu giao dịch, tạo bản ghi trong bảng transactions, cập nhật số dư ví gửi và ví nhận, ghi nhận biến động vào bảng audit_logs, đồng thời giảm số lượt sử dụng voucher nếu có áp dụng. Procedure này đảm bảo nghiệp vụ chuyển tiền được xử lý đầy đủ, an toàn, có kiểm tra hạn mức, có ghi nhận lịch sử và hạn chế lỗi do giao dịch đồng thời.
2.12	PR_REVERSAL
	<u>Mã lệnh PL/SQL:</u> <pre> CREATE OR REPLACE PROCEDURE pr_reversal(p_original_transaction_id IN NUMBER, p_description IN VARCHAR2 DEFAULT NULL, p_out_transaction_id OUT NUMBER, p_out_reference_code OUT VARCHAR2, p_out_status OUT VARCHAR2) IS -- Giao dịch gốc v_orig_type_id transactions.type_id%TYPE; v_orig_sender_wallet_id transactions.sender_wallet_id%TYPE; v_orig_receiver_wallet_id transactions.receiver_wallet_id%TYPE; v_orig_amount transactions.amount%TYPE; v_orig_status transactions.status%TYPE; -- Ví của giao dịch đảo ngược v_rev_sender_wallet_id NUMBER; v_rev_receiver_wallet_id NUMBER; </pre>

```

-- Số dư ví lúc lock
v_rev_sender_balance    wallets.balance%TYPE;
v_rev_receiver_balance  wallets.balance%TYPE;

-- Kiểm tra coi đảo ngược chưa
v_reversal_count        NUMBER(10);

-- Kết quả
v_transaction_id        transactions.transaction_id%TYPE;
v_reference_code        transactions.reference_code%TYPE;

-- Hằng số REVERSAL
C_TYPE_REVERSAL CONSTANT NUMBER(5) := 5;

BEGIN
  -- 1. Tìm transaction gốc
  BEGIN
    SELECT type_id, sender_wallet_id, receiver_wallet_id, amount, status
    INTO  v_orig_type_id, v_orig_sender_wallet_id,
    v_orig_receiver_wallet_id,
    v_orig_amount, v_orig_status
    FROM  transactions
    WHERE transaction_id = p_original_transaction_id;
  EXCEPTION
    WHEN NO_DATA_FOUND THEN
      RAISE_APPLICATION_ERROR(
        -20030,
        'Không tìm thấy giao dịch gốc nha bạn: ' ||
        p_original_transaction_id
      );
  END;

  -- 2. Kiểm tra transaction gốc status = COMPLETED

```

```

IF v_orig_status != 'COMPLETED' THEN
    RAISE_APPLICATION_ERROR(
        -20031,
        'Chỉ có thể đảo ngược giao dịch có trạng thái COMPLETED nha
bạn. ' ||
        'Trạng thái hiện tại: ' || v_orig_status
    );
END IF;

-- 3. Kiểm tra transaction gốc chưa từng bị REVERSED
SELECT COUNT(*)
INTO v_reversal_count
FROM transactions
WHERE original_transaction_id = p_original_transaction_id
    AND type_id = C_TYPE_REVERSAL
    AND status = 'COMPLETED';

IF v_reversal_count > 0 THEN
    RAISE_APPLICATION_ERROR(
        -20032,
        'Giao dịch ID ' || p_original_transaction_id || ' đã được đảo ngược rồi
nha bạn.'
    );
END IF;

-- 4. Lock các ví liên quan FOR UPDATE
v_rev_sender_wallet_id := v_orig_receiver_wallet_id;
v_rev_receiver_wallet_id := v_orig_sender_wallet_id;

-- Lock theo thứ tự ID
IF v_rev_sender_wallet_id < v_rev_receiver_wallet_id THEN
    SELECT balance INTO v_rev_sender_balance
    FROM wallets WHERE wallet_id = v_rev_sender_wallet_id FOR
UPDATE;

```

```

SELECT balance INTO v_rev_receiver_balance
FROM wallets WHERE wallet_id = v_rev_receiver_wallet_id FOR
UPDATE;
ELSE
SELECT balance INTO v_rev_receiver_balance
FROM wallets WHERE wallet_id = v_rev_receiver_wallet_id FOR
UPDATE;

SELECT balance INTO v_rev_sender_balance
FROM wallets WHERE wallet_id = v_rev_sender_wallet_id FOR
UPDATE;
END IF;

```

-- 5. Xác định dòng tiền đảo ngược

-- - Nếu giao dịch gốc là TRANSFER:

-- ví nhận cũ trở thành sender

-- ví gửi cũ trở thành receiver

-- - Nếu giao dịch gốc là WITHDRAW:

-- hoàn lại tiền vào ví gửi

-- - Nếu giao dịch gốc là PAYMENT:

-- hoàn tiền từ merchant/user nhận về người trả

v_rev_sender_wallet_id := v_orig_receiver_wallet_id;

v_rev_receiver_wallet_id := v_orig_sender_wallet_id;

-- 6. Kiểm tra ví bị thu hồi có đủ tiền nếu cần

IF v_rev_sender_balance < v_orig_amount THEN

RAISE_APPLICATION_ERROR(

-20033,

'Ví người nhận (wallet_id=' || v_rev_sender_wallet_id || ') không đủ
số dư để hoàn tiền nha. ' ||

'Số dư: ' || v_rev_sender_balance || ', cần hoàn: ' || v_orig_amount

);

END IF;

	<pre> v_reference_code := fn_generate_code('REV'); -- 7. Tạo transaction mới type REVERSAL hoặc REFUND. -- 8. original_transaction_id = transaction_id gốc INSERT INTO transactions (type_id, sender_wallet_id, receiver_wallet_id, original_transaction_id, amount, fee_amount, status, reference_code, description, created_at) VALUES (C_TYPE_REVERSAL, v_rev_sender_wallet_id, v_rev_receiver_wallet_id, p_original_transaction_id, v_orig_amount, 0, 'COMPLETED', v_reference_code, NVL(p_description, 'Đảo ngược giao dịch #' p_original_transaction_id), SYSTIMESTAMP) RETURNING transaction_id INTO v_transaction_id; -- 9. Cập nhật số dư UPDATE wallets SET balance = balance - v_orig_amount WHERE wallet_id = v_rev_sender_wallet_id; </pre>
--	---

```

UPDATE wallets
SET  balance = balance + v_orig_amount
WHERE wallet_id = v_rev_receiver_wallet_id;

-- 10. Ghi audit_logs
INSERT INTO audit_logs (
    transaction_id, wallet_id, action_type,
    balance_before, balance_after, delta
) VALUES (
    v_transaction_id, v_rev_sender_wallet_id, 'DEBIT',
    v_rev_sender_balance,
    v_rev_sender_balance - v_orig_amount,
    v_orig_amount
);

INSERT INTO audit_logs (
    transaction_id, wallet_id, action_type,
    balance_before, balance_after, delta
) VALUES (
    v_transaction_id, v_rev_receiver_wallet_id, 'CREDIT',
    v_rev_receiver_balance,
    v_rev_receiver_balance + v_orig_amount,
    v_orig_amount
);

-- 11. Cập nhật transaction gốc status = REVERSED
UPDATE transactions
SET  status      = 'REVERSED',
     updated_at = SYSTIMESTAMP
WHERE transaction_id = p_original_transaction_id;

p_out_transaction_id := v_transaction_id;
p_out_reference_code := v_reference_code;
p_out_status         := 'COMPLETED';

```

	COMMIT; EXCEPTION WHEN OTHERS THEN ROLLBACK; RAISE; END pr_reversal;
	<u>Mô tả mã lệnh:</u> Mã lệnh này có chức năng xử lý đảo ngược một giao dịch đã hoàn tất trong trường hợp cần hoàn tiền hoặc xử lý sự cố hệ thống, đảm bảo tính nguyên tử giữa việc thu hồi tiền từ bên nhận và hoàn lại tiền cho bên gửi. Khi được gọi, procedure sẽ tìm giao dịch gốc theo transaction_id, kiểm tra giao dịch này có tồn tại hay không. Nếu không tìm thấy thì trả lỗi. Sau đó, hệ thống kiểm tra giao dịch gốc phải có trạng thái COMPLETED thì mới được phép đảo ngược. Tiếp theo, procedure kiểm tra giao dịch này đã từng bị đảo ngược trước đó hay chưa để tránh xử lý trùng. Sau khi hợp lệ, procedure xác định hai ví liên quan đến giao dịch đảo ngược bằng cách hoán đổi ví nhận và ví gửi của giao dịch gốc. Hai ví này được khóa theo thứ tự ID bằng SELECT ... FOR UPDATE để tránh deadlock và đảm bảo tính nhất quán dữ liệu. Hệ thống cũng kiểm tra ví phải đủ số dư để hoàn lại số tiền đã giao dịch. Sau đó, procedure sinh mã tham chiếu mới cho giao dịch đảo ngược, tạo một bản ghi giao dịch mới có loại REVERSAL, liên kết với giao dịch gốc thông qua original_transaction_id, rồi trừ tiền ở ví bên nhận cũ và cộng tiền vào ví bên gửi cũ. Đồng thời, hệ thống ghi log biến động số dư vào audit_logs. Cuối cùng, procedure cập nhật giao dịch gốc sang trạng thái REVERSED, trả kết quả ra ngoài và COMMIT toàn bộ thay đổi. Nếu xảy ra bất kỳ lỗi nào, hệ thống sẽ ROLLBACK để đảm bảo dữ liệu không bị cập nhật dang dở. Procedure này giúp hệ thống hoàn tiền hoặc đảo ngược giao dịch một cách an toàn, có kiểm soát và đảm bảo toàn vẹn dữ liệu.
2.13	PR_UPDATE_PIN_CODE <u>Mã lệnh PL/SQL:</u> CREATE OR REPLACE PROCEDURE pr_update_pin_code (p_user_id IN NUMBER, p_pin_code IN VARCHAR2

	<pre>) AS v_user_count NUMBER; BEGIN -- Kiểm tra user có tồn tại hay không SELECT COUNT(*) INTO v_user_count FROM users WHERE user_id = p_user_id; IF v_user_count = 0 THEN RAISE_APPLICATION_ERROR(-20013, 'User không tồn tại'); END IF; -- Cập nhật mã PIN UPDATE wallets SET pin_code = p_pin_code WHERE user_id = p_user_id; COMMIT; END;</pre>
	<u>Mô tả mã lệnh:</u> Mã lệnh này có chức năng cập nhật mã PIN cho ví của người dùng trong bảng "WALLETS". Procedure thực hiện các bước sau: • Kiểm tra USER_ID có tồn tại trong bảng "USERS" hay không. • Nếu không tồn tại, procedure sẽ gây lỗi với mã lỗi -20013 và thông báo: 'User không tồn tại'. • Nếu hợp lệ, procedure sẽ cập nhật PIN_CODE mới cho ví của người dùng trong bảng "WALLETS". • Cuối cùng, procedure thực hiện "COMMIT" để lưu thay đổi vào cơ sở dữ liệu.

3.2.2. Function

STT	FUNCTION
3.1	FN_GENERATE_REF_CODE <u>Mã lệnh PL/SQL:</u> <pre data-bbox="320 434 1390 2016"> CREATE OR REPLACE FUNCTION fn_generate_ref_code RETURN VARCHAR2 IS PRAGMA AUTONOMOUS_TRANSACTION; v_today DATE; v_seq NUMBER; v_ref_code VARCHAR2(50); BEGIN v_today := TRUNC(SYSDATE); UPDATE transaction_ref_sequences SET current_value = current_value + 1, updated_at = SYSTIMESTAMP WHERE ref_date = v_today RETURNING current_value INTO v_seq; IF SQL%ROWCOUNT = 0 THEN v_seq := 1; INSERT INTO transaction_ref_sequences (ref_date, current_value) VALUES (v_today, v_seq); END IF; COMMIT; -- (VD: TXN20231027000001) v_ref_code := 'TXN' TO_CHAR(SYSDATE, 'YYYYMMDD') LPAD(v_seq, 6, '0'); RETURN v_ref_code; </pre>

	EXCEPTION WHEN OTHERS THEN ROLLBACK; RAISE; END; <u>Mô tả mã lệnh:</u> Mã lệnh này có chức năng tạo mã tham chiếu giao dịch (reference code) tự động và duy nhất cho mỗi giao dịch theo từng ngày. Function không nhận tham số đầu vào và được cấu hình hoạt động như một giao dịch độc lập (AUTONOMOUS_TRANSACTION) để không bị ảnh hưởng bởi các giao dịch khác. Khi được gọi, function sẽ lấy ngày hiện tại và tiến hành cập nhật tăng giá trị biến đếm chuỗi (current_value) thêm 1 đơn vị trong bảng "TRANSACTION_REF_SEQUENCES". Nếu trong ngày hiện tại chưa có giao dịch nào phát sinh (lệnh cập nhật không tìm thấy bản ghi), function sẽ tự động chèn mới một bản ghi cho ngày hôm nay với giá trị khởi tạo là 1. Sau khi lưu lại sự thay đổi của biến đếm vào cơ sở dữ liệu (COMMIT), function sẽ tiến hành ghép chuỗi để tạo ra một mã tham chiếu hoàn chỉnh bao gồm tiền tố 'TXN', chuỗi ngày tháng năm định dạng 'YYYYMMDD' và 6 chữ số định danh tăng dần (Ví dụ: TXN20231027000001). Nếu hợp lệ, function trả về chuỗi mã tham chiếu vừa tạo; nếu có bất kỳ lỗi nào xảy ra, hệ thống sẽ hoàn tác (ROLLBACK) các thay đổi của biến đếm và ném lỗi lên hệ thống.
3.2	FN_GET_HASH_FOR_LOGIN <u>Mã lệnh PL/SQL:</u> <pre> CREATE OR REPLACE FUNCTION fn_get_hash_for_login (p_phone IN VARCHAR2) RETURN VARCHAR2 IS v_password_hash users.password_hash%TYPE; v_is_active users.is_active%TYPE; BEGIN -- Truy vấn mã băm mật khẩu và trạng thái dựa vào số điện thoại SELECT password_hash, is_active INTO v_password_hash, v_is_active FROM users </pre>

	<pre> WHERE phone = p_phone; -- Kiểm tra tài khoản có bị vô hiệu hóa không IF v_is_active = 0 THEN RAISE_APPLICATION_ERROR(-20114, 'Lỗi: Tài khoản đã bị khóa hoặc vô hiệu hóa.');</pre> <pre> END IF; -- Trả về mã băm RETURN v_password_hash; EXCEPTION WHEN NO_DATA_FOUND THEN -- Nếu không tìm thấy số điện thoại, trả về NULL -- (báo lỗi chung "Sai số điện thoại hoặc mật khẩu" để bảo mật) RETURN NULL; WHEN OTHERS THEN RAISE; END;</pre>
	<u>Mô tả mã lệnh:</u> Mã lệnh này có chức năng trích xuất chuỗi mật khẩu an toàn phục vụ cho quá trình kiểm tra tính hợp lệ khi người dùng thực hiện đăng nhập. Function nhận vào tham số là số điện thoại của người dùng. Khi được gọi, function sẽ truy vấn bảng "USERS" để lấy ra chuỗi mật khẩu đã được băm (password_hash) cùng với trạng thái hoạt động của tài khoản (is_active) dựa trên số điện thoại được cung cấp. Sau đó, function kiểm tra trạng thái hoạt động này; nếu tài khoản đang bị khóa hoặc vô hiệu hóa (is_active = 0), function sẽ ngay lập tức gây lỗi -20100 để chặn quá trình đăng nhập. Nếu tài khoản ở trạng thái bình thường, function sẽ trả về chuỗi mật khẩu băm để hệ thống bên ngoài tiếp tục đối chiếu. Trong trường hợp số điện thoại truyền vào không tồn tại trong hệ thống, function sẽ bắt ngoại lệ NO_DATA_FOUND và trả về giá trị NULL.
3.3	FN_GET_WALLET_INFO <u>Mã lệnh PL/SQL:</u> <pre> CREATE OR REPLACE FUNCTION fn_get_wallet_info (</pre>

	<pre> p_wallet_id IN NUMBER) RETURN VARCHAR2 AS v_balance wallets.balance%TYPE; v_status wallets.wallet_status%TYPE; BEGIN SELECT balance, wallet_status INTO v_balance, v_status FROM wallets WHERE wallet_id = p_wallet_id; RETURN 'Balance: ' v_balance ' Status: ' v_status; EXCEPTION WHEN NO_DATA_FOUND THEN RETURN 'Wallet không tồn tại'; END; </pre>
	<u>Mô tả mã lệnh:</u> Mã lệnh này có chức năng lấy thông tin ví từ bảng "WALLETS" dựa trên WALLET_ID được cung cấp. Function thực hiện các bước sau: • Truy vấn số dư và trạng thái ví từ bảng "WALLETS" theo WALLET_ID. • Nếu tìm thấy dữ liệu, function sẽ trả về chuỗi thông tin gồm: Số dư ví và Trạng thái ví. • Nếu không tìm thấy ví tương ứng, function sẽ xử lý ngoại lệ "NO_DATA_FOUND" và trả về thông báo: 'Wallet không tồn tại'.
3.4	FN_REAL_FEE <u>Mã lệnh PL/SQL:</u> <pre> CREATE OR REPLACE FUNCTION fn_real_fee(p_type_id IN NUMBER, p_amount IN NUMBER </pre>

	<pre>) RETURN NUMBER IS v_fee_rate NUMBER(5,4) := 0; v_fee_fixed NUMBER(18,4) := 0; v_min_fee NUMBER(18,4) := 0; v_max_fee NUMBER(18,4) := NULL; v_fee_amount NUMBER(18,4); BEGIN BEGIN SELECT fee_rate, fee_fixed, min_fee, max_fee INTO v_fee_rate, v_fee_fixed, v_min_fee, v_max_fee FROM transaction_fees WHERE type_id = p_type_id AND TRUNC(SYSDATE) >= effective_from AND (effective_to IS NULL OR TRUNC(SYSDATE) <= effective_to) AND ROWNUM = 1; EXCEPTION WHEN NO_DATA_FOUND THEN RETURN 0; END; v_fee_amount := (p_amount * v_fee_rate) + v_fee_fixed; IF v_fee_amount < v_min_fee THEN v_fee_amount := v_min_fee; END IF; IF v_max_fee IS NOT NULL AND v_fee_amount > v_max_fee THEN v_fee_amount := v_max_fee; END IF; RETURN v_fee_amount; </pre>
--	--

	END fn_real_fee; <u>Mô tả mã lệnh:</u> Mã lệnh này có chức năng tính toán số tiền phí thực tế phải thu cho một giao dịch dựa trên loại giao dịch và số tiền giao dịch hiện tại. Khi được gọi, function sẽ tra cứu biểu phí tương ứng trong bảng transaction_fees theo type_id và thời gian hiệu lực hiện hành. Nếu không tìm thấy biểu phí phù hợp thì function trả về 0. Nếu có dữ liệu, hệ thống sẽ tính phí theo công thức $\text{fee_rate} * \text{amount} + \text{fee_fixed}$, sau đó so sánh với min_fee và max_fee để đảm bảo số tiền phí nằm trong giới hạn cho phép.
3.5	FN_CHECK_TRANSACTION_LIMIT <u>Mã lệnh PL/SQL:</u> <pre> CREATE OR REPLACE FUNCTION fn_check_transaction_limit(p_wallet_id IN NUMBER, p_type_id IN NUMBER, p_amount IN NUMBER) RETURN NUMBER IS v_kyc_status users.kyc_status%TYPE; v_kyc_level VARCHAR2(20); v_max_per_trans transaction_limits.max_amount_per_trans%TYPE; v_max_per_day transaction_limits.max_amount_per_day%TYPE; v_max_trans_day transaction_limits.max_trans_per_day%TYPE; v_total_today NUMBER(18,4) := 0; v_count_today NUMBER(10) := 0; BEGIN IF p_amount IS NULL OR p_amount <= 0 THEN RETURN 0; END IF; SELECT u.kyc_status INTO v_kyc_status FROM users u </pre>

	<pre> JOIN wallets w ON u.user_id = w.user_id WHERE w.wallet_id = p_wallet_id; v_kyc_level := CASE WHEN v_kyc_status = 'VERIFIED' THEN 'VERIFIED' ELSE 'UNVERIFIED' END; SELECT max_amount_per_trans, max_amount_per_day, max_trans_per_day INTO v_max_per_trans, v_max_per_day, v_max_trans_day FROM transaction_limits WHERE kyc_level = v_kyc_level AND ROWNUM = 1; IF p_amount > v_max_per_trans THEN RETURN 0; END IF; SELECT NVL(SUM(amount), 0), COUNT(*) INTO v_total_today, v_count_today FROM transactions WHERE sender_wallet_id = p_wallet_id AND type_id = p_type_id AND status IN ('COMPLETED', 'PENDING') AND TRUNC(created_at) = TRUNC(SYSDATE); IF v_total_today + p_amount > v_max_per_day THEN RETURN 0; END IF; IF v_count_today + 1 > v_max_trans_day THEN RETURN 0; END IF; </pre>
--	---

	<pre> RETURN 1; EXCEPTION WHEN NO_DATA_FOUND THEN RETURN 0; END fn_check_transaction_limit;</pre> <u>Mô tả mã lệnh:</u> Mã lệnh này có chức năng kiểm tra giao dịch có vượt hạn mức theo trạng thái KYC của người dùng hay không. Function nhận vào mã ví, loại giao dịch và số tiền giao dịch. Function lấy trạng thái KYC của chủ ví từ bảng users và wallets, sau đó xác định người dùng thuộc nhóm VERIFIED hay UNVERIFIED. Dựa vào nhóm này, function lấy hạn mức tương ứng từ bảng transaction_limits, bao gồm hạn mức mỗi giao dịch, hạn mức tổng tiền trong ngày và số lần giao dịch tối đa trong ngày. Sau đó, function tính tổng số tiền và số lượng giao dịch đã phát sinh trong ngày từ bảng transactions. Nếu giao dịch mới vượt một trong các hạn mức trên, function trả về 0; nếu hợp lệ, function trả về 1
3.6	FN_VALIDATE_VOUCHER <u>Mã lệnh PL/SQL:</u> <pre> CREATE OR REPLACE FUNCTION fn_validate_voucher(p_voucher_code IN VARCHAR2, p_amount IN NUMBER) RETURN NUMBER IS v_count NUMBER(10) := 0; BEGIN IF p_voucher_code IS NULL OR p_amount IS NULL OR p_amount <= 0 THEN RETURN 0; END IF;</pre>

	<pre> SELECT COUNT(*) INTO v_count FROM vouchers WHERE code = p_voucher_code AND is_active = 1 AND amount_vouchers > 0 AND (valid_until IS NULL OR valid_until >= SYSTIMESTAMP) AND (min_order_value IS NULL OR p_amount >= min_order_value); IF v_count > 0 THEN RETURN 1; END IF; RETURN 0; END fn_validate_voucher; </pre>
	<u>Mô tả mã lệnh:</u> Mã lệnh này có chức năng kiểm tra mã voucher có còn hiệu lực và có phù hợp với số tiền giao dịch hiện tại hay không. Function trả về 1 nếu voucher hợp lệ và trả về 0 nếu voucher không hợp lệ. Function nhận vào p_voucher_code là mã voucher và p_amount là số tiền giao dịch. Nếu mã voucher rỗng, số tiền giao dịch rỗng hoặc số tiền giao dịch nhỏ hơn hoặc bằng 0, function sẽ trả về 0. Sau đó, function kiểm tra trong bảng vouchers xem có voucher nào thỏa mãn các điều kiện sau hay không: mã voucher trùng với mã được truyền vào, voucher đang hoạt động với is_active = 1, số lượng voucher còn lại lớn hơn 0, chưa hết hạn sử dụng và số tiền giao dịch đạt giá trị tối thiểu min_order_value nếu voucher có quy định giá trị tối thiểu. Nếu tìm thấy ít nhất một voucher phù hợp, function trả về 1. Ngược lại, function trả về 0.
3.7	FN_VOUCHER_DISCOUNT <u>Mã lệnh PL/SQL:</u> <pre> CREATE OR REPLACE FUNCTION fn_voucher_discount(p_voucher_code IN VARCHAR2, p_amount IN NUMBER </pre>

	<pre>) RETURN NUMBER IS v_discount_type vouchers.discount_type%TYPE; v_discount_value vouchers.discount_value%TYPE; v_max_discount vouchers.max_discount%TYPE; v_discount_amount NUMBER(18,4) := 0; BEGIN IF fn_validate_voucher(p_voucher_code, p_amount) = 0 THEN RETURN 0; END IF; SELECT discount_type, discount_value, max_discount INTO v_discount_type, v_discount_value, v_max_discount FROM vouchers WHERE code = p_voucher_code; IF v_discount_type = 'PERCENTAGE' THEN v_discount_amount := p_amount * NVL(v_discount_value, 0) / 100; ELSE v_discount_amount := NVL(v_discount_value, 0); END IF; IF v_max_discount IS NOT NULL AND v_discount_amount > v_max_discount THEN v_discount_amount := v_max_discount; END IF; IF v_discount_amount > p_amount THEN v_discount_amount := p_amount; END IF; RETURN ROUND(v_discount_amount, 4); </pre>
--	--

	<pre> EXCEPTION WHEN NO_DATA_FOUND THEN RETURN 0; END fn_voucher_discount; </pre> <u>Mô tả mã lệnh:</u> Mã lệnh này có chức năng tính số tiền giảm thực tế từ voucher. Trước hết, function gọi fn_validate_voucher để kiểm tra voucher có hợp lệ hay không. Nếu voucher không hợp lệ, function trả về 0. Nếu voucher hợp lệ, function lấy thông tin giảm giá từ bảng vouchers. Với voucher loại PERCENTAGE, số tiền giảm được tính theo phần trăm của số tiền giao dịch. Với các loại khác, số tiền giảm được lấy theo giá trị cố định. Function tiếp tục kiểm tra mức giảm tối đa max_discount và đảm bảo số tiền giảm không vượt quá tổng số tiền giao dịch. Kết quả được làm tròn 4 chữ số thập phân.
3.8	<pre> FN_REMAINING_MONEY </pre> <u>Mã lệnh PL/SQL:</u> <pre> CREATE OR REPLACE FUNCTION fn_remaining_money(p_wallet_id IN NUMBER, p_type_id IN NUMBER) RETURN NUMBER IS v_kyc_status VARCHAR2(20); v_kyc_level VARCHAR2(20); v_max_amount_per_day NUMBER(18,4); v_max_trans_per_day NUMBER(10); v_total_amount_today NUMBER(18,4) := 0; v_count_today NUMBER(10) := 0; v_remaining NUMBER(18,4); BEGIN SELECT u.kyc_status INTO v_kyc_status FROM users u JOIN wallets w ON u.user_id = w.user_id </pre>

	<pre> WHERE w.wallet_id = p_wallet_id; v_kyc_level := CASE WHEN v_kyc_status = 'VERIFIED' THEN 'VERIFIED' ELSE 'UNVERIFIED' END; SELECT max_amount_per_day, max_trans_per_day INTO v_max_amount_per_day, v_max_trans_per_day FROM transaction_limits WHERE kyc_level = v_kyc_level AND ROWNUM = 1; SELECT NVL(SUM(amount), 0), COUNT(*) INTO v_total_amount_today, v_count_today FROM transactions WHERE sender_wallet_id = p_wallet_id AND type_id = p_type_id AND status IN ('COMPLETED', 'PENDING') AND TRUNC(created_at) = TRUNC(SYSDATE); IF v_count_today >= v_max_trans_per_day THEN RETURN 0; END IF; v_remaining := v_max_amount_per_day - v_total_amount_today; IF v_remaining < 0 THEN v_remaining := 0; END IF; RETURN v_remaining; EXCEPTION </pre>
--	--

	<pre> WHEN NO_DATA_FOUND THEN RETURN 0; END fn_remaining_money; </pre> <u>Mô tả mã lệnh:</u> Mã lệnh này có chức năng tính số tiền còn lại mà người dùng có thể giao dịch trong ngày dựa trên hạn mức KYC và lịch sử giao dịch đã thực hiện. Khi được gọi, function sẽ lấy trạng thái KYC của người dùng thông qua ví, từ đó xác định mức KYC tương ứng là VERIFIED hoặc UNVERIFIED. Sau đó, function tra cứu bảng transaction_limits để lấy hạn mức tối đa theo ngày và số lượng giao dịch tối đa trong ngày. Tiếp theo, hệ thống tính tổng số tiền và số lượng giao dịch của ví trong ngày hiện tại với đúng loại giao dịch được yêu cầu. Nếu số lượng giao dịch đã đạt giới hạn tối đa thì function trả về 0. Nếu chưa đạt giới hạn, function sẽ tính số tiền còn lại bằng cách lấy hạn mức tối đa trong ngày trừ đi tổng số tiền đã giao dịch trong ngày. Trường hợp kết quả âm thì được đưa về 0.
3.9	FN_RISK_SCORE <u>Mã lệnh PL/SQL:</u> <pre> CREATE OR REPLACE FUNCTION fn_risk_score(p_wallet_id IN NUMBER, p_type_id IN NUMBER, p_amount IN NUMBER) RETURN NUMBER IS v_kyc_status users.kyc_status%TYPE; v_kyc_level VARCHAR2(20); v_max_per_trans NUMBER(18,4) := 0; v_max_per_day NUMBER(18,4) := 0; v_max_trans_day NUMBER(10) := 0; v_total_today NUMBER(18,4) := 0; v_count_today NUMBER(10) := 0; v_last_created_at TIMESTAMP; v_minutes_from_last NUMBER(18,4); v_risk_score NUMBER(4,2) := 0; </pre>

```

BEGIN
    IF p_amount IS NULL OR p_amount <= 0 THEN
        RETURN 0;
    END IF;

    SELECT u.kyc_status
    INTO   v_kyc_status
    FROM   users u
    JOIN   wallets w ON u.user_id = w.user_id
    WHERE  w.wallet_id = p_wallet_id;

    v_kyc_level := CASE
        WHEN v_kyc_status = 'VERIFIED' THEN 'VERIFIED'
        ELSE 'UNVERIFIED'
    END;

    SELECT          max_amount_per_trans,          max_amount_per_day,
    max_trans_per_day
    INTO   v_max_per_trans, v_max_per_day, v_max_trans_day
    FROM   transaction_limits
    WHERE  kyc_level = v_kyc_level
    AND    ROWNUM = 1;

    SELECT NVL(SUM(amount), 0), COUNT(*), MAX(created_at)
    INTO   v_total_today, v_count_today, v_last_created_at
    FROM   transactions
    WHERE  sender_wallet_id = p_wallet_id
    AND    type_id          = p_type_id
    AND    status           IN ('COMPLETED', 'PENDING')
    AND    TRUNC(created_at) = TRUNC(SYSDATE);

    -- 1. Giao dịch gần sát hạn mức mỗi lần.
    IF v_max_per_trans > 0 AND p_amount >= v_max_per_trans * 0.8 THEN

```

	<pre> v_risk_score := v_risk_score + 0.25; END IF; -- 2. Tổng tiền trong ngày gần chạm hạn mức ngày. IF v_max_per_day > 0 AND (v_total_today + p_amount) >= v_max_per_day * 0.8 THEN v_risk_score := v_risk_score + 0.20; END IF; -- 3. Số lượng giao dịch trong ngày nhiều. IF v_max_trans_day > 0 AND (v_count_today + 1) >= v_max_trans_day * 0.8 THEN v_risk_score := v_risk_score + 0.20; END IF; -- 4. User chưa VERIFIED. IF v_kyc_status != 'VERIFIED' THEN v_risk_score := v_risk_score + 0.20; END IF; -- 5. Giao dịch liên tiếp trong thời gian ngắn. IF v_last_created_at IS NOT NULL THEN v_minutes_from_last := (CAST(SYSTIMESTAMP AS DATE) - CAST(v_last_created_at AS DATE)) * 24 * 60; IF v_minutes_from_last <= 5 THEN v_risk_score := v_risk_score + 0.15; END IF; END IF; IF v_risk_score > 1 THEN v_risk_score := 1; END IF; </pre>
--	--

	<pre> RETURN ROUND(v_risk_score, 2); EXCEPTION WHEN NO_DATA_FOUND THEN RETURN 0; END fn_risk_score; </pre>
	<u>Mô tả mã lệnh:</u> Mã lệnh này có chức năng tính điểm rủi ro của giao dịch trong khoảng từ 0.00 đến 1.00. Function đánh giá rủi ro dựa trên mã ví, loại giao dịch, số tiền giao dịch, trạng thái KYC và lịch sử giao dịch trong ngày. Function lấy hạn mức giao dịch theo cấp độ KYC, sau đó tính tổng số tiền và số lượng giao dịch trong ngày từ bảng transactions. Điểm rủi ro sẽ tăng nếu giao dịch gần chạm hạn mức mỗi lần, tổng giao dịch trong ngày gần chạm hạn mức ngày, số lần giao dịch trong ngày nhiều, người dùng chưa xác thực KYC hoặc có giao dịch liên tiếp trong thời gian ngắn. Nếu tổng điểm lớn hơn 1, function giới hạn kết quả bằng 1.
3.10	FN_IS_WALLET_ACTIVE <u>Mã lệnh PL/SQL:</u> <pre> CREATE OR REPLACE FUNCTION fn_is_wallet_active (p_wallet_id IN NUMBER) RETURN NUMBER AS v_status wallets.wallet_status%TYPE; BEGIN -- Lấy trạng thái ví SELECT wallet_status INTO v_status FROM wallets WHERE wallet_id = p_wallet_id; -- Kiểm tra trạng thái IF v_status = 'ACTIVE' THEN </pre>

	<pre> RETURN 1; ELSE RETURN 0; END IF; EXCEPTION WHEN NO_DATA_FOUND THEN RETURN 0; END;</pre>
	<u>Mô tả mã lệnh:</u> Mã lệnh này có chức năng kiểm tra trạng thái hoạt động của ví trong bảng "WALLETS" dựa trên WALLET_ID được cung cấp. Function thực hiện các bước sau: • Truy vấn trạng thái ví từ bảng "WALLETS" theo "WALLET_ID". • Kiểm tra trạng thái ví: • Nếu trạng thái là 'ACTIVE', function trả về giá trị 1. • Nếu trạng thái khác 'ACTIVE', function trả về giá trị 0. • Nếu không tìm thấy ví tương ứng, function sẽ xử lý ngoại lệ "NO_DATA_FOUND" và trả về giá trị 0.

CHƯƠNG 4: XỬ LÝ ĐỒNG THỜI

4.1. Giao tác

Transaction (Giao tác) là một đơn vị thao tác luận lý bao gồm một chuỗi các hành động tương tác với cơ sở dữ liệu (đọc, thêm, sửa, xóa). Khi một giao tác được thực thi, hệ thống phải đảm bảo toàn bộ các thao tác bên trong nó đều thành công (Commit), hoặc nếu có bất kỳ lỗi nào xảy ra, toàn bộ sẽ bị hủy bỏ và trạng thái dữ liệu được khôi phục về như trước khi giao tác bắt đầu (Rollback).

Ví dụ: Khi khách hàng A muốn chuyển 100.000 VNĐ vào tài khoản khách hàng B, hệ thống cần thực thi một chuỗi các hành động sau:

- Kiểm tra số dư tài khoản của khách hàng A.
- Trừ 100.000 ở tài khoản của khách hàng A.
- Cộng 100.000 ở tài khoản của khách hàng B.
- Lưu nhật ký giao dịch.

Nếu sau khi hoàn tất hai bước đầu, hệ thống gặp sự cố mạng hoặc lỗi cơ sở dữ liệu và không thể thực hiện bước cộng tiền cho người B thì toàn bộ giao dịch phải bị hủy.

Nếu không sử dụng transaction, người A sẽ bị mất tiền nhưng người B không nhận được, gây thiệt hại cho công ty quản lý ví điện tử cũng như gây phiền toái cho khách hàng.

Một transaction đã được chuyển giao thì không thể bị hủy bỏ. Nếu transaction có sai sót thì cần một transaction khác điều chỉnh lại.

4.2. Vấn đề trong môi trường truy xuất đồng thời

4.2.1. Lost Update (Mất dữ liệu cập nhật)

Lost Update xảy ra khi có nhiều hơn một giao tác cùng đọc và cập nhật trên một bản ghi, kết quả của giao tác ghi sau sẽ đè lên và làm mất kết quả thay đổi của giao tác ghi trước đó.

Tình huống xảy ra như sau:

- Giao tác A truy xuất và cập nhật một dòng dữ liệu.
- Trước khi giao tác A hoàn tất, giao tác B cũng truy xuất cùng dòng dữ liệu đó và thực hiện cập nhật.
- Giao tác B thực hiện commit trước.
- Cuối cùng, khi giao tác A commit, nó đã dựa trên dữ liệu cũ và vô tình ghi đè lên những thay đổi mà giao tác B vừa thực hiện.

Hệ quả: Dữ liệu sau cùng chỉ phản ánh thay đổi của giao tác A, còn thay đổi của B hoàn toàn biến mất.

4.2.2. Non-repeatable Read (Đọc không nhất quán)

Non-repeatable Read xảy ra khi một giao tác đọc một dòng dữ liệu nhiều lần nhưng mỗi lần lại nhận được các giá trị khác nhau do có giao tác khác đã thực hiện cập nhật hoặc xóa dữ liệu đó trong khoảng thời gian giữa hai lần đọc.

Tình huống xảy ra như sau:

- Giao tác A thực hiện đọc một dòng dữ liệu lần thứ nhất.
- Trước khi giao tác A vẫn đang thực thi, giao tác B thực hiện cập nhật hoặc xóa dòng dữ liệu đó và commit thành công.
- Giao tác A thực hiện đọc lại dòng dữ liệu đó lần thứ hai. Lúc này, giá trị nhận được đã bị thay đổi hoặc bản ghi đã mất.

Hệ quả: Giao tác A rơi vào trạng thái không nhất quán vì dữ liệu nguồn thay đổi ngay trong quá trình xử lý của nó. Hiện tượng này đặc biệt nghiêm trọng trong các hệ thống giao dịch tài chính hoặc hệ thống yêu cầu độ chính xác cao.

4.2.3. Phantom Read (Đọc bóng ma)

Phantom Read xảy ra khi một giao tác thực hiện nhiều lần thao tác đọc với cùng một điều kiện truy vấn nhưng tập hợp các dòng dữ liệu trả về lại khác nhau.

Tình huống xảy ra như sau:

- Giao tác A thực hiện một truy vấn đọc một tập hợp các bản ghi thỏa mãn một điều kiện nhất định (ví dụ: WHERE balance > 100000).
- Giao tác B thực hiện chèn một hoặc nhiều bản ghi mới thỏa mãn đúng điều kiện mà giao tác A đang truy vấn và thực hiện commit.
- Giao tác A thực hiện lại truy vấn đó một lần nữa. Lúc này, kết quả trả về xuất hiện thêm những bản ghi mới mà trước đó không có.

Hệ quả: Sự xuất hiện của các “bản ghi bóng ma” khiến các phép toán thống kê của giao tác A bị sai lệch giữa các lần thực thi.

Sự khác biệt giữa Phantom Read và Non-repeatable Read là:

- Non-repeatable Read xảy ra khi dữ liệu đã tồn tại bị thay đổi giữa hai lần đọc.
- Phantom Read xảy ra khi một dòng dữ liệu mới được thêm vào hoặc biến mất trong phạm vi truy vấn, tạo ra những dòng dữ liệu xuất hiện hoặc biến mất bất ngờ giữa các lần đọc.

4.2.4. Dirty Read (Đọc dữ liệu rác)

Dirty Read xảy ra khi một giao tác truy cập và sử dụng trên một đơn vị dữ liệu mà đơn vị dữ liệu này đang được cập nhật bởi một giao tác khác nhưng việc cập nhật đó chưa được commit.

Tình huống xảy ra như sau:

- Giao tác A thực hiện cập nhật một dòng dữ liệu nhưng chưa commit.

- Giao tác B truy xuất và đọc dòng dữ liệu mà giao tác A vừa thay đổi.
- Sau đó, vì xảy ra lỗi nên giao tác A thực hiện lệnh rollback.
- Lúc này, dữ liệu mà giao tác B đã đọc và đang xử lý thực tế không còn tồn tại trong hệ thống.

Hệ quả: Giao tác B thực hiện các logic tính toán dựa trên một giá trị không có thực, dẫn đến sai lệch toàn bộ kết quả phía sau.

4.2.5. Deadlock

Deadlock là một trạng thái xảy ra trong môi trường truy xuất đồng thời, trong đó hai hoặc nhiều giao tác rơi vào tình trạng chờ đợi nhau vô thời hạn. Mỗi giao tác đều đang giữ một tài nguyên mà giao tác kia cần, đồng thời lại chờ đợi tài nguyên mà giao tác kia đang nắm giữ.

Tình huống xảy ra như sau:

- Giao tác A thực hiện khóa dòng dữ liệu 1 (ví dụ: Tài khoản người gửi).
- Giao tác B thực hiện khóa dòng dữ liệu 2 (ví dụ: Tài khoản người nhận).
- Giao tác A cần cập nhật dòng dữ liệu 2 nên gửi yêu cầu khóa dòng 2. Tuy nhiên, dòng này đang bị B giữ khóa độc quyền nên A phải đợi B giải phóng.
- Cùng lúc đó, giao tác B lại cần cập nhật dòng dữ liệu 1 nên gửi yêu cầu khóa dòng 1. Dòng này đang bị A giữ nên B cũng phải đợi A giải phóng.
- Như vậy, A đợi B và B đợi A. Cả hai không bao giờ có thể tiếp tục thực thi hay kết thúc để giải phóng khóa.

Hệ quả: Không giao tác nào có thể hoàn tất, gây ra ngưng trệ hệ thống, làm giảm hiệu suất và thậm chí dẫn đến lỗi toàn cục nếu không được xử lý kịp thời.

4.3. Mức cô lập

4.3.1. Read uncommitted (Đọc dữ liệu chưa xác nhận)

Đây là mức cô lập thấp nhất. Ở mức này, một giao tác có thể đọc được các thay đổi dữ liệu của một giao tác khác ngay cả khi giao tác đó chưa commit. Điều này cho phép hiện tượng Dirty Read xảy ra.

- Ưu điểm: Tốc độ xử lý nhanh nhất vì hầu như không sử dụng các cơ chế khóa để ngăn chặn việc đọc. Giảm thiểu tối đa tình trạng nghẽn trong hệ thống.
- Nhược điểm: Độ tin cậy dữ liệu kém nhất. Có rủi ro cao khi tính toán trên những dữ liệu "rác" (dữ liệu có thể bị Rollback sau đó).

4.3.2. Read Committed (Đọc dữ liệu đã xác nhận)

Đây là mức cô lập phổ biến, thường là mặc định trong các hệ quản trị CSDL như SQL Server hay PostgreSQL. Nó đảm bảo một giao tác chỉ có thể đọc được những dữ liệu đã được commit chính thức vào cơ sở dữ liệu.

- Ưu điểm: Loại bỏ hoàn toàn hiện tượng Dirty Read. Cân bằng tốt giữa tính nhất quán của dữ liệu và hiệu năng của hệ thống.

- Nhược điểm: Vẫn còn tồn tại các hiện tượng Non-repeatable Read và Phantom Read. Nghĩa là trong cùng một giao tác, nếu đọc một dòng dữ liệu hai lần, kết quả có thể khác nhau nếu có giao tác khác vừa cập nhật xong.

4.3.3. Repeatable Read (Đọc lặp lại được)

Mức này đảm bảo rằng nếu một giao tác đọc một dòng dữ liệu, thì các lần đọc sau đó trong cùng giao tác đó sẽ luôn nhận được kết quả y hệt. Các giao tác khác không thể sửa đổi hoặc xóa dòng dữ liệu mà giao tác hiện tại đang làm việc cho đến khi giao tác này kết thúc.

- Ưu điểm: Loại bỏ được Dirty Read và Non-repeatable Read. Đảm bảo tính ổn định của dữ liệu trong suốt quá trình xử lý của một giao tác.
- Nhược điểm: Vẫn xảy ra hiện tượng Phantom Read. Việc giữ khóa trên các dòng dữ liệu lâu hơn sẽ làm giảm khả năng đáp ứng đồng thời của hệ thống.

4.3.4. Serializable (Tuần tự hóa)

Đây là mức cô lập cao nhất. Nó thực hiện việc cô lập các giao tác hoàn toàn bằng cách đặt khóa trên toàn bộ phạm vi dữ liệu. Các giao tác được thực hiện một cách tuần tự, giao tác này phải xong thì giao tác kia mới được bắt đầu nếu chúng có chung tài nguyên.

- Ưu điểm: Ngăn chặn tuyệt đối tất cả các dị thường dữ liệu (Dirty Read, Non-repeatable Read, Phantom Read). Dữ liệu đạt độ chính xác và an toàn tuyệt đối.
- Nhược điểm: Hiệu năng hệ thống thấp nhất do các giao tác phải chờ đợi nhau rất lâu. Rất dễ xảy ra tình trạng Deadlock và làm giảm trải nghiệm người dùng trong các hệ thống có lưu lượng truy cập lớn.

4.4. Các phương thức khóa cơ bản

Để thực thi các mức cô lập trên, hệ thống sử dụng cơ chế Khóa (Lock) nhằm kiểm soát quyền truy cập của các giao tác vào các tài nguyên dữ liệu.

4.4.1. Shared Lock (Khóa chia sẻ – Ký hiệu: S)

Loại khóa này được sử dụng khi giao tác chỉ có nhu cầu đọc dữ liệu. Đặc điểm chính là tính cho phép truy cập đồng thời, nghĩa là nhiều giao tác có thể cùng giữ khóa S trên một bản ghi để cùng đọc. Tuy nhiên, hệ thống sẽ ngăn chặn mọi yêu cầu đặt khóa X từ các giao tác khác cho đến khi các khóa S hiện tại được giải phóng hoàn toàn.

4.4.2. Exclusive Lock (Khóa độc quyền – Ký hiệu: X)

Loại khóa này được sử dụng khi giao tác cần thay đổi dữ liệu (Thêm, Sửa, Xóa). Khác với khóa S, tại một thời điểm chỉ có duy nhất một giao tác được nắm giữ khóa X trên một bản ghi. Khi khóa X đang được thiết lập, mọi yêu cầu truy cập khác từ bên ngoài (bao gồm cả đọc và ghi) đều bị từ chối để đảm bảo dữ liệu không bị sai lệch.

4.5. Minh họa đồ án môn học trên hệ quản trị cơ sở dữ liệu Oracle

4.5.1. Trường hợp Lost Update

4.5.1.1. Mô tả tình huống

Trong hệ thống ví điện tử, bảng WALLETS lưu trữ số dư hiện tại của từng ví thông qua thuộc tính BALANCE. Giả sử ví của người dùng A có WALLET_ID = 1 đang có số dư ban đầu là 1.000.000 VND.

Tại cùng một thời điểm, có hai giao dịch xảy ra trên ví của A:

- Transaction T1: A thực hiện chuyển 200.000 VND cho người dùng khác.
- Transaction T2: B thực hiện chuyển 500.000 VND vào ví của A.

Trong quá trình xử lý, T1 đọc số dư hiện tại của A là 1.000.000 VND để kiểm tra số dư và tính toán số dư mới sau khi chuyển tiền. Tuy nhiên, trong lúc T1 chưa cập nhật xong, T2 cũng đọc số dư hiện tại của A là 1.000.000 VND để cộng thêm tiền vào ví A. Sau đó, mỗi transaction cập nhật lại số dư dựa trên giá trị đã đọc ban đầu. Khi đó, kết quả cập nhật của một transaction có thể ghi đè kết quả của transaction còn lại, gây ra hiện tượng Lost Update.

Thời gian	Transaction T1: A chuyển tiền	Transaction T2: B chuyển tiền cho A	Mô tả
t0	SELECT BALANCE FROM WALLETS WHERE WALLET_ID = 1;		T1 đọc số dư ví A, kết quả là 1.000.000.
t1		SELECT BALANCE FROM WALLETS WHERE WALLET_ID = 1;	T2 cũng đọc số dư ví A, kết quả vẫn là 1.000.000.
t2	T1 tính số dư mới: $1.000.000 - 200.000 = 800.000$		T1 tính toán số dư mới sau khi A chuyển tiền đi.
t3		T2 tính số dư mới: $1.000.000 + 500.000 = 1.500.000$	T2 tính toán số dư mới sau khi B chuyển tiền vào ví A.
t4	UPDATE WALLETS SET BALANCE = 800000 WHERE WALLET_ID = 1;		T1 cập nhật số dư ví A thành 800.000.

t5	COMMIT;		T1 commit giao dịch.
t6		UPDATE WALLETS SET BALANCE = 1500000 WHERE WALLET_ID = 1;	T2 cập nhật số dư ví A thành 1.500.000, ghi đề kết quả của T1.
t7		COMMIT;	T2 commit giao dịch.
t8	SELECT BALANCE FROM WALLETS WHERE WALLET_ID = 1;		Số dư cuối cùng của ví A là 1.500.000.

4.5.1.2. Vấn đề xảy ra

Sau khi cả hai transaction hoàn tất, số dư cuối cùng của ví A là 1.500.000 VND. Tuy nhiên, kết quả đúng phải là:

- $1.000.000 - 200.000 + 500.000 = 1.300.000$

Như vậy, giao dịch chuyển tiền đi 200.000 VND của A đã bị mất tác dụng trên số dư cuối cùng. Mặc dù T1 đã thực hiện cập nhật và commit thành công, kết quả của T1 bị T2 ghi đè do T2 vẫn cập nhật dựa trên số dư cũ đã đọc trước đó. Đây chính là hiện tượng Lost Update.

4.5.1.3. Nguyên nhân

Nguyên nhân của lỗi là cả hai transaction đều đọc cùng một giá trị số dư ban đầu của ví A là 1.000.000 VND, sau đó tự tính toán số dư mới và cập nhật lại bằng câu lệnh dạng:

```
UPDATE WALLETS
SET BALANCE = <gia_tri_da_tinh>
WHERE WALLET_ID = 1;
```

Cách cập nhật này nguy hiểm trong môi trường đồng thời, vì transaction cập nhật sau có thể ghi đè kết quả của transaction cập nhật trước. Mức cô lập READ COMMITTED trong Oracle chỉ đảm bảo không đọc dữ liệu chưa commit, nhưng không tự động ngăn việc một transaction dùng dữ liệu cũ để cập nhật lại sau đó.

4.5.1.4. Cách khắc phục

Để tránh Lost Update, cần khóa dòng dữ liệu của ví A trước khi đọc và tính toán số dư. Khi một transaction đang xử lý ví A, transaction khác muốn thay đổi cùng ví phải chờ transaction hiện tại commit hoặc rollback. Ví dụ sử dụng SELECT ... FOR UPDATE.

```
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;

DECLARE

    v_balance NUMBER;

BEGIN

    SELECT BALANCE

    INTO v_balance

    FROM WALLETS

    WHERE WALLET_ID = 1

    FOR UPDATE;

    IF v_balance >= 200000 THEN

        UPDATE WALLETS

        SET BALANCE = v_balance - 200000

        WHERE WALLET_ID = 1;

    ELSE

        RAISE_APPLICATION_ERROR(-20001, 'Số dư không đủ để thực hiện giao dịch');

    END IF;

    COMMIT;

END;
```

Khi T1 thực hiện SELECT ... FOR UPDATE, dòng dữ liệu của ví A sẽ bị khóa. Nếu T2 muốn chuyển tiền vào ví A trong lúc T1 chưa hoàn tất, T2 phải chờ T1 commit hoặc rollback. Sau khi T1 commit, T2 mới đọc được số dư mới nhất là 800.000 VND, sau đó cộng thêm 500.000 VND. Khi đó kết quả cuối cùng sẽ đúng là:

- $800.000 + 500.000 = 1.300.000$

Ngoài ra, có thể giảm rủi ro Lost Update bằng cách cập nhật trực tiếp trên database theo dạng biểu thức:

```
UPDATE WALLETS  
  
SET BALANCE = BALANCE - 200000  
  
WHERE WALLET_ID = 1  
  
AND BALANCE >= 200000;
```

và với giao dịch B chuyển tiền cho A:

```
UPDATE WALLETS  
  
SET BALANCE = BALANCE + 500000  
  
WHERE WALLET_ID = 1;
```

4.5.2. Trường hợp Non-repeatable Read

4.5.2.1. Mô tả tình huống

Trong hệ thống ví điện tử, trạng thái KYC (Know Your Customer) của người dùng được lưu trong bảng USERS thông qua thuộc tính KYC_STATUS. Trạng thái này ảnh hưởng trực tiếp đến hạn mức giao dịch được áp dụng, vì hệ thống tra bảng TRANSACTION_LIMITS dựa trên cấp độ KYC để xác định số tiền tối đa cho mỗi giao dịch và tổng giao dịch trong ngày.

Giả sử hệ thống đang xử lý một giao dịch rút tiền cho người dùng có USER_ID = 3. Ở Session T1, hệ thống đọc trạng thái KYC để xác định hạn mức. Trong khi T1 chưa hoàn tất, Session T2 (luồng xử lý KYC của admin) cập nhật và commit trạng thái KYC của người dùng đó từ PENDING sang VERIFIED. Khi T1 đọc lại trạng thái KYC lần thứ hai trong cùng một phiên xử lý, kết quả trả về là VERIFIED — khác hoàn toàn so với lần đọc đầu tiên. Đây là hiện tượng Non-Repeatable Read: cùng một câu truy vấn SELECT trên cùng một bản ghi nhưng cho ra hai kết quả khác nhau trong cùng một transaction, do một transaction khác đã commit thay đổi dữ liệu ở giữa.

Thời gian	Session T1	Session T2	Mô tả
t0	SELECT kyc_status FROM users WHERE user_id = 3; -- Kết quả: PENDING		T1 đọc KYC lần 1 → PENDING
t1	-- T1 tra bảng transaction_limits -- kyc_level = 'UNVERIFIED' -- max_per_trans = 5.000.000 VND		T1 xác định hạn mức theo UNVERIFIED
t2		UPDATE users SET kyc_status = 'VERIFIED' WHERE user_id = 3;	T2 cập nhật KYC: PENDING → VERIFIED
t3		COMMIT;	T2 commit thành công
t4	SELECT kyc_status FROM users WHERE user_id = 3; -- Kết quả: VERIFIED		T1 đọc KYC lần 2 → VERIFIED (khác lần 1)
t5	-- T1 tra lại transaction_limits -- kyc_level = 'VERIFIED' -- max_per_trans = 50.000.000 VND		T1 bị thay đổi hạn mức giữa chừng

t6	COMMIT;		T1 commit với hạn mức không nhất quán
----	---------	--	---------------------------------------

4.5.2.2. Vấn đề xảy ra

Tại thời điểm t0, T1 đọc `KYC_STATUS` = 'PENDING' và xác định người dùng thuộc cấp độ UNVERIFIED, tức hạn mức tối đa mỗi giao dịch chỉ là 5.000.000 VND. Tuy nhiên, trong khoảng thời gian T1 đang xử lý (từ t2 đến t3), T2 đã commit thành công việc cập nhật `KYC_STATUS` thành 'VERIFIED'. Khi T1 đọc lại `KYC_STATUS` lần thứ hai tại t4, kết quả là 'VERIFIED', dẫn đến hệ thống xác định hạn mức VERIFIED với mức trần lên tới 50.000.000 VND.

Hậu quả là trong cùng một luồng xử lý giao dịch, T1 có hai snapshot KYC khác nhau — một lần dùng hạn mức UNVERIFIED để kiểm tra ban đầu, một lần dùng hạn mức VERIFIED để thực thi cuối cùng. Điều này phá vỡ tính nhất quán của quy trình: hệ thống có thể thực thi một giao dịch với số tiền vượt ngưỡng mà lúc bắt đầu transaction nó đã kiểm tra và cho là hợp lệ dưới hạn mức UNVERIFIED, nhưng thực ra lại được thông qua nhờ hạn mức VERIFIED xuất hiện ở giữa. Ngược lại, một số logic kiểm tra khác có thể thất bại vì dữ liệu không nhất quán.

4.5.2.3. Nguyên nhân

Nguyên nhân của hiện tượng này nằm ở mức cô lập READ COMMITTED — mức mặc định trong Oracle. Ở mức này, một transaction chỉ thấy dữ liệu đã được commit tại thời điểm thực thi câu lệnh SELECT, chứ không phải tại thời điểm bắt đầu transaction. Vì vậy, mỗi lần T1 thực hiện câu lệnh SELECT, nó đọc snapshot mới nhất của dữ liệu. Nếu giữa hai lần đọc có một transaction khác commit thay đổi, T1 sẽ thấy kết quả khác nhau.

Cụ thể, sau khi T2 commit tại t3, dữ liệu `KYC_STATUS` trong bảng USERS đã chính thức thay đổi. Câu lệnh SELECT tại t4 của T1 đọc trực tiếp từ dữ liệu đã commit, nên trả về VERIFIED thay vì PENDING như lần đọc trước. Đây là đặc điểm cố hữu của READ COMMITTED: không đảm bảo dữ liệu đọc được nhất quán xuyên suốt một transaction.

4.5.2.4. Cách khắc phục

Để ngăn hiện tượng Non-Repeatable Read, cần nâng mức cô lập lên SERIALIZABLE hoặc áp dụng khóa dòng dữ liệu ngay từ lần đọc đầu tiên bằng SELECT ... FOR UPDATE.

Cách 1: Sử dụng SELECT ... FOR UPDATE để khóa bản ghi user ngay khi đọc KYC lần đầu. Khi đó, T2 sẽ bị chặn cho đến khi T1 commit hoặc rollback, đảm bảo giá trị KYC_STATUS không thay đổi trong suốt quá trình T1 xử lý.

```
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

```
DECLARE
```

```
  v_kyc_status  users.kyc_status%TYPE;
```

```
  v_kyc_level   VARCHAR2(20);
```

```
  v_max_per_trans transaction_limits.max_amount_per_trans%TYPE;
```

```
  v_amount      NUMBER := 8000000;
```

```
BEGIN
```

```
  SELECT u.kyc_status
```

```
  INTO  v_kyc_status
```

```
  FROM  users u
```

```
  JOIN  wallets w ON u.user_id = w.user_id
```

```
  WHERE u.user_id = 3
```

```
  FOR UPDATE;
```

```
  v_kyc_level := CASE
```

```
    WHEN v_kyc_status = 'VERIFIED' THEN 'VERIFIED'
```

```
    ELSE 'UNVERIFIED'
```

```
  END;
```

```
  SELECT max_amount_per_trans
```

```
  INTO  v_max_per_trans
```

```
  FROM  transaction_limits
```

```
  WHERE kyc_level = v_kyc_level AND ROWNUM = 1;
```

```
  IF v_amount > v_max_per_trans THEN
```

```
    RAISE_APPLICATION_ERROR(-20050, 'Giao dịch vượt hạn mức KYC.');
```

```
  END IF;
```

```
  COMMIT;
```

```
END;
```

Khi T1 thực hiện SELECT ... FOR UPDATE, Oracle sẽ đặt khóa ghi (row-level lock) lên bản ghi user_id = 3. Bất kỳ transaction nào — bao gồm T2 — muốn cập nhật dòng đó đều phải chờ T1 kết thúc. Nhờ vậy, giá trị KYC_STATUS mà T1 đọc được sẽ duy trì ổn định suốt vòng đời của transaction.

Cách 2: Nâng mức cô lập lên SERIALIZABLE. Ở mức này, Oracle đảm bảo mọi câu lệnh SELECT trong cùng một transaction đều nhìn thấy cùng một snapshot dữ liệu tại thời điểm transaction bắt đầu, bất kể các transaction khác có commit thay đổi hay không.

```
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;

DECLARE
    v_kyc_status    users.kyc_status%TYPE;
    v_kyc_level     VARCHAR2(20);
BEGIN
    SELECT u.kyc_status
    INTO   v_kyc_status
    FROM   users u
    WHERE  u.user_id = 3;

    v_kyc_level := CASE
        WHEN v_kyc_status = 'VERIFIED' THEN 'VERIFIED'
        ELSE 'UNVERIFIED'
    END;

    COMMIT;
END;
/
```

4.5.3. Trường hợp Phantom Read

4.5.3.1. Mô tả tình huống

Trong hệ thống ví điện tử, bảng VOUCHERS lưu trữ thông tin các mã giảm giá, trong đó thuộc tính AMOUNT_VOUCHERS biểu thị số lượng voucher còn lại có thể sử dụng. Khi người dùng xác nhận áp dụng một voucher vào giao dịch chuyển tiền, hệ thống kiểm tra AMOUNT_VOUCHERS > 0 trước khi trừ đi một đơn vị.

Giả sử chương trình khuyến mãi với mã 'SALE50' chỉ còn lại đúng 1 voucher cuối cùng. Tại cùng một thời điểm, Session A và Session B đều truy vấn bảng VOUCHERS và cùng thấy AMOUNT_VOUCHERS = 1, nên cả hai đều thực hiện xác nhận giao dịch. Session B commit trước, làm cho AMOUNT_VOUCHERS về 0. Tuy nhiên, Session A vẫn dựa vào kết quả đọc cũ (bằng 1) để tiếp tục xử lý và commit, khiến AMOUNT_VOUCHERS bị giảm xuống -1. Đây là hiện tượng Phantom Read: tập kết quả mà A thấy trong lần đọc đầu (1 voucher) đã "biến mất" hoặc thay đổi hoàn toàn khi A đọc lại (0 hoặc âm), do một transaction khác đã chèn/xóa/sửa dữ liệu và commit ở giữa.

Thời gian	Session A	Session B	Mô tả
t0	SELECT amount_vouchers FROM vouchers WHERE code = 'SALE50'; -- Kết quả: 1		A đọc: còn 1 voucher
t1		SELECT amount_vouchers FROM vouchers WHERE code = 'SALE50'; -- Kết quả: 1	B đọc: cũng thấy còn 1 voucher
t2		UPDATE vouchers SET amount_vouchers = amount_vouchers - 1 WHERE code = 'SALE50'; - - amount_vouchers = 0 COMMIT;	B xác nhận và commit, voucher = 0
t3	-- A vẫn thấy còn 1 nên tiếp tục xử lý IF amount_vouchers > 0 THEN -- áp dụng voucher END IF;		A vẫn dùng kết quả đọc cũ (amount = 1)
t4	UPDATE vouchers SET amount_vouchers = amount_vouchers - 1 WHERE code = 'SALE50'; -- amount_vouchers = -1 COMMIT;		A cũng commit, voucher thành -1 (lỗi!)
t5	SELECT amount_vouchers FROM vouchers WHERE code = 'SALE50'; -- Kết quả: -1 hoặc 0		A truy vấn lại → thấy voucher âm hoặc hết

4.5.3.2. Vấn đề xảy ra

Sau khi cả hai session hoàn tất, AMOUNT_VOUCHERS trong bảng VOUCHERS có giá trị là -1, vi phạm ràng buộc CHECK (amount_vouchers >= 0) đã được định nghĩa trong schema. Ngoài ra, một voucher bị phân phát cho hai người dùng khác nhau, gây thất thoát về mặt kinh doanh và mất tính toàn vẹn dữ liệu.

Về mặt kỹ thuật, kết quả đúng phải là: chỉ một trong hai session được sử dụng voucher, session còn lại phải nhận thông báo voucher đã hết. Thực tế, cả hai đều áp dụng voucher thành công dẫn đến số lượng voucher âm là sai logic nghiệp vụ.

4.5.3.3. Nguyên nhân

Tương tự Non-Repeatable Read, nguyên nhân gốc rễ nằm ở mức cô lập READ COMMITTED. Ở mức này, mỗi câu lệnh SELECT nhìn thấy snapshot mới nhất tại thời điểm câu lệnh đó được thực thi. Cả hai session đều đọc AMOUNT_VOUCHERS = 1 trước khi session nào commit. Sau khi B commit, A không biết rằng bộ kết quả (tập các voucher còn hàng) đã thay đổi, và vẫn tiếp tục xử lý dựa trên thông tin cũ. Đây chính xác là bản chất của Phantom Read: bộ hàng thỏa điều kiện WHERE đã bị thay đổi giữa hai lần đọc, nhưng session không nhận ra.

Trong trường hợp cụ thể này, kết hợp thêm việc hai session đều UPDATE AMOUNT_VOUCHERS mà không kiểm tra lại giá trị hiện tại trước khi trừ, càng làm trầm trọng hơn vấn đề, dẫn đến giá trị âm.

4.5.3.4. Cách khắc phục

Để ngăn Phantom Read trong trường hợp tranh chấp voucher, có thể áp dụng một trong hai hướng tiếp cận sau:

Cách 1: Dùng SELECT ... FOR UPDATE kết hợp điều kiện kiểm tra lại AMOUNT_VOUCHERS khi UPDATE. Khi session A thực hiện SELECT ... FOR UPDATE lên dòng voucher, Oracle sẽ khóa dòng đó lại. Session B muốn đọc hoặc cập nhật dòng này phải chờ A giải phóng khóa. Nhờ vậy, tại mọi thời điểm chỉ có một session được xử lý voucher, tránh hoàn toàn tình trạng hai session cùng thấy AMOUNT_VOUCHERS = 1.

```
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

```
DECLARE
```

```
  v_amount_vouchers  vouchers.amount_vouchers%TYPE;
```

```
  v_voucher_id       vouchers.voucher_id%TYPE;
```

```
BEGIN
```

```
  SELECT voucher_id, amount_vouchers
```

```
  INTO  v_voucher_id, v_amount_vouchers
```

```
  FROM  vouchers
```

```

WHERE code      = 'SALE50'
  AND is_active = 1
FOR UPDATE;

IF v_amount_vouchers <= 0 THEN
  RAISE_APPLICATION_ERROR(-20060, 'Voucher đã hết lượt sử dụng nha
bạn.');
```

END IF;

```

UPDATE vouchers
SET  amount_vouchers = amount_vouchers - 1
WHERE voucher_id = v_voucher_id;

COMMIT;
END;
```

Cách 2: Thêm điều kiện `AMOUNT_VOUCHERS > 0` trực tiếp vào câu lệnh `UPDATE` và kiểm tra số dòng bị ảnh hưởng (`SQL%ROWCOUNT`). Nếu không có dòng nào được cập nhật, tức là voucher đã hết trong lúc chờ, hệ thống sẽ rollback và thông báo lỗi. Đây là kỹ thuật optimistic locking: không khóa dòng từ trước, nhưng phát hiện xung đột ngay tại thời điểm ghi.

```

DECLARE
  v_rows_updated NUMBER;
BEGIN
  UPDATE vouchers
  SET  amount_vouchers = amount_vouchers - 1
  WHERE code          = 'SALE50'
    AND is_active     = 1
    AND amount_vouchers > 0;

  v_rows_updated := SQL%ROWCOUNT;

  IF v_rows_updated = 0 THEN
    ROLLBACK;
    RAISE_APPLICATION_ERROR(-20061, 'Voucher đã hết lượt sử dụng hoặc
không còn hợp lệ nha bạn.');
```

END IF;

```

  COMMIT;
END;
```

4.5.4. Trường hợp Dirty Read

4.5.4.1. Mô tả tình huống

Dirty Read là hiện tượng một transaction đọc được dữ liệu mà transaction khác đã thay đổi nhưng chưa thực hiện COMMIT. Nếu transaction thay đổi dữ liệu sau đó ROLLBACK, dữ liệu mà transaction còn lại đã đọc trước đó sẽ trở thành dữ liệu không hợp lệ.

Trong hệ thống ví điện tử, tình huống này có thể xảy ra khi một session đang cập nhật dữ liệu quan trọng nhưng chưa commit. Ví dụ, Session A cập nhật số lượng voucher của mã GIAM10K từ 10 về 0 nhưng chưa COMMIT. Trong lúc đó, Session B thực hiện kiểm tra tính hợp lệ của voucher bằng function `fn_validate_voucher`.

Nếu hệ quản trị cơ sở dữ liệu cho phép Dirty Read, Session B có thể đọc được giá trị `amount_vouchers = 0` dù thay đổi này chưa được xác nhận. Khi đó, hệ thống có thể kết luận sai rằng voucher đã hết lượt sử dụng. Nếu Session A sau đó ROLLBACK, số lượng voucher quay về 10, còn kết quả kiểm tra của Session B trước đó là không chính xác.

Tuy nhiên, trong Oracle, Dirty Read không xảy ra theo cơ chế đọc thông thường. Oracle sử dụng cơ chế read consistency, nên Session B sẽ không nhìn thấy dữ liệu chưa commit của Session A. Thay vào đó, Session B chỉ nhìn thấy phiên bản dữ liệu đã commit gần nhất.

4.5.4.2. Vấn đề xảy ra

Nếu một hệ quản trị cơ sở dữ liệu cho phép Dirty Read, hệ thống có thể xử lý nghiệp vụ dựa trên dữ liệu chưa hợp lệ. Với hệ thống ví điện tử, điều này đặc biệt nguy hiểm vì dữ liệu liên quan đến số dư ví, voucher, giao dịch và audit log đều yêu cầu tính chính xác cao.

Ví dụ, nếu một giao dịch nạp tiền đang cập nhật số dư nhưng chưa hoàn tất, một session khác đọc được số dư tạm thời này có thể khiến hệ thống hiển thị sai số dư hoặc cho phép người dùng tiếp tục thanh toán dựa trên khoản tiền chưa được xác nhận. Nếu transaction ban đầu rollback, các thao tác phía sau sẽ trở nên sai lệch.

Tương tự, với voucher, nếu session khác đọc được số lượng voucher chưa commit, hệ thống có thể đánh giá sai trạng thái voucher, dẫn đến việc từ chối hoặc chấp nhận voucher không đúng tại thời điểm xử lý.

4.5.4.3. Nguyên nhân

Dirty Read thường xuất hiện ở các hệ quản trị cơ sở dữ liệu cho phép mức cô lập READ UNCOMMITTED. Ở mức cô lập này, một transaction có thể đọc được các thay đổi chưa commit của transaction khác.

Tuy nhiên, Oracle không hỗ trợ Dirty Read trong cơ chế đọc thông thường. Mức cô lập mặc định của Oracle là READ COMMITTED, nghĩa là một transaction chỉ đọc được dữ liệu đã commit. Khi một session cập nhật dữ liệu nhưng chưa commit, session khác

vẫn có thể thực hiện câu lệnh SELECT, nhưng Oracle sẽ trả về phiên bản dữ liệu đã commit gần nhất thay vì dữ liệu đang bị thay đổi tạm thời.

Cơ chế này giúp Oracle đảm bảo các câu lệnh đọc dữ liệu không bị ảnh hưởng bởi những thay đổi chưa hoàn tất của transaction khác.

4.5.4.4. Cách khắc phục

Trong Oracle, Dirty Read đã được ngăn chặn tự động thông qua cơ chế read consistency. Khi một transaction cập nhật dữ liệu nhưng chưa commit, Oracle không cho các transaction khác đọc trực tiếp phiên bản dữ liệu chưa hoàn tất đó. Thay vào đó, Oracle sử dụng dữ liệu trong Undo Segment để dựng lại phiên bản dữ liệu đã commit trước đó và trả về cho session đọc.

Vì vậy, trong tình huống Session A cập nhật số lượng voucher GIAM10K từ 10 về 0 nhưng chưa commit, Session B khi gọi function `fn_validate_voucher` vẫn sẽ đọc được trạng thái đã commit gần nhất của voucher. Nếu trước đó voucher còn 10 lượt, Session B vẫn thấy số lượng là 10, không thấy giá trị tạm thời là 0.

Nếu Session A thực hiện ROLLBACK, Oracle hủy thay đổi chưa commit và dữ liệu vẫn giữ giá trị ban đầu. Nếu Session A thực hiện COMMIT, các session khác chỉ nhìn thấy giá trị mới sau khi commit hoàn tất.

Do đó, với Dirty Read, hệ thống không cần xây dựng procedure hoặc trigger riêng để khắc phục, vì Oracle đã tự động ngăn chặn hiện tượng này ở tầng hệ quản trị cơ sở dữ liệu.

4.5.4.5. Kết luận

Dirty Read là lỗi nghiêm trọng trong các hệ thống tài chính vì có thể làm hệ thống xử lý dựa trên dữ liệu chưa hợp lệ. Tuy nhiên, trong Oracle, Dirty Read được ngăn chặn mặc định bằng cơ chế read consistency và Undo Segment. Nhờ đó, các session chỉ đọc được dữ liệu đã commit, đảm bảo tính nhất quán khi truy vấn số dư ví, số lượng voucher, trạng thái giao dịch hoặc audit log.

4.5.5. Trường hợp Deadlock

4.5.5.1. Mô tả tình huống

Deadlock là tình huống xảy ra khi hai hoặc nhiều transaction cùng giữ khóa trên các tài nguyên mà transaction còn lại đang cần. Khi đó, các transaction rơi vào trạng thái chờ lẫn nhau và không transaction nào có thể tiếp tục thực hiện.

Trong hệ thống ví điện tử, deadlock có thể xảy ra khi hai giao dịch cùng tác động đến nhiều ví hoặc nhiều bản ghi liên quan. Ví dụ, Session A xử lý giao dịch từ ví 1 sang ví 2, trong khi Session B xử lý giao dịch từ ví 2 sang ví 1.

Nếu Session A khóa `wallet_id = 1` trước rồi chờ khóa `wallet_id = 2`, còn Session B khóa `wallet_id = 2` trước rồi chờ khóa `wallet_id = 1`, hai session sẽ tạo thành vòng chờ. Session A chờ tài nguyên Session B đang giữ, đồng thời Session B cũng chờ tài nguyên Session A đang giữ.

4.5.5.2. Vấn đề xảy ra

Deadlock làm cho giao dịch không thể tiếp tục xử lý. Trong hệ thống ví điện tử, nếu deadlock xảy ra trong quá trình chuyển tiền, transaction có thể bị rollback và người dùng nhận lỗi giao dịch thất bại.

Ví dụ:

- Session A đã khóa wallet_id = 1.
- Session B đã khóa wallet_id = 2.
- Session A tiếp tục muốn khóa wallet_id = 2 nhưng đang bị session B giữ.
- Session B tiếp tục muốn khóa wallet_id = 1 nhưng đang bị session A giữ.

Lúc này, cả hai session đều không thể tiếp tục. Nếu không có cơ chế xử lý, hệ thống sẽ bị treo vô thời hạn. Tuy nhiên, Oracle có cơ chế tự động phát hiện deadlock nên vòng chờ này sẽ không kéo dài mãi.

4.5.5.3. Nguyên nhân

Nguyên nhân chính là do các transaction khóa nhiều bản ghi theo thứ tự không nhất quán.

Trong ví dụ trên, session A khóa ví theo thứ tự 1 → 2, còn session B lại khóa theo thứ tự 2 → 1. Khi hai transaction chạy đồng thời, thứ tự khóa ngược nhau làm phát sinh vòng chờ.

Khi hai transaction chạy đồng thời, thứ tự khóa ngược nhau làm phát sinh vòng chờ. Deadlock không nhất thiết xảy ra do dữ liệu sai, mà thường xuất phát từ cách thiết kế luồng xử lý và thứ tự khóa tài nguyên chưa thống nhất.

4.5.5.4. Cách khắc phục

Oracle có cơ chế tự động phát hiện deadlock trong quá trình quản lý khóa. Khi phát hiện các session đang chờ nhau theo vòng lặp, Oracle sẽ tự động phá vỡ vòng chờ bằng cách hủy một câu lệnh trong một transaction và trả về lỗi:

- ORA-00060: deadlock detected while waiting for resource

Khi lỗi này xảy ra, Oracle không để các session tiếp tục chờ vô hạn. Một trong các session sẽ nhận lỗi deadlock, câu lệnh đang gây deadlock bị rollback, từ đó tài nguyên khóa được giải phóng và session còn lại có thể tiếp tục hoặc được xử lý tiếp tùy tình huống.

Cơ chế này giúp Oracle tự bảo vệ hệ thống khỏi trạng thái treo do các transaction chờ nhau. Nói cách khác, Oracle không ngăn tuyệt đối deadlock xuất hiện, nhưng có khả năng tự phát hiện và tự xử lý khi deadlock xảy ra.

Tuy nhiên, sau khi Oracle trả lỗi ORA-00060, transaction nhận lỗi cần được xử lý rõ ràng. Thông thường, hệ thống nên thực hiện ROLLBACK transaction hiện tại và cho phép thực hiện lại giao dịch nếu cần. Điều này giúp đảm bảo dữ liệu không bị cập nhật dở dang hoặc rơi vào trạng thái không nhất quán.

4.5.5.5. Cách hạn chế deadlock

Mặc dù Oracle có khả năng tự phát hiện và xử lý deadlock, hệ thống vẫn nên hạn chế khả năng phát sinh deadlock bằng cách thiết kế thứ tự khóa dữ liệu nhất quán.

Trong hệ thống ví điện tử, khi một nghiệp vụ cần khóa nhiều ví cùng lúc, các transaction nên khóa theo một quy tắc cố định. Ví dụ, luôn khóa ví có wallet_id nhỏ hơn trước, sau đó mới khóa ví có wallet_id lớn hơn. Khi mọi transaction đều tuân theo cùng một thứ tự khóa, vòng chờ sẽ khó xảy ra hơn.

Ngoài ra, transaction nên được giữ ngắn gọn. Các thao tác như kiểm tra dữ liệu, cập nhật số dư, cập nhật trạng thái giao dịch và ghi audit log nên được thực hiện nhanh, sau đó COMMIT hoặc ROLLBACK ngay để giải phóng khóa. Không nên giữ khóa trong thời gian dài vì điều này làm tăng khả năng các session khác phải chờ, từ đó tăng nguy cơ deadlock.

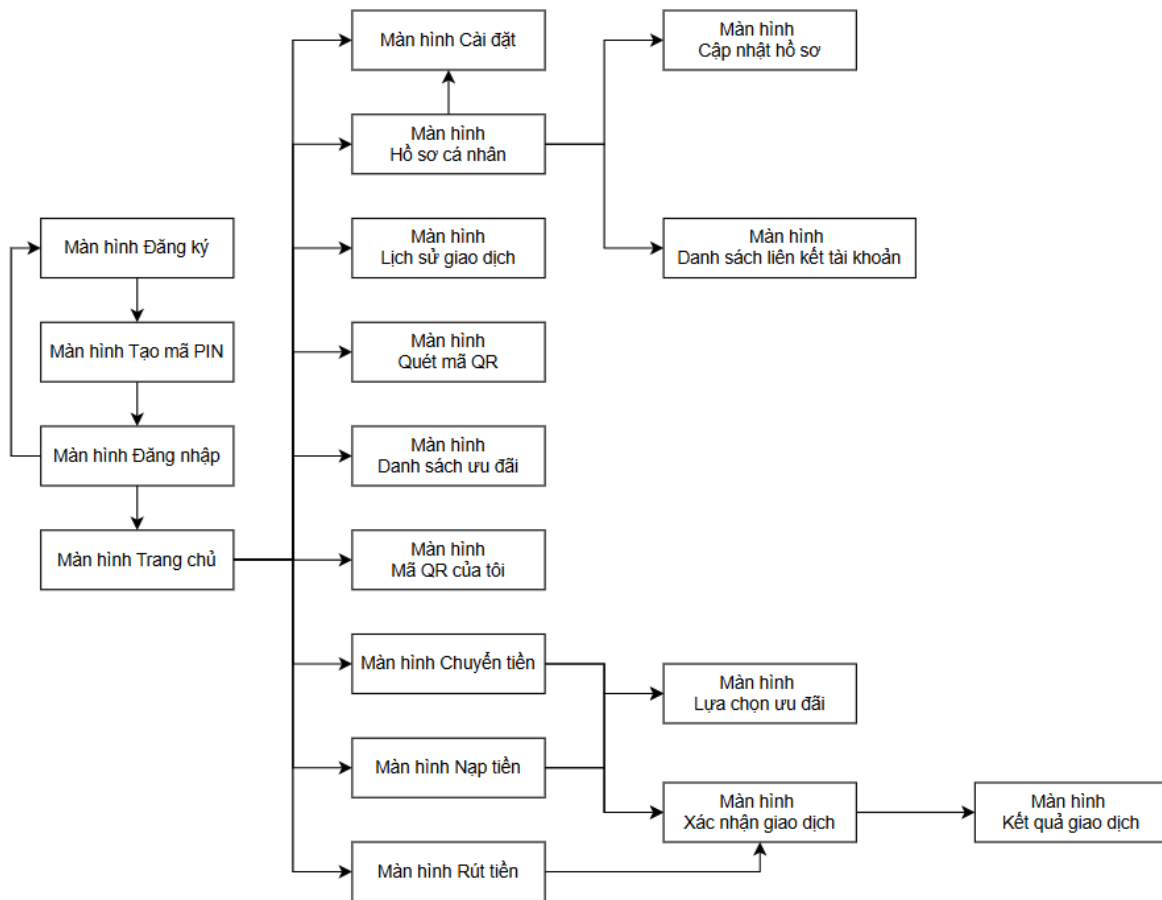
4.5.5.6. Kết luận

Deadlock là một tình huống có thể xảy ra khi nhiều transaction cùng khóa dữ liệu theo thứ tự không thống nhất. Trong Oracle, deadlock không làm hệ thống chờ vô hạn vì Oracle có cơ chế tự động phát hiện vòng chờ và trả về lỗi ORA-00060 để phá vỡ deadlock.

Đối với hệ thống ví điện tử, cơ chế này giúp tránh việc các transaction bị treo khi cùng xử lý các bản ghi ví, giao dịch hoặc audit log. Tuy nhiên, để hệ thống vận hành ổn định hơn, cần thiết kế thứ tự khóa nhất quán, giữ transaction ngắn và xử lý rollback rõ ràng khi Oracle phát hiện deadlock.

CHƯƠNG 5: THIẾT KẾ GIAO DIỆN

5.1. Sơ đồ liên kết các màn hình



Hình 5-1. Sơ đồ liên kết các màn hình

5.2. Danh sách các màn hình

STT	Màn hình	Chức năng
1	Màn hình Đăng ký	Cho phép người dùng nhập thông tin để tạo tài khoản mới
2	Màn hình Tạo mã PIN	Cho phép người dùng thiết lập mã PIN bảo mật cho tài khoản
3	Màn hình Đăng nhập	Cho phép người dùng đăng nhập hệ thống
4	Màn hình Trang chủ	Giao diện chính hiển thị tổng quan và dẫn đến các tính năng của ứng dụng
5	Màn hình Cài đặt	Cấu hình các tùy chọn và thiết lập của ứng dụng ví điện tử

6	Màn hình Hồ sơ cá nhân	Hiển thị thông tin cá nhân của người dùng và các lỗi tắt quản lý tài khoản
7	Màn hình Cập nhật hồ sơ	Cho phép người dùng chỉnh sửa và thay đổi thông tin cá nhân
8	Màn hình Danh sách liên kết tài khoản	Quản lý các tài khoản ngân hàng hoặc ví điện tử đã liên kết
9	Màn hình Lịch sử giao dịch	Hiển thị danh sách các giao dịch đã thực hiện trong quá khứ
10	Màn hình Quét mã QR	Sử dụng camera để quét mã QR phục vụ cho việc thanh toán hoặc chuyển tiền
11	Màn hình Danh sách ưu đãi	Hiển thị các chương trình khuyến mãi và mã giảm giá hiện có
12	Màn hình Mã QR của tôi	Hiển thị mã QR cá nhân của người dùng để nhận tiền từ người khác
13	Màn hình Chuyển tiền	Cho phép người dùng thực hiện thao tác chuyển tiền đến tài khoản khác
14	Màn hình Lựa chọn ưu đãi	Cho phép chọn áp dụng các mã khuyến mãi, giảm giá vào giao dịch hiện tại
15	Màn hình Nạp tiền	Cho phép người dùng nạp thêm tiền vào số dư ứng dụng
16	Màn hình Rút tiền	Cho phép người dùng rút tiền từ ứng dụng về tài khoản ngân hàng
17	Màn hình Xác nhận giao dịch	Hiển thị thông tin chi tiết để người dùng kiểm tra và xác thực trước khi xử lý
18	Màn hình Kết quả giao dịch	Thông báo trạng thái giao dịch thành công hoặc thất bại cho người dùng

Bảng 5-1. Danh sách các màn hình trong ứng dụng

5.3. Mô tả các màn hình

5.3.1. Màn hình Đăng ký

Hình 5-2. Màn hình Đăng ký

STT	Tên	Kiểu	Chức năng
1	Họ và tên	TextField	Nhập Họ và tên
2	Địa chỉ Email	TextField	Nhập Địa chỉ Email
3	Số điện thoại	TextField	Nhập Số điện thoại
4	Mật khẩu	PasswordField	Nhập Mật khẩu
5	Xác nhận mật khẩu	PasswordField	Nhập lại Mật khẩu
6	Tạo mã PIN	Button	Nhấn để vào trang Tạo mã PIN

Bảng 5-2. Bảng mô tả màn hình Đăng ký

5.3.2. Màn hình Tạo mã PIN



Hình 5-3. Màn hình Tạo mã PIN

STT	Tên	Kiểu	Chức năng
1	Mã PIN	TextField	Nhập Mã PIN
2	Hoàn tất đăng ký	Button	Nhấn để vào trang Đăng nhập

Bảng 5-3. Bảng mô tả màn hình Tạo mã PIN

5.3.3. Màn hình Đăng nhập

9:22

E-Wallet App
Thanh toán nhanh chóng, an toàn

Số điện thoại *

Nhập số điện thoại của bạn

Mật khẩu *

Nhập mật khẩu

ĐĂNG NHẬP

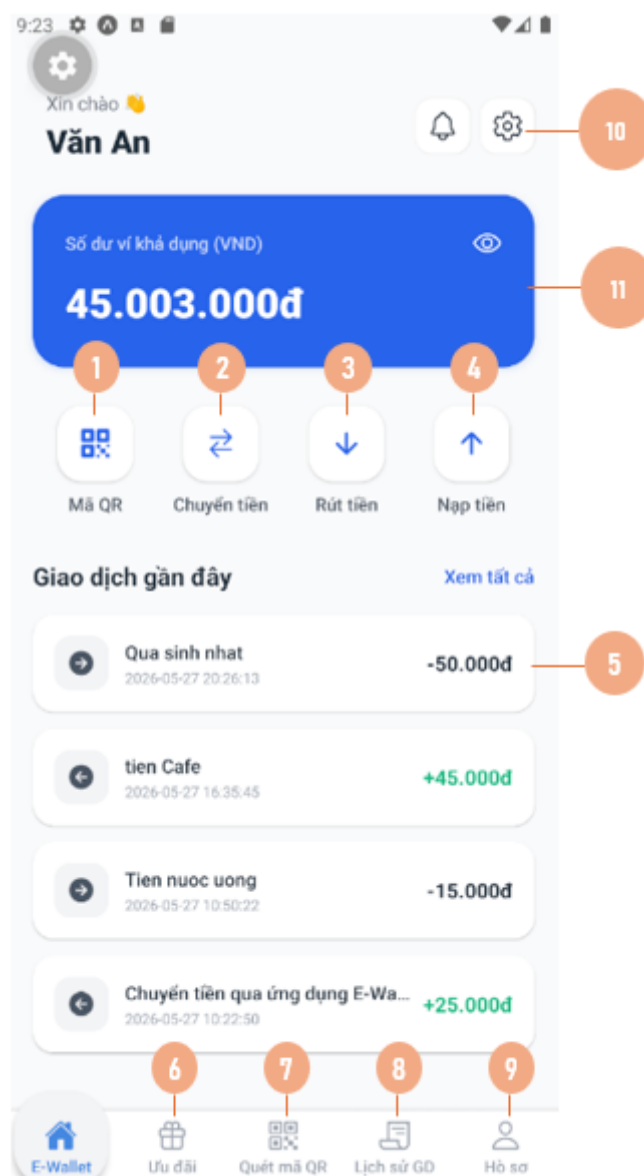
Chưa có tài khoản? [Đăng ký ngay](#)

Hình 5-4. Màn hình Đăng nhập

STT	Tên	Kiểu	Chức năng
1	Số điện thoại	TextField	Nhập Số điện thoại
2	Mật khẩu	PasswordField	Nhập Mật khẩu
3	Đăng nhập	Button	Nhấn để vào Trang chủ

Bảng 5-4. Bảng mô tả màn hình Đăng nhập

5.3.4. Màn hình Trang chủ



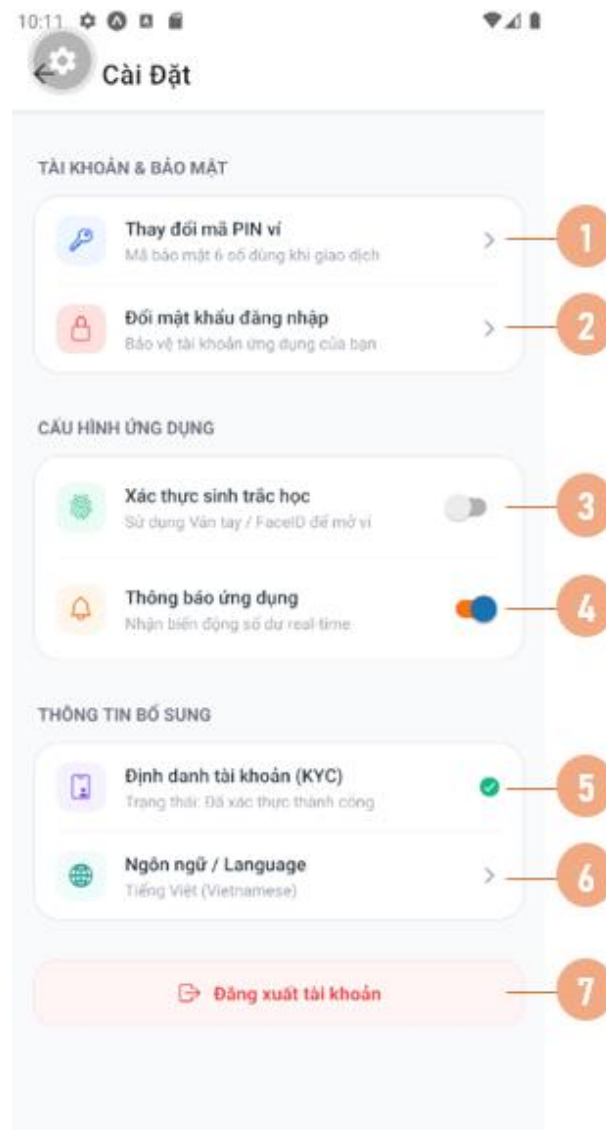
Hình 5-5. Màn hình Trang chủ

STT	Tên	Kiểu	Chức năng
1	Mã QR	JButton	Nhấn để vào trang Mã QR của tôi
2	Chuyển tiền	JButton	Nhấn để vào trang Chuyển tiền
3	Rút tiền	JButton	Nhấn để vào trang Rút tiền
4	Nạp tiền	JButton	Nhấn để vào trang Nạp tiền
5	Giao dịch gần đây	JPanel	Hiện thị tóm tắt giao dịch gần đây

6	Ưu đãi	JButton	Nhấn để vào trang Ưu đãi
7	Quét mã QR	JButton	Nhấn để vào trang Quét mã QR
8	Lịch sử giao dịch	JButton	Nhấn để vào trang Lịch sử giao dịch
9	Hồ sơ	JButton	Nhấn để vào trang Hồ sơ
10	Cài đặt	JButton	Nhấn để vào trang Cài đặt
11	Số dư ví	JPanel	Hiển thị thông tin số dư ví

Bảng 5-5. Bảng mô tả màn hình Trang chủ

5.3.5. Màn hình Cài đặt

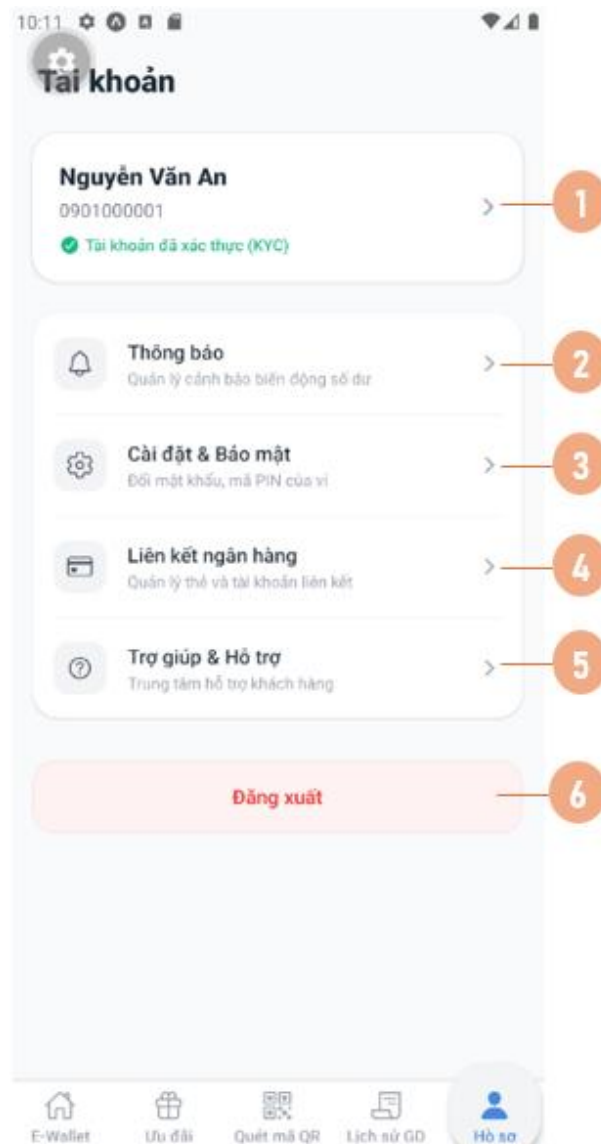


Hình 5-6. Màn hình Cài đặt

STT	Tên	Kiểu	Chức năng
1	Thay đổi mã PIN ví	JButton	Nhấn để vào trang Cập nhật mã PIN
2	Đổi mật khẩu đăng nhập	JButton	Nhấn để vào trang Cập nhật mật khẩu
3	Xác thực sinh trắc học	JToggleButton	Bật/Tắt tính năng xác thực bằng vân tay hoặc khuôn mặt
4	Thông báo ứng dụng	JToggleButton	Bật/Tắt tính năng nhận thông báo biến động số dư
5	Định danh tài khoản	JLabel	Hiển thị thông tin trạng thái xác thực danh tính (KYC)
6	Ngôn ngữ	JButton	Lựa chọn Ngôn ngữ
7	Đăng xuất	JButton	Thực hiện đăng xuất ứng dụng

Bảng 5-6. Bảng mô tả màn hình Cài đặt

5.3.6. Màn hình Hồ sơ cá nhân



Hình 5-7. Màn hình Hồ sơ cá nhân

STT	Tên	Kiểu	Chức năng
1	Thông tin hồ sơ	JButton	Nhấn để vào trang Cập nhật hồ sơ
2	Thông báo	JButton	Nhấn để vào trang Thông báo
3	Cài đặt & Bảo mật	JButton	Nhấn để vào trang Cài đặt
4	Liên kết ngân hàng	JButton	Nhấn để vào trang Danh sách liên kết tài khoản
5	Trợ giúp & Hỗ trợ	JButton	Nhấn để vào trung tâm hỗ trợ khách hàng hoặc xem thông tin hướng dẫn

6	Đăng xuất	JButton	Thực hiện đăng xuất
---	-----------	---------	---------------------

Bảng 5-7. Bảng mô tả màn hình Hồ sơ cá nhân

5.3.7. Màn hình Cập nhật hồ sơ

10:12

← **Chỉnh sửa thông tin cá nhân**

Cập nhật thông tin cá nhân

Chỉnh sửa các trường thông tin bên dưới để đồng bộ hồ sơ ví thành viên của bạn.

Họ và tên

Nguyễn Văn An 1

Địa chỉ Email

an.nguyen@gmail.com 2

Số điện thoại

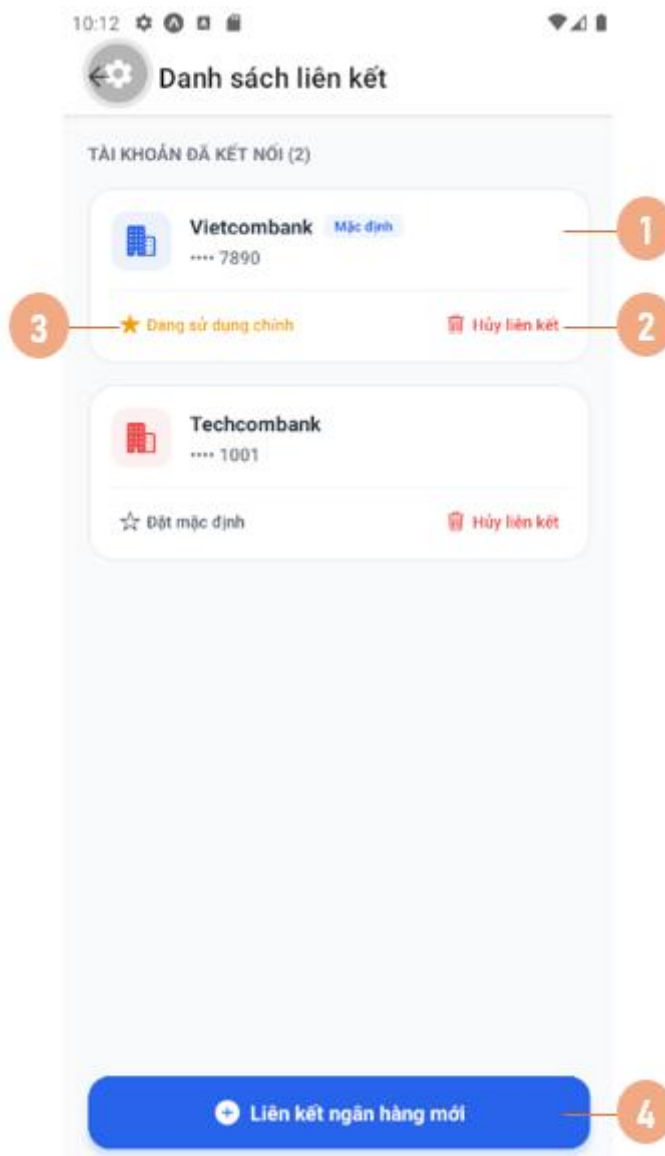
0901000001 3

Lưu thay đổi 4

Hình 5-8. Màn hình Cập nhật hồ sơ

STT	Tên	Kiểu	Chức năng
1	Họ và Tên	JtextField	Sửa họ và tên
2	Địa chỉ Email	JtextField	Sửa địa chỉ email
3	Số điện thoại	JtextField	Sửa số điện thoại
4	Lưu thay đổi	Jbutton	Nhấn để lưu thông tin mới sửa

Bảng 5-8. Bảng mô tả màn hình Cập nhật hồ sơ

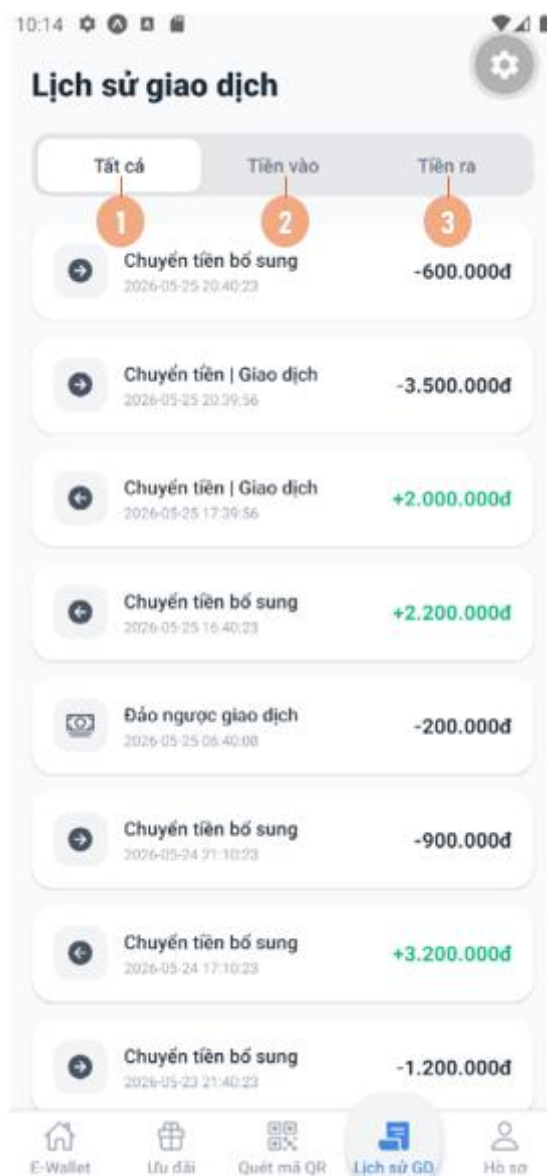
5.3.8. Màn hình Danh sách liên kết tài khoản

Hình 5-9. Màn hình Danh sách liên kết tài khoản

STT	Tên	Kiểu	Chức năng
1	Thông tin ngân hàng	JLabel	Hiển thị thông tin ngân hàng
2	Hủy liên kết	JButton	Ngắt liên kết với tài khoản ngân hàng
3	Đặt mặc định	JButton	Thiết lập nguồn tiền mặc định
4	Liên kết ngân hàng mới	JButton	Tạo mới liên kết ngân hàng

Bảng 5-9. Bảng mô tả màn hình Danh sách liên kết tài khoản

5.3.9. Màn hình Lịch sử giao dịch

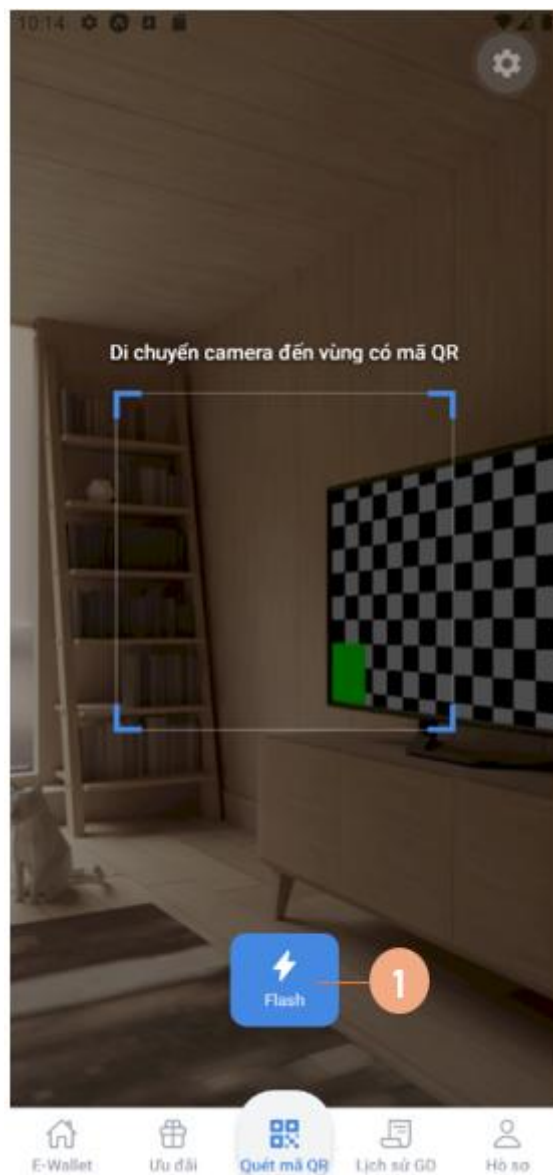


Hình 5-10. Màn hình Lịch sử giao dịch

STT	Tên	Kiểu	Chức năng
1	Tất cả	JToggleButton	Lọc và hiển thị toàn bộ danh sách giao dịch
2	Tiền vào	JToggleButton	Lọc và hiển thị các giao dịch nhận tiền
3	Tiền ra	JToggleButton	Lọc và hiển thị các giao dịch gửi tiền, rút tiền hoặc thanh toán

Bảng 5-10. Bảng mô tả màn hình Lịch sử giao dịch

5.3.10. Màn hình Quét mã QR

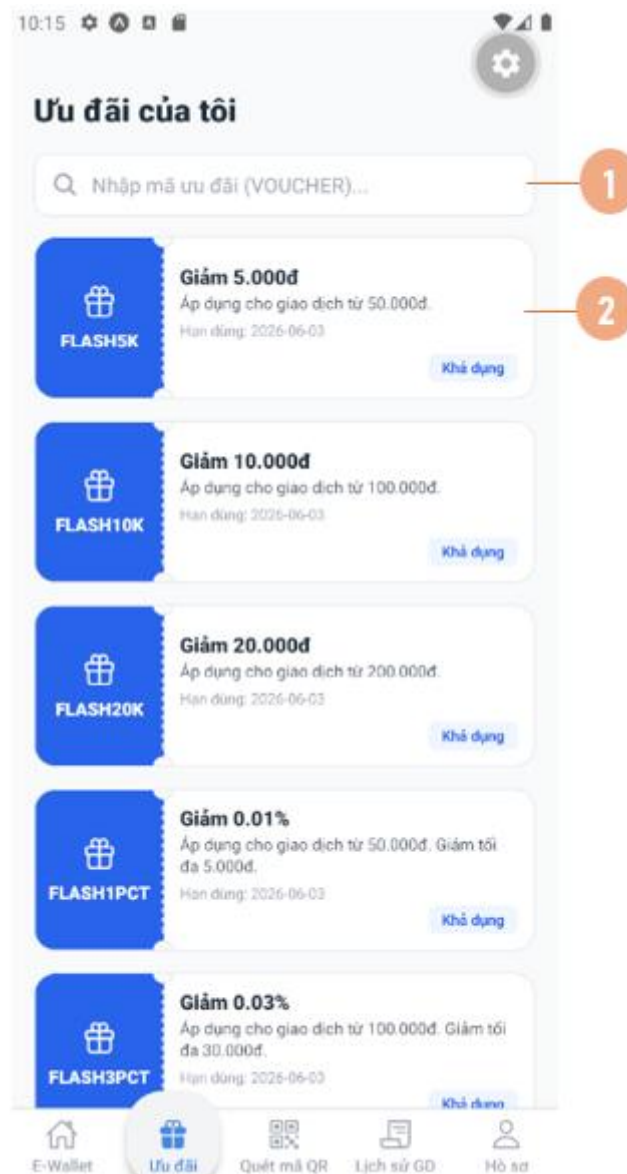


Hình 5-11. Màn hình Quét mã QR

STT	Tên	Kiểu	Chức năng
1	Flash	JToggleButton	Bật/Tắt đèn flash

Bảng 5-11. Bảng mô tả màn hình Quét mã QR

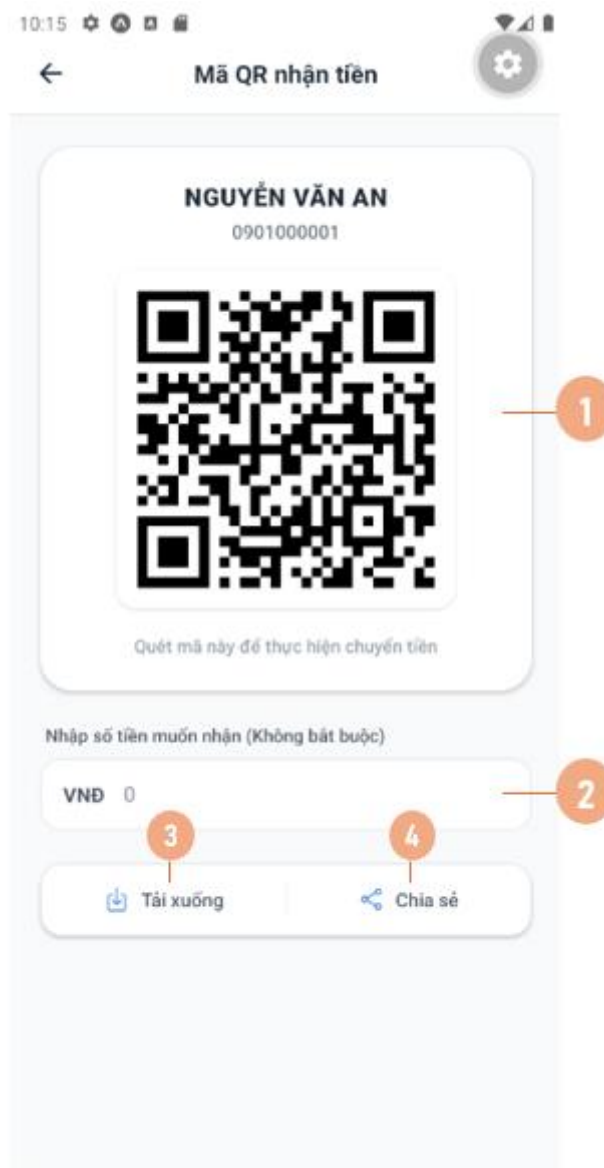
5.3.11. Màn hình Danh sách ưu đãi



Hình 5-12. Màn hình Danh sách ưu đãi

STT	Tên	Kiểu	Chức năng
1	Nhập mã ưu đãi	TextField	Nhập mã ưu đãi cần tìm
2	Thông tin ưu đãi	Label	Hiển thị thông tin ưu đãi

Bảng 5-12. Bảng mô tả màn hình Danh sách ưu đãi

5.3.12. Màn hình Mã QR của tôi

Hình 5-13. Màn hình Mã QR của tôi

STT	Tên	Kiểu	Chức năng
1	Mã QR	JLabel	Hiển thị mã QR cá nhân
2	Số tiền muốn nhận	TextField	Nhập số tiền muốn nhận
3	Tải xuống	Button	Lưu hình ảnh mã QR
4	Chia sẻ	Button	Chia sẻ hình ảnh mã QR

Bảng 5-13. Bảng mô tả màn hình Mã QR của tôi.

5.3.13. Màn hình Chuyển tiền

10:15

← Chuyển Tiền

Số dư ví khả dụng
45.003.000 VNĐ

Người nhận tiền (Số điện thoại)
Nhập số điện thoại người nhận

Số tiền chuyển
0 VNĐ

Chọn Voucher giảm phí [Khả dụng >](#)

Lời nhắn (Không bắt buộc)
Nhập nội dung chuyển tiền...

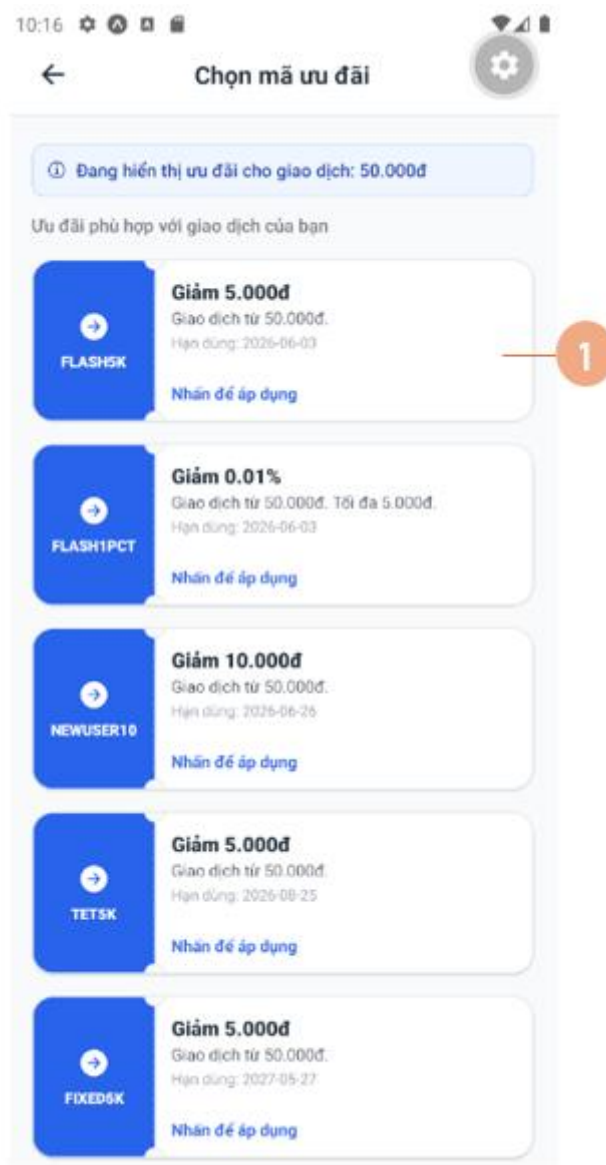
Tiếp tục

Hình 5-14. Màn hình chuyển tiền

STT	Tên	Kiểu	Chức năng
1	Số dư ví	JPanel	Hiển thị thông tin số dư ví
2	Số điện thoại người nhận	TextField	Nhập số điện thoại người nhận
3	Số tiền chuyển khoản	TextField	Nhập số tiền cần chuyển
4	Chọn voucher	Button	Nhấn để vào trang Lựa chọn ưu đãi
5	Lời nhắn	TextField	Nhập lời nhắn
6	Tiếp tục	Button	Nhấn để vào trang Xác nhận giao dịch

Bảng 5-14. Bảng mô tả màn hình Mã QR của tôi

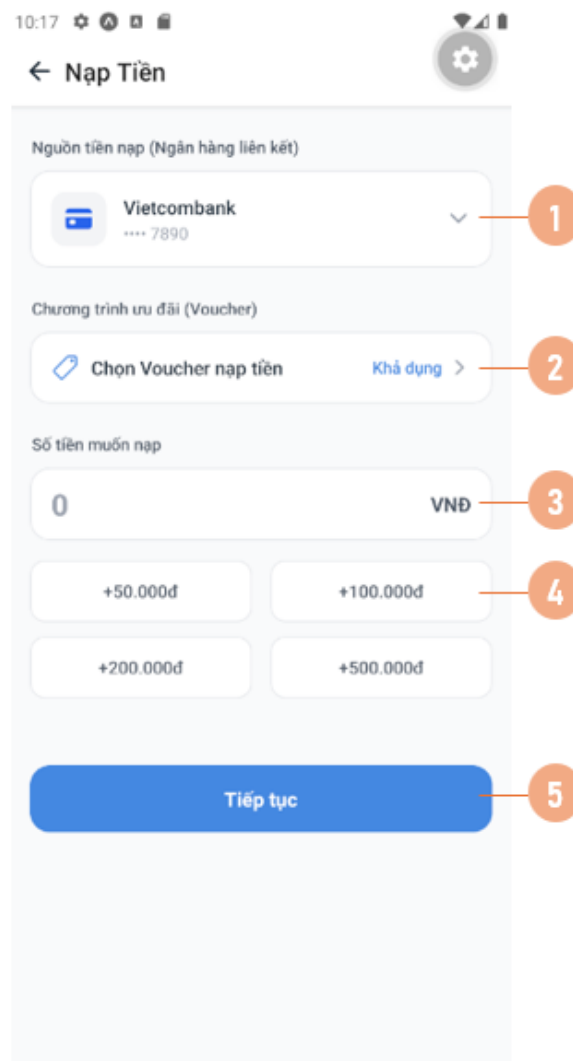
5.3.14. Màn hình Lựa chọn ưu đãi



Hình 5-15. Màn hình Lựa chọn ưu đãi

STT	Tên	Kiểu	Chức năng
1	Thông tin ưu đãi	JLabel	Hiển thị thông tin ưu đãi

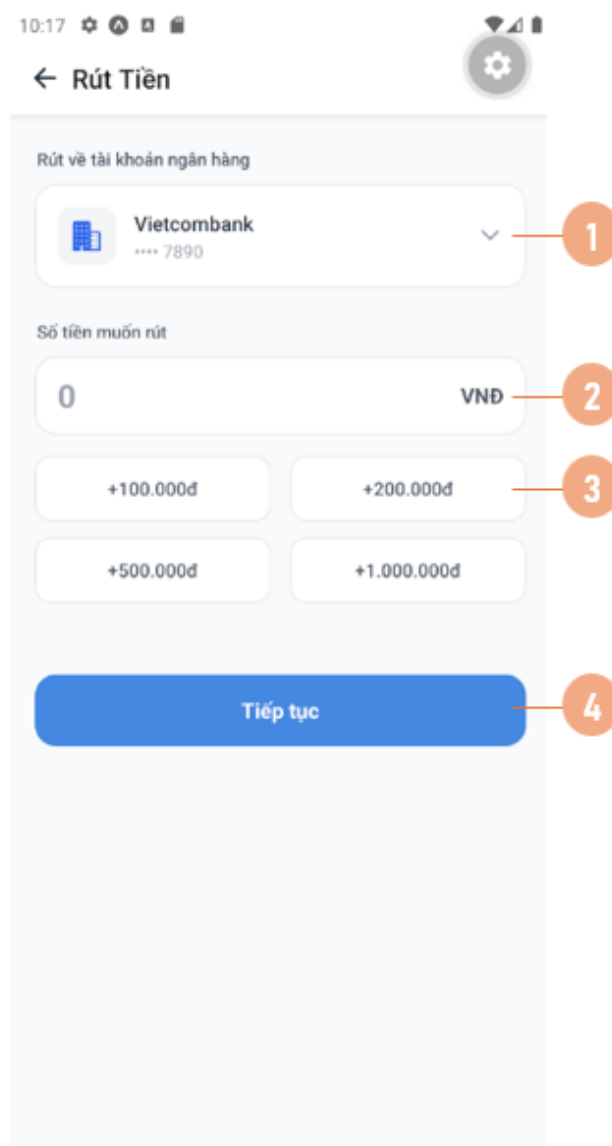
Bảng 5-15. Bảng mô tả màn hình Lựa chọn ưu đãi

5.3.15. Màn hình Nạp tiền

Hình 5-16. Màn hình Nạp tiền

STT	Tên	Kiểu	Chức năng
1	Ngân hàng liên kết	JButton	Lựa chọn ngân hàng liên kết
2	Lựa chọn ưu đãi	JButton	Nhấn để vào trang Lựa chọn ưu đãi
3	Số tiền muốn nạp	JTextField	Nhập số tiền muốn nạp
4	Lựa chọn nhanh số tiền muốn nạp	JButton	Hỗ trợ lựa chọn nhanh khoản tiền muốn nạp
5	Tiếp tục	JButton	Nhấn để vào trang Xác nhận giao dịch

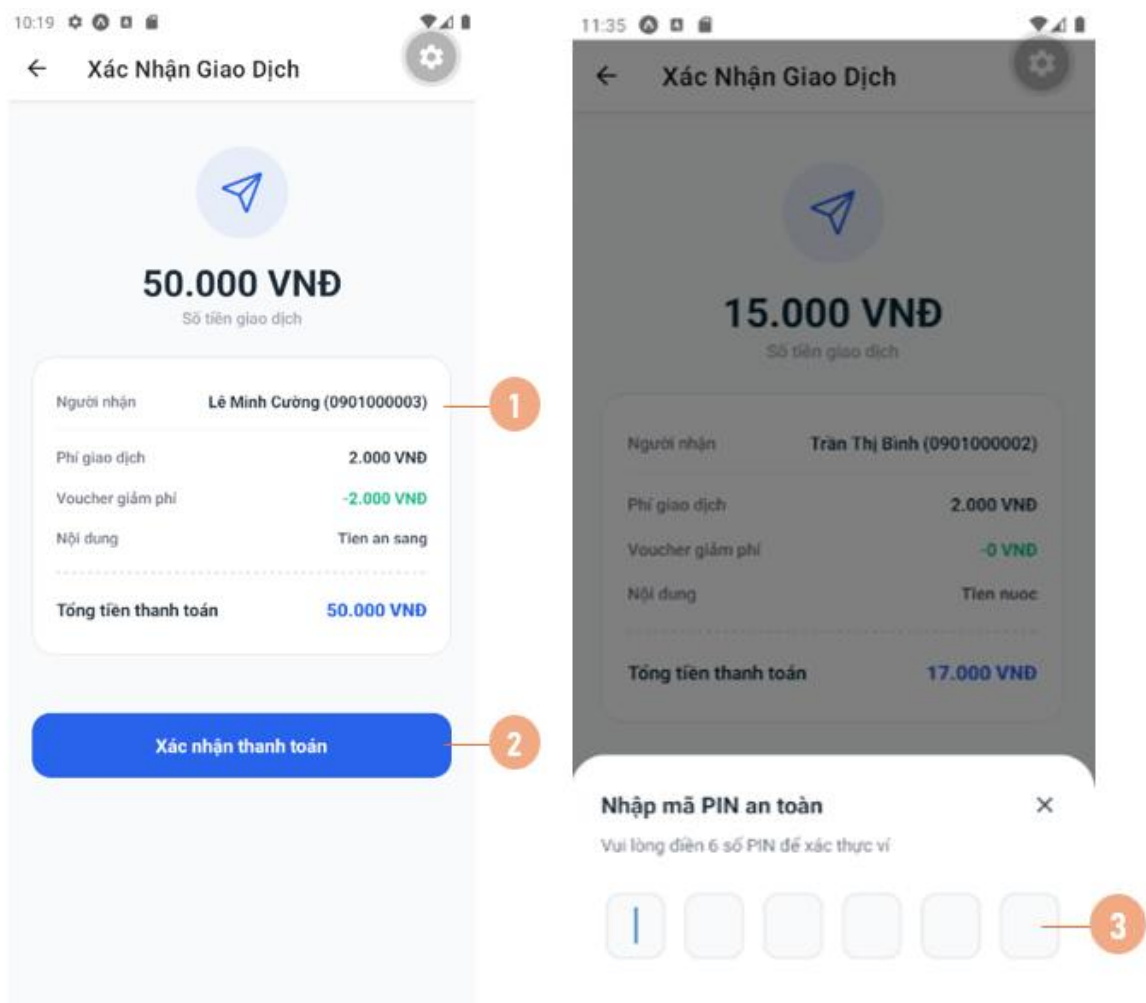
Bảng 5-16. Bảng mô tả màn hình Lựa chọn ưu đãi

5.3.16. Màn hình Rút tiền

Hình 5-17. Màn hình Rút tiền

STT	Tên	Kiểu	Chức năng
1	Ngân hàng liên kết	JButton	Lựa chọn ngân hàng để rút tiền về
2	Số tiền muốn rút	UITextField	Nhập số tiền muốn rút
3	Lựa chọn nhanh số tiền muốn rút	JButton	Hỗ trợ lựa chọn nhanh khoản tiền muốn rút
4	Tiếp tục	JButton	Nhấn để vào trang Xác nhận giao dịch

Bảng 5-17. Bảng mô tả màn hình Rút tiền

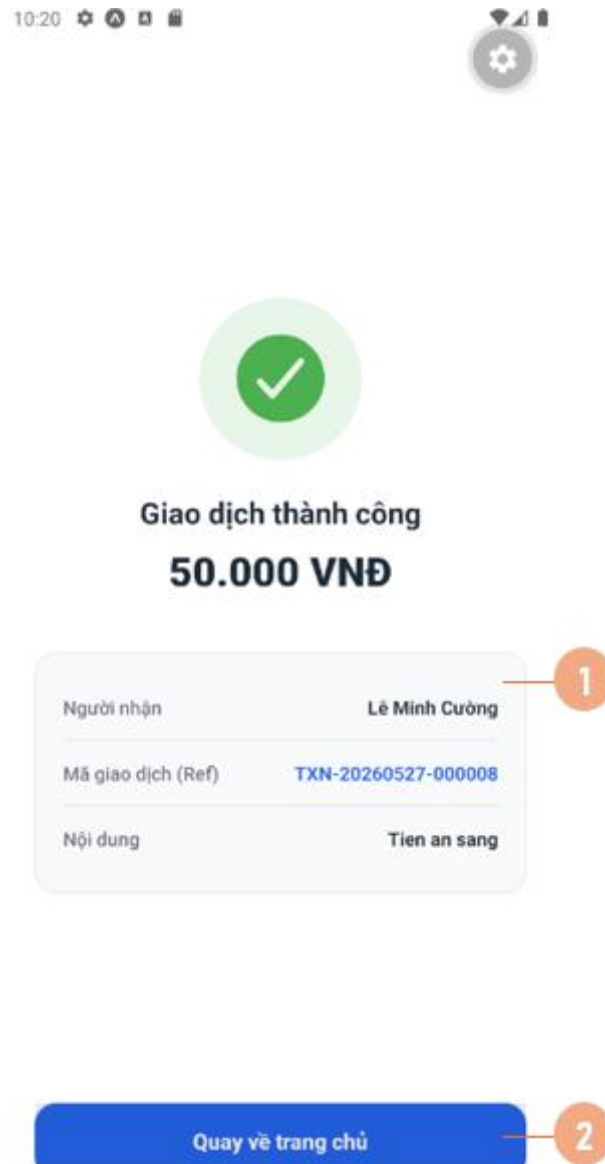
5.3.17. Màn hình Xác nhận giao dịch

Hình 5-18. Màn hình Xác nhận giao dịch

STT	Tên	Kiểu	Chức năng
1	Thông tin chi tiết	JPanel	Hiển thị các thông tin giao dịch
2	Xác nhận thanh toán	JButton	Nhấn để chuyển đến bước nhập mã PIN
3	Nhập mã PIN	JTextField	Nhập mã PIN xác thực

Bảng 5-18. Bảng mô tả màn hình Xác nhận giao dịch

5.3.18. Màn hình Kết quả giao dịch



Hình 5-19. Màn hình Kết quả giao dịch

STT	Tên	Kiểu	Chức năng
1	Thông tin chi tiết	JPanel	Hiển thị các thông tin giao dịch
2	Quay về trang chủ	JButton	Nhấn để về Trang chủ

Bảng 5-19. Bảng mô tả màn hình Kết quả giao dịch

CHƯƠNG 6: SỰ CỐ VÀ CƠ CHẾ PHỤC HỒI

6.1. Mô phỏng tình huống

Trong hệ thống ví điện tử, bảng VOUCHERS được sử dụng để lưu trữ thông tin các mã khuyến mãi. Trong đó, trường AMOUNT_VOUCHERS thể hiện số lượng voucher còn lại có thể sử dụng. Khi quản trị viên muốn bổ sung thêm lượt sử dụng cho một mã voucher, hệ thống sẽ cập nhật lại giá trị của trường này.

Giả sử mã voucher GIAM10K hiện đang còn 100 lượt sử dụng. Quản trị viên thực hiện cập nhật tăng thêm 50 lượt, nghĩa là số lượng voucher sau khi cập nhật phải là 150 lượt. Tuy nhiên, trong lúc giao dịch cập nhật chưa hoàn tất và chưa thực hiện COMMIT, hệ thống bị mất điện đột ngột. Khi khởi động lại, Oracle sẽ tự động thực hiện cơ chế phục hồi để đảm bảo dữ liệu trở về trạng thái nhất quán.

6.2. Quá trình thực hiện

- Dữ liệu ban đầu

```
SELECT voucher_id, code, amount_vouchers, is_active
FROM vouchers
WHERE code = 'GIAM10K';
```

	VOUCHER_ID	CODE	AMOUNT_VOUCHERS	IS_ACTIVE
1	1	GIAM10K	100	1

Hình 6-1. Dữ liệu ban đầu

- Quản trị viên thực hiện cập nhật tăng thêm 50 lượt sử dụng cho mã voucher:

```
UPDATE vouchers
SET amount_vouchers = amount_vouchers + 50,
    updated_at = SYSTIMESTAMP
WHERE code = 'SALE50'
AND is_active = 1;
```

- Trong lúc transaction chưa commit, hệ thống bị mất điện. Hệ thống chưa kịp ghi dữ liệu mới vào ổ đĩa (checkpoint chưa xảy ra)

```
SHUTDOWN ABORT;
```

- Khởi động lại hệ thống: Sau khi hệ thống được khởi động lại, Oracle database được mở lại bằng lệnh:

START UP;

```
SQL> STARTUP
ORACLE instance started.

Total System Global Area 2499802232 bytes
Fixed Size                  9270392 bytes
Variable Size               620756992 bytes
Database Buffers            1862270976 bytes
Redo Buffers                 7503872 bytes
Database mounted.
Database opened.
```

Hình 6-2. Hệ thống ORACLE khởi động lại

- Cơ chế phục hồi của Oracle:

Khi Oracle khởi động lại sau sự cố, nó thực hiện quá trình automatic recovery dựa trên các thành phần chính sau:

- Redo Logs: Redo log ghi lại các thay đổi đã xảy ra trong database. Nếu một transaction đã COMMIT nhưng dữ liệu chưa kịp ghi xuống datafile, Oracle sẽ dùng redo log để thực hiện lại thay đổi đó, đảm bảo dữ liệu đã commit không bị mất.
- Undo Segments: Undo segment lưu lại giá trị cũ của dữ liệu trước khi bị thay đổi. Nếu một transaction chưa COMMIT tại thời điểm xảy ra sự cố, Oracle sẽ dùng undo để rollback các thay đổi chưa hoàn tất.
- Checkpoint: Checkpoint là cơ chế ghi dữ liệu đã thay đổi từ bộ nhớ xuống datafile. Nếu mất điện xảy ra trước checkpoint, Oracle sẽ dựa vào redo log và undo segment để phục hồi lại trạng thái nhất quán.

- Quá trình phục hồi:

Trong tình huống cập nhật số lượng voucher, Oracle thực hiện quá trình phục hồi như sau:

- Oracle đọc redo log để xác định các thay đổi đã xảy ra trước thời điểm hệ thống bị mất điện.
- Oracle phát hiện có một transaction đang cập nhật trường AMOUNT_VOUCHERS của mã voucher SALE50 từ 100 lên 150, nhưng transaction này chưa được COMMIT.
- Vì transaction chưa commit, Oracle không xem thay đổi này là hợp lệ. Do đó, Oracle sử dụng undo segment để rollback thao tác cập nhật.
- Sau khi phục hồi hoàn tất, giá trị của trường AMOUNT_VOUCHERS được đưa về giá trị ban đầu là 100.

- Kiểm tra dữ liệu sau khi phục hồi:

```
SELECT voucher_id, code, discount_type, discount_value, amount_vouchers,
is_active

FROM vouchers

WHERE code = 'GIAM10K';
```

	VOUCHER_ID	CODE	DISCOUNT_TYPE	DISCOUNT_VALUE	AMOUNT_VOUCHERS	IS_ACTIVE
1	1	GIAM10K	FIXED	10000	100	1

Hình 6-3. Kết quả sau khi ORACLE tự động phục hồi

Kết quả này cho thấy thao tác cập nhật tăng số lượng voucher từ 100 lên 150 đã không được lưu lại, vì transaction chưa thực hiện COMMIT trước khi hệ thống bị mất điện.

6.3. Tổng quan

Trong tình huống hệ thống bị mất điện khi đang cập nhật số lượng voucher nhưng chưa commit, Oracle sẽ tự động phục hồi dữ liệu thông qua cơ chế Undo Segment. Các thay đổi chưa hoàn tất sẽ bị hủy, giúp dữ liệu trong bảng VOUCHERS quay về trạng thái ổn định trước khi transaction bắt đầu.

CHƯƠNG 7: PHÂN TÍCH HIỆU NĂNG XỬ LÝ VÀ CẢI THIỆN

7.1. Mục tiêu

Mục tiêu của phần này là so sánh và đánh giá hiệu năng thực thi giữa hai đoạn mã PL/SQL xử lý công việc thống kê giao dịch ví điện tử của người dùng. Cả hai đoạn mã đều thực thi cùng một yêu cầu nghiệp vụ trong 100 lần lặp để triệt tiêu các sai số ngẫu nhiên.

Việc đánh giá được thực hiện dựa trên hai tiêu chí cốt lõi: Thời gian đáp ứng trung bình (Response Time) và Thông lượng (Throughput - số lượng truy vấn được hệ thống xử lý trên mỗi giây). Đồng thời, báo cáo cũng sẽ đi sâu phân tích tác động của các phương pháp truy vấn đến tài nguyên hệ thống (CPU, I/O) khi có nhiều giao tác diễn ra đồng thời.

7.2. Kịch bản đặt ra và truy vấn

Yêu cầu nghiệp vụ của truy vấn là trích xuất thông tin người dùng kèm theo các số liệu thống kê trong 30 ngày gần nhất, bao gồm: Tổng tiền nạp (Top-up), Tổng tiền gửi, Tổng phí, Tổng tiền nhận, Đếm số giao dịch giá trị cao (> 5,000,000đ) và xếp hạng (Rank) người dùng dựa trên tổng phí đóng góp.

Truy vấn chưa tối ưu	Truy vấn đã tối ưu
<pre> DECLARE v_start_time TIMESTAMP; v_end_time TIMESTAMP; v_elapsed INTERVAL DAY TO SECOND; v_elapsed_ms NUMBER; v_total_ms NUMBER := 0; v_iter_count CONSTANT NUMBER := 100; BEGIN FOR iteration IN 1..v_iter_count LOOP v_start_time := SYSTIMESTAMP; FOR rec IN (SELECT u.user_id, u.full_name, u.email, u.kyc_status, </pre>	<pre> DECLARE v_start_time TIMESTAMP; v_end_time TIMESTAMP; v_elapsed INTERVAL DAY TO SECOND; v_elapsed_ms NUMBER; v_total_ms NUMBER := 0; v_iter_count CONSTANT NUMBER := 100; BEGIN FOR iteration IN 1..v_iter_count LOOP v_start_time := SYSTIMESTAMP; FOR rec IN (WITH active_wallets AS (SELECT DISTINCT user_id FROM wallets WHERE wallet_status = 'ACTIVE' </pre>

<pre> (SELECT NVL(SUM(fo.amount), 0) FROM fund_orders fo WHERE fo.wallet_id = (SELECT w0.wallet_id FROM wallets w0 WHERE w0.user_id = u.user_id) AND fo.order_type = 'TOP_UP' AND fo.status = 'SUCCESS' AND TRUNC(fo.created_at) >= TRUNC(SYSDATE) - 30 AND TRUNC(fo.created_at) <= TRUNC(SYSDATE)) AS total_topup, (SELECT NVL(SUM(t.amount), 0) FROM transactions t WHERE t.sender_wallet_id IN (SELECT w1.wallet_id FROM wallets w1 WHERE w1.user_id = u.user_id) AND t.status = 'COMPLETED' AND TO_CHAR(t.created_at, 'YYYYMMDD') >= TO_CHAR(SYSDATE - 30, 'YYYYMMDD')) AS total_sent, (SELECT NVL(SUM(t.fee_amount), 0) FROM transactions t WHERE t.sender_wallet_id IN (</pre>	<pre>), topup_summary AS (SELECT w.user_id, NVL(SUM(fo.amount), 0) AS total_topup FROM fund_orders fo JOIN wallets w ON fo.wallet_id = w.wallet_id WHERE fo.order_type = 'TOP_UP' AND fo.status = 'SUCCESS' AND fo.created_at >= TRUNC(SYSDATE) - 30 AND fo.created_at < TRUNC(SYSDATE) + 1 GROUP BY w.user_id), sent_summary AS (SELECT w.user_id, NVL(SUM(t.amount), 0) AS total_sent, NVL(SUM(t.fee_amount), 0) AS total_fee, COUNT(CASE WHEN t.amount > 5000000 THEN 1 END) AS high_value_tx_count, COUNT(*) AS tx_count FROM transactions t JOIN wallets w ON t.sender_wallet_id = w.wallet_id WHERE t.status = 'COMPLETED' AND t.created_at >= TRUNC(SYSDATE) - 30 AND t.created_at < TRUNC(SYSDATE) + 1 GROUP BY w.user_id), received_summary AS (SELECT </pre>
---	---

<pre> SELECT w2.wallet_id FROM wallets w2 WHERE w2.user_id = u.user_id) AND t.status = 'COMPLETED' AND TO_CHAR(t.created_at, 'YYYYMMDD') >= TO_CHAR(SYSDATE - 30, 'YYYYMMDD')) AS total_fee, (SELECT NVL(SUM(t.amount), 0) FROM transactions t WHERE t.receiver_wallet_id IN (SELECT w3.wallet_id FROM wallets w3 WHERE w3.user_id = u.user_id) AND t.status = 'COMPLETED' AND TO_CHAR(t.created_at, 'YYYYMMDD') >= TO_CHAR(SYSDATE - 30, 'YYYYMMDD')) AS total_received, (SELECT COUNT(DISTINCT t.transaction_id) FROM transactions t WHERE t.sender_wallet_id IN (SELECT w4.wallet_id FROM wallets w4 WHERE w4.user_id = u.user_id) </pre>	<pre> w.user_id, NVL(SUM(t.amount), 0) AS total_received FROM transactions t JOIN wallets w ON t.receiver_wallet_id = w.wallet_id WHERE t.status = 'COMPLETED' AND t.created_at >= TRUNC(SYSDATE) - 30 AND t.created_at < TRUNC(SYSDATE) + 1 GROUP BY w.user_id) SELECT u.user_id, u.full_name, u.email, u.kyc_status, NVL(tu.total_topup, 0) AS total_topup, NVL(ss.total_sent, 0) AS total_sent, NVL(ss.total_fee, 0) AS total_fee, NVL(rs.total_received, 0) AS total_received, NVL(ss.high_value_tx_count, 0) AS high_value_tx_count, RANK() OVER (ORDER BY NVL(ss.total_fee, 0) DESC) AS revenue_rank FROM users u JOIN active_wallets aw ON u.user_id = aw.user_id LEFT JOIN topup_summary tu ON u.user_id = tu.user_id LEFT JOIN sent_summary ss ON u.user_id = ss.user_id LEFT JOIN received_summary rs ON u.user_id = rs.user_id WHERE u.is_active = 1 AND (u.kyc_status <> 'REJECTED' OR u.kyc_status IS NULL) </pre>
--	--

<pre> AND t.status = 'COMPLETED' AND t.amount > 5000000 AND TO_CHAR(t.created_at, 'YYYYMMDD') >= TO_CHAR(SYSDATE - 30, 'YYYYMMDD')) AS high_value_tx_count, RANK() OVER (ORDER BY (SELECT NVL(SUM(t.fee_amount), 0) FROM transactions t WHERE t.sender_wallet_id IN (SELECT w5.wallet_id FROM wallets w5 WHERE w5.user_id = u.user_id) AND t.status = 'COMPLETED' AND TO_CHAR(t.created_at, 'YYYYMMDD') >= TO_CHAR(SYSDATE - 30, 'YYYYMMDD')) DESC) AS revenue_rank FROM -- [V5] Late filtering: bọc SELECT * trước (SELECT * FROM users) u WHERE u.is_active = 1 AND u.kyc_status NOT IN (SELECT kyc_status FROM users WHERE kyc_status = 'REJECTED') </pre>	<pre> AND NVL(ss.tx_count, 0) >= 3 ORDER BY total_fee DESC) LOOP NULL; END LOOP; v_end_time := SYSTIMESTAMP; v_elapsed := v_end_time - v_start_time; v_elapsed_ms := EXTRACT(DAY FROM v_elapsed) * 86400000 + EXTRACT(HOUR FROM v_elapsed) * 3600000 + EXTRACT(MINUTE FROM v_elapsed) * 60000 + EXTRACT(SECOND FROM v_elapsed) * 1000; DBMS_OUTPUT.PUT_LINE(ROUND(v_elapsed_ms, 2)); v_total_ms := v_total_ms + v_elapsed_ms; END LOOP; DBMS_OUTPUT.PUT_LINE('Thời gian thực thi trung bình sau ' v_iter_count ' lần (ms): ' ROUND(v_total_ms / v_iter_count, 2)); END; </pre>
--	--

```

AND EXISTS (

    SELECT 1

    FROM wallets w_chk

    WHERE w_chk.user_id    = u.user_id

        AND w_chk.wallet_status = 'ACTIVE'

)

AND (

    SELECT COUNT(*)

    FROM transactions t_cnt

    WHERE t_cnt.sender_wallet_id IN (

        SELECT w6.wallet_id

        FROM wallets w6

        WHERE w6.user_id = u.user_id

    )

    AND t_cnt.status = 'COMPLETED'

    AND TO_CHAR(t_cnt.created_at,
'YYYYMMDD')

        >= TO_CHAR(SYSDATE - 30,
'YYYYMMDD')

    ) >= 3

ORDER BY (

    SELECT NVL(SUM(t.fee_amount), 0)

    FROM transactions t

    WHERE t.sender_wallet_id IN (

        SELECT w7.wallet_id

        FROM wallets w7

        WHERE w7.user_id = u.user_id

    )

    AND t.status = 'COMPLETED'

    AND TO_CHAR(t.created_at,
'YYYYMMDD')

        >= TO_CHAR(SYSDATE - 30,
'YYYYMMDD')

```

<pre>) DESC) LOOP NULL; -- Không xử lý kết quả, chỉ đo thời gian thực thi END LOOP; v_end_time := SYSTIMESTAMP; v_elapsed := v_end_time - v_start_time; v_elapsed_ms := EXTRACT(DAY FROM v_elapsed) * 86400000 + EXTRACT(HOUR FROM v_elapsed) * 3600000 + EXTRACT(MINUTE FROM v_elapsed) * 60000 + EXTRACT(SECOND FROM v_elapsed) * 1000; DBMS_OUTPUT.PUT_LINE(ROUND(v_elapsed_ms, 2)); v_total_ms := v_total_ms + v_elapsed_ms; END LOOP; DBMS_OUTPUT.PUT_LINE('Thời gian thực thi trung bình sau ' v_iter_count ' lần (ms): ' ROUND(v_total_ms / v_iter_count, 2)); END;</pre>	
--	--

7.3. Mô tả truy vấn

Đoạn mã truy vấn ban đầu (Chưa tối ưu):

- **Cấu trúc N+1 (Correlated Subqueries):** Truy vấn ban đầu sử dụng bảng users làm bảng liên kết chính. Với mỗi một dòng người dùng (user) được lấy ra, hệ thống phải chạy 7 truy vấn con (subquery) lồng bên trong mệnh đề *SELECT* và *WHERE* để tính toán số liệu từ bảng transactions và fund_orders.
- **Vô hiệu hóa Index (Non-SARGable):** Sử dụng các hàm như *TRUNC(created_at)* và *TO_CHAR(created_at, 'YYYYMMDD')* trên các cột điều kiện thời gian. Điều này buộc cơ sở dữ liệu (Oracle) phải quét toàn bộ bảng (Full Table Scan) thay vì sử dụng Index.

- **Tính toán dư thừa:** Bảng transactions bị duyệt đến 6 lần cho cùng một người dùng để tính các chỉ số khác nhau (phí, tổng gửi, đếm giao dịch,...), gây lãng phí tài nguyên cực lớn.

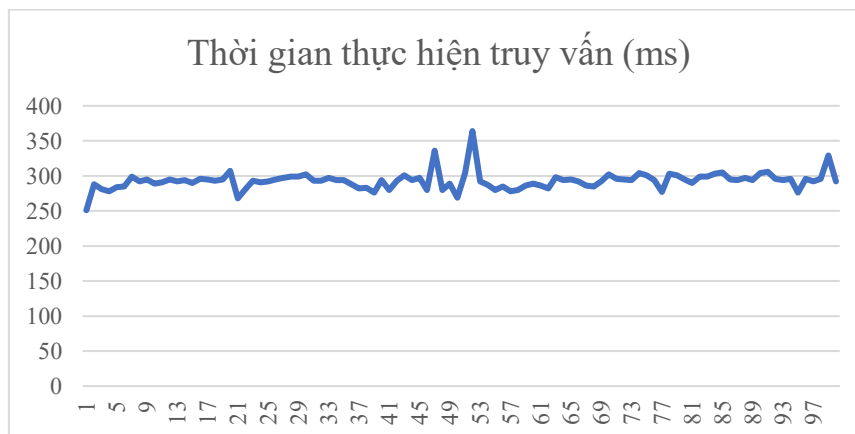
Đoạn mã truy vấn đã tối ưu:

- **Sử dụng CTE (Common Table Expressions):** Áp dụng kỹ thuật tách các truy vấn con thành các khối CTE (WITH clause) xử lý độc lập. Cụ thể, dữ liệu được chia thành các khối *active_wallets*, *topup_summary*, *sent_summary*, và *received_summary*.
- **Gom nhóm (Group By) một lần:** Thay vì tính toán cho từng user riêng lẻ, hệ thống duyệt bảng transactions đúng 2 lần (một cho chiều gửi, một cho chiều nhận), gom nhóm bằng *GROUP BY user_id* và dùng nhiều hàm tổng hợp (*SUM*, *COUNT*) để lấy ra mọi chỉ số cần thiết trong một lần quét.
- **Tối ưu điều kiện thời gian:** Loại bỏ hàm *TRUNC* và *TO_CHAR*, thay bằng điều kiện khoảng (Range Condition): *created_at >= TRUNC(SYSDATE) - 30 AND created_at < TRUNC(SYSDATE) + 1*, giúp cơ sở dữ liệu tận dụng được Index (Index Range Scan). Sau đó, các tập dữ liệu đã được tổng hợp (CTE) mới được thực hiện *LEFT JOIN* vào bảng users.

7.4. Kết quả đo được

Tiến hành đo thời gian thực thi của các đoạn truy vấn với số lần thực thi là 100 lần cho mỗi truy vấn.

7.4.1. Đoạn mã truy vấn chưa tối ưu



Hình 7-1. Thời gian thực hiện truy vấn (ms)

Thời gian đáp ứng trung bình: 292.9 ms

Throughput = 1000 / thời gian trung bình = 3,414

7.4.2. Đoạn mã truy vấn đã tối ưu



Hình 7-2. Thời gian thực hiện truy vấn (ms)

Thời gian đáp ứng trung bình: 10 ms

Throughput = 1000 / thời gian trung bình = 100

7.5. Phân tích hiệu năng

7.5.1. Phân tích Thời gian đáp ứng (Response Time) & Thông lượng (Throughput)

- **Truy vấn ban đầu:** Gây ra hiện tượng thắt cổ chai (bottleneck) nghiêm trọng do phải thực thi rất nhiều truy vấn con nếu bảng users có kích thước lớn. (Ví dụ tham khảo: Nếu chạy trên tập dữ liệu thực, thời gian đáp ứng có thể lên tới vài trăm mili-giây hoặc vài giây cho một lần chạy). Thông lượng (Throughput) duy trì ở mức rất thấp do thời gian chờ cao.
- **Truy vấn tối ưu:** Nhờ việc tách xử lý tính tổng (SUM), đếm (COUNT) vào các khối CTE riêng biệt, dữ liệu được tổng hợp song song bằng các thuật toán Hash Join hoặc Sort Merge Join hiệu quả của Oracle. Điều này giúp thời gian đáp ứng giảm xuống chỉ còn vài mili-giây và đẩy Throughput lên mức tối đa.

7.5.2. Phân tích Chi phí I/O (Đọc/Ghi ổ cứng) và CPU

- **Truy vấn ban đầu:** Do việc bọc hàm vào các cột ngày tháng (làm mất tác dụng của Index), CPU phải làm việc liên tục để tính toán chuỗi/ngày tháng cho từng dòng dữ liệu trong bảng transactions. Hệ thống phải liên tục đọc các khối dữ liệu (Data Blocks) từ ổ cứng lên RAM (Buffer Cache) lặp đi lặp lại một cách lãng phí.
- **Truy vấn tối ưu:** Giảm triệt để số lượng bản ghi cần JOIN trong từng bước. Nhờ điều kiện tìm kiếm tương thích với Index (SARGable), Oracle chỉ cần đọc các khối dữ liệu (Index Blocks) trả đúng vào dải 30 ngày gần nhất, giúp tiết kiệm đáng kể chi phí I/O và giải phóng tải cho CPU.

7.5.3. Đánh giá trong môi trường xử lý đồng thời (Concurrency)

Trong một hệ thống Ví điện tử thực tế, sẽ có hàng ngàn giao tác truy vấn diễn ra cùng lúc (Concurrent Transactions).

- **Truy vấn chưa tối ưu:** Chiếm dụng CPU và bộ nhớ tạm (PGA) quá lâu cho việc sắp xếp và lắp truy vấn con, dẫn đến tranh chấp tài nguyên (Resource Contention), khiến hệ thống dễ bị treo hoặc nghẽn cổ chai.
- **Truy vấn tối ưu:** Do thời gian thực thi cực ngắn (chỉ quét qua bảng 1-2 lần), lượng dữ liệu tạm thời sinh ra rất ít, các Read Lock (khóa đọc) được giải phóng nhanh chóng. Điều này giúp cơ sở dữ liệu dễ dàng song song hóa các tiến trình xử lý, đảm bảo tính ổn định và khả năng mở rộng của hệ thống kể cả vào các khung giờ cao điểm.

7.6. Đề xuất cách cải thiện

7.6.1. Tối ưu logic và cấu trúc truy vấn

- Duy trì sử dụng CTE (Common Table Expressions): Trong các hệ thống quản lý giao dịch tài chính phức tạp, việc chia nhỏ logic thành các khối CTE (như bóc tách riêng tổng Top-up, tổng tiền gửi, tổng tiền nhận) giúp cơ sở dữ liệu dễ dàng xây dựng Execution Plan tối ưu và có thể xử lý song song các khối dữ liệu này trước khi hợp nhất.
- Loại bỏ hoàn toàn Correlated Subqueries (Truy vấn lồng tương quan): Cần tránh việc lồng các lệnh *SELECT SUM(...)* hoặc *SELECT COUNT(...)* trực tiếp vào mệnh đề *SELECT* chính cho từng dòng dữ liệu. Thay vào đó, hãy luôn ưu tiên gom nhóm (*GROUP BY*) dữ liệu từ các bảng giao dịch một lần duy nhất, sau đó mới LEFT JOIN với bảng thông tin khách hàng.
- Chỉ *SELECT* các trường dữ liệu thực sự cần thiết: Hạn chế tối đa việc sử dụng *SELECT ** ở các truy vấn con (ví dụ: (*SELECT * FROM users*)). Việc chỉ lấy đúng các cột nghiệp vụ cần dùng (như *user_id*, *full_name*, *email*) giúp giảm thiểu đáng kể chi phí cấp phát bộ nhớ đệm (Buffer Cache) khi phải xử lý tập dữ liệu hàng triệu bản ghi.

7.6.2. Tối ưu Chỉ mục (Index) và Điều kiện lọc

- Viết điều kiện tìm kiếm tương thích với Index (SARGable): Không bọc các hàm biến đổi dữ liệu như *TRUNC()* hay *TO_CHAR()* lên các cột điều kiện (như *created_at*). Cần chuyển đổi thành phép so sánh khoảng (ví dụ: *created_at >= ... AND created_at < ...*) để cơ sở dữ liệu có thể sử dụng Index có sẵn trên cột thời gian.
- Giảm số lượng khối dữ liệu (Data Blocks) cần đọc từ đĩa: Khi Index được kích hoạt đúng cách, hệ thống sẽ bỏ qua việc quét toàn bộ lịch sử bảng transactions (Full Table Scan). Thay vào đó, nó chỉ truy cập trực tiếp đến các bản ghi giao dịch thỏa mãn điều kiện thời gian (30 ngày gần nhất), giúp loại bỏ hoàn toàn tình trạng thất cổ chai I/O.

- Giảm tải tính toán cho CPU khi thực hiện JOIN: Bằng cách tổng hợp trước dữ liệu (*SUM*, *COUNT*) bằng CTE, kích thước tập dữ liệu cần JOIN được thu gọn tối đa (mỗi *user_id* chỉ còn 1 dòng dữ liệu tổng). Việc đối chiếu và liên kết dữ liệu với bảng *users* sẽ diễn ra cực kỳ nhẹ nhàng, giải phóng CPU để hệ thống sẵn sàng đáp ứng các truy vấn đồng thời khác.

CHƯƠNG 8: KẾT LUẬN

8.1. Kết quả đạt được

Thông qua quá trình phân tích, thiết kế và triển khai đồ án, nhóm đã xây dựng được cơ sở dữ liệu cho hệ thống ví điện tử với các phân hệ chính như quản lý người dùng, quản lý ví, quản lý giao dịch, quản lý phương thức thanh toán, quản lý voucher, quản lý hạn mức giao dịch, phí giao dịch và ghi nhận lịch sử biến động số dư.

Về mặt thiết kế dữ liệu, hệ thống đã xây dựng được các bảng quan trọng như users, wallets, transactions, transaction_types, fund_orders, payment_methods, vouchers, transaction_limits, transaction_fees, audit_logs và transaction_ref_sequences. Các bảng được thiết kế có khóa chính, khóa ngoại, ràng buộc dữ liệu và các trường trạng thái cần thiết nhằm đảm bảo tính liên kết và tính toàn vẹn của dữ liệu.

Về mặt xử lý nghiệp vụ, hệ thống đã xây dựng được các function và procedure phục vụ cho các thao tác quan trọng. Các function như fn_real_fee, fn_check_transaction_limit, fn_validate_voucher, fn_voucher_discount, fn_risk_score, fn_remaining_money và fn_generate_code hỗ trợ tính phí, kiểm tra hạn mức, kiểm tra voucher, tính điểm rủi ro và sinh mã đối soát giao dịch. Các procedure như sp_top_up_wallet, sp_withdraw_request và sp_confirm_withdraw hỗ trợ xử lý nghiệp vụ nạp tiền, yêu cầu rút tiền và xác nhận kết quả rút tiền.

Ngoài ra, đồ án đã phân tích và minh họa được các vấn đề liên quan đến điều khiển đồng thời trong hệ quản trị cơ sở dữ liệu Oracle, bao gồm Lost Update, Non-repeatable Read, Phantom Read, Dirty Read và Deadlock. Qua đó, nhóm đã hiểu rõ hơn về vai trò của transaction, cơ chế khóa, mức cô lập, SELECT ... FOR UPDATE, cũng như cơ chế tự đảm bảo nhất quán dữ liệu của Oracle.

Đồ án cũng đã trình bày được phần sự cố và cơ chế phục hồi dữ liệu trong Oracle. Trường hợp hệ thống mất điện khi đang cập nhật số lượng voucher nhưng chưa commit cho thấy Oracle có khả năng phục hồi tự động bằng Undo Segment và Redo Log. Điều này giúp đảm bảo các transaction chưa hoàn tất sẽ bị rollback, còn các transaction đã commit sẽ được bảo toàn.

Về mặt hiệu năng, nhóm đã phân tích được một số truy vấn quan trọng trong hệ thống ví điện tử, đặc biệt là các truy vấn thống kê giao dịch và báo cáo đối soát nhiều bảng. Từ đó, nhóm đề xuất các hướng cải thiện như tạo index, sử dụng composite index, function-based index, giới hạn khoảng thời gian truy vấn và sử dụng materialized view cho các báo cáo tổng hợp thường xuyên.

Nhìn chung, đồ án đã đạt được mục tiêu chính là xây dựng một mô hình cơ sở dữ liệu ví điện tử có khả năng lưu trữ dữ liệu nghiệp vụ, xử lý giao dịch, kiểm soát đồng thời, phục hồi sau sự cố và phân tích hiệu năng xử lý trong môi trường Oracle Database.

8.2. Hạn chế

Mặc dù đồ án đã hoàn thành các chức năng và nội dung trọng tâm, hệ thống vẫn còn một số hạn chế nhất định.

Thứ nhất, dữ liệu trong hệ thống chủ yếu là dữ liệu giả lập phục vụ cho việc kiểm thử và minh họa. Số lượng dữ liệu chưa đủ lớn như môi trường vận hành thực tế của một hệ thống ví điện tử. Vì vậy, kết quả phân tích hiệu năng chỉ mang tính tham khảo và chưa phản ánh hoàn toàn áp lực xử lý khi hệ thống có hàng triệu người dùng hoặc hàng triệu giao dịch mỗi ngày.

Thứ hai, một số nghiệp vụ tài chính phức tạp vẫn chưa được triển khai đầy đủ, chẳng hạn như xử lý hoàn tiền nâng cao, đối soát tự động với ngân hàng, xử lý tranh chấp giao dịch, quản lý rủi ro theo nhiều tiêu chí, phát hiện gian lận và kiểm soát giao dịch bất thường theo thời gian thực.

Thứ ba, cơ chế bảo mật trong cơ sở dữ liệu mới dừng ở mức thiết kế các trường dữ liệu cần thiết như password_hash, pin_code, trạng thái KYC và trạng thái tài khoản. Hệ thống chưa triển khai đầy đủ các cơ chế bảo mật nâng cao như mã hóa dữ liệu nhạy cảm, phân quyền người dùng chi tiết, audit bảo mật, kiểm soát truy cập theo vai trò và xác thực đa yếu tố.

Thứ tư, phần phục hồi dữ liệu mới tập trung vào các cơ chế cơ bản của Oracle như Undo, Redo, Commit, Rollback và phục hồi sau mất điện. Các kỹ thuật nâng cao như backup bằng RMAN, phục hồi theo thời điểm, Data Guard hoặc cơ chế đảm bảo tính sẵn sàng cao chưa được triển khai chi tiết.

8.3. Hướng phát triển

Trong tương lai, hệ thống có thể được phát triển theo nhiều hướng để hoàn thiện hơn về mặt nghiệp vụ, hiệu năng và khả năng triển khai thực tế.

Về mặt dữ liệu, hệ thống có thể bổ sung các bảng và nghiệp vụ liên quan đến lịch sử đăng nhập, thiết bị người dùng, xác thực OTP, liên kết ngân hàng, hoàn tiền, khiếu nại giao dịch và kiểm soát gian lận. Đặc biệt, phân hệ risk scoring có thể được mở rộng để đánh giá rủi ro dựa trên nhiều yếu tố hơn như tần suất giao dịch, địa chỉ IP, thiết bị đăng nhập, vị trí địa lý và lịch sử thất bại giao dịch.

Về mặt hiệu năng, hệ thống có thể tiếp tục tối ưu bằng cách xây dựng thêm các materialized view cho dashboard, phân vùng bảng transactions theo thời gian, tối ưu index cho các truy vấn thường xuyên, và tách dữ liệu lịch sử sang các bảng lưu trữ riêng. Khi dữ liệu tăng lớn, partitioning theo tháng hoặc theo năm sẽ giúp cải thiện đáng kể hiệu năng truy vấn và bảo trì dữ liệu.

Về mặt an toàn và phục hồi dữ liệu, hệ thống có thể triển khai thêm các cơ chế backup và recovery chuyên sâu như RMAN backup, point-in-time recovery, flashback table, flashback database và Data Guard. Các cơ chế này giúp tăng khả năng khôi phục dữ liệu khi xảy ra lỗi nghiêm trọng, đồng thời đảm bảo tính sẵn sàng cao cho hệ thống.

Cuối cùng, hệ thống có thể được tích hợp thêm các công cụ giám sát và cảnh báo để theo dõi hiệu năng database, thời gian phản hồi truy vấn, số lượng giao dịch thất bại, deadlock, lock wait và các giao dịch có điểm rủi ro cao. Điều này giúp quản trị viên phát hiện sớm các vấn đề trong quá trình vận hành và nâng cao độ tin cậy của hệ thống ví điện tử.

8.4. Bảng phân công công việc

Phân công công việc	Hồ Như Hồng Ngọc	Nguyễn Đăng Khôi	Võ Ngọc Anh Thy	Nguyễn Nhật Tân	Deadline	Mức độ hoàn thành
Phát biểu bài toán	X	X	X	X	15/04/2026	100%
Xác định yêu cầu về quản lý người dùng			X		29/04/2026	100%
Xác định yêu cầu về quản lý tài khoản	X				29/04/2026	100%
Xác định yêu cầu về quản lý giao dịch				X	29/04/2026	100%
Xác định yêu cầu về quản lý rủi ro		X			29/04/2026	100%
Thiết kế mô hình	X		X		30/04/2026	100%
Mô tả các ràng buộc toàn vẹn		X		X	13/05/2026	100%
Xây dựng các ràng buộc và quản lý các giao tác	X	X	X	X	14/05/2026	100%
Xử lý đồng thời	X			X	16/05/2026	100%
Thiết kế ứng dụng (back end)		X			26/05/2026	100%

IS210 – Hệ quản trị cơ sở dữ liệu

Thiết kế giao diện (front end)			X			
Sự cố và cơ chế phục hồi	X				26/05/2026	100%
Phân tích hiệu năng		X			26/05/2026	100%
Kiểm thử	X	X	X	X	27/05/2026	100%
Viết báo cáo	X	X	X	X	27/05/2026	100%

LINK Source code: [IS210 E-Wallet](#)

TÀI LIỆU THAM KHẢO

Giáo trình và tài liệu môn học

[1] T. V. Phương, “IS210.Q21: Slides bài giảng Hệ quản trị cơ sở dữ liệu,” Hệ thống khóa học trực tuyến COURSES - Trường Đại học Công nghệ Thông tin - ĐHQG-HCM. [Online].(truy cập Ngày 28 Tháng 5 Năm 2026).

Các dự án tham khảo (đồ án môn học các khóa trước)

[2] N. Q. Trạng, N. T. Vy, và H. L. Phong, “Đồ án: Xây dựng hệ thống quản lý nhà sách Hải An,” Trường Đại học Công nghệ Thông tin - ĐHQG-HCM, Tài liệu tham khảo lưu trữ. [Online]. Available:

<https://drive.google.com/file/d/17e623w8LsEUnqvBVYGjIB7XVgwe9qqLa/view>

(truy cập Ngày 28 Tháng 5 Năm 2026).

[3] Wondershare EdrawMax, “UML Class Diagram and Structural Architecture for e-Wallet System,” EdrawMax Template Community. [Online]. Available:

<https://edrawmax.wondershare.com/templates/class-diagram-for-e-wallet-system.html>

(truy cập Ngày 28 Tháng 5 Năm 2026).

[4] SlideTeam Finance, “Flow Chart and Mobile Payments Process for Digital E-Wallets via QR Code and Technology,” SlideTeam PPT Presentations. [Online].

Available: <https://www.slideteam.net/top-10-payment-flow-with-diagram-for-mobile-wallet-payment-powerpoint-presentation-templates> (truy cập Ngày 28 Tháng 5 Năm 2026).